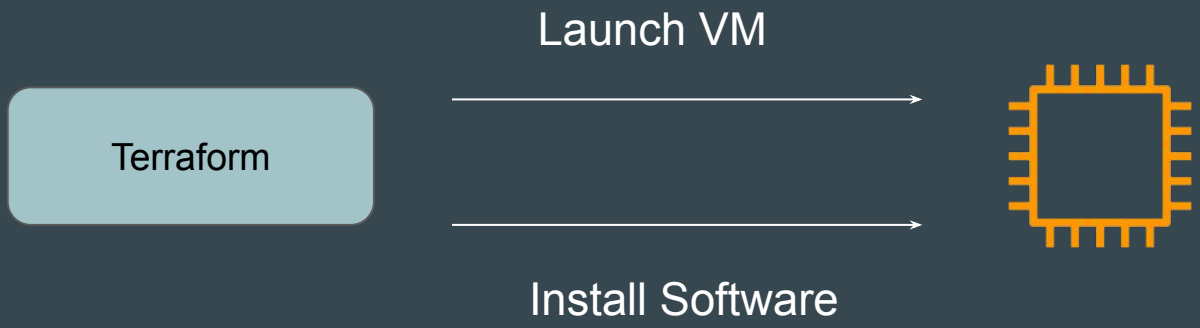
Type 2 - remote-exec provisioner

remote-exec provisioners allow to invoke scripts or run commands directly on the remote server.

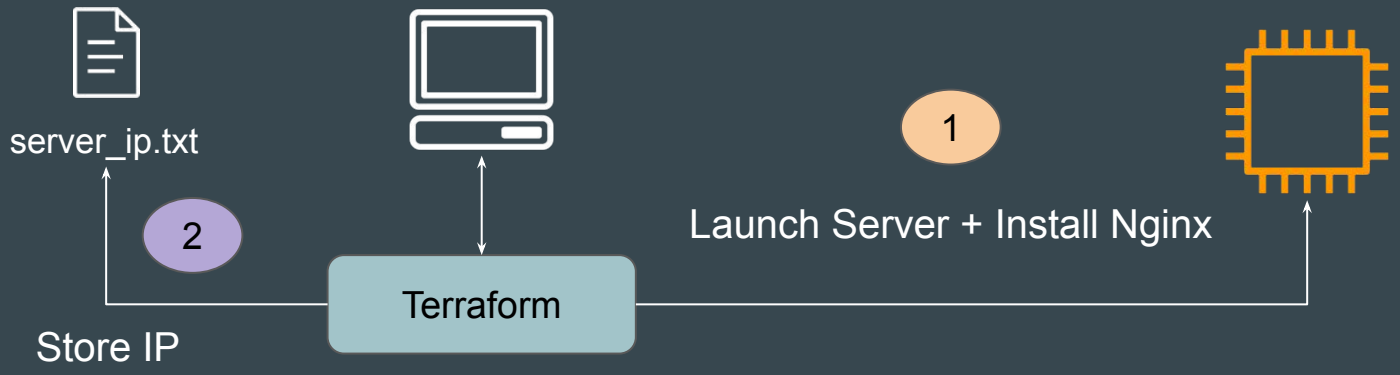
Example: After EC2 is launched, install “apache” software



Today's Demo

For today's demo, the Terraform code will run two provisioners.

1. Remote-Exec will install Nginx software in EC2 to have basic website.
2. Local-Exec will fetch the Public IP of EC2 and store it in a new file.



Format of Provisioners

1 - Defining Provisioners

Provisioners are defined inside a specific resource.

```
resource "aws_instance" "myec2" {
    ami = "ami-001843b876406202a"
    instance_type = "t2.micro"

    <provisioners-need-to-be-defined-inside-resource>

}
```

2 - Defining provisioner

Provisioners are defined by “provisioner” followed by type of provisioner

```
resource "aws_instance" "myec2" {
   ami = "ami-001843b876406202a"
   instance_type = "t2.micro"

   provisioner "local-exec" {}

   provisioner "remote-exec" {}

}
```

3 - Local Provisioner Approach

For local provisioners, we have to specify command that needs to be run locally

```
resource "aws_instance" "example" {
   ami = "ami-001843b876406202a"
   instance_type = "t2.micro"

   provisioner "local-exec" {
    command = "echo Server has been created through Terraform"
   }
}
```

4 - Remote Exec Provisioner Approach

Since commands are executed on remote-server, we have to provide way for Terraform to connect to remote server.

Details to connect to the Server



Commands to Run on the Server



```
resource "aws_instance" "myec2" {
   ami = "ami-001843b876406202a"
   instance_type = "t2.micro"

 connection {
     type     = "ssh"
     user     = "ec2-user"
     private_key = file("./terraform-key.pem")
     host     = self.public_ip
    }

 provisioner "remote-exec" {
   inline = [
     "sudo yum install -y nginx",
     "sudo systemctl start nginx"
   ]
 }
}
```

Points to Note - Provisioners

Provisioners are Defined inside Resource Block

It is not necessary to define a aws_instance resource block for provisioner to run.

They can be defined inside other resource types as well.

```
resource "aws_iam_user" "lb" {
  name = "loadbalancer"

  provisioner "local-exec" {
    command = "echo local-exec provisioner is starting"
  }
}
```

Multiple Provisioners Blocks for Single Resource

We can define multiple provisioners block in a single resource block.

```
resource "aws_iam_user" "lb" {
  name = "loadbalancer"

  provisioner "local-exec" {
    command = "echo local-exec provisioner is starting"
  }

    provisioner "local-exec" {
    command = "echo local-exec-2 provisioner is starting"
  }

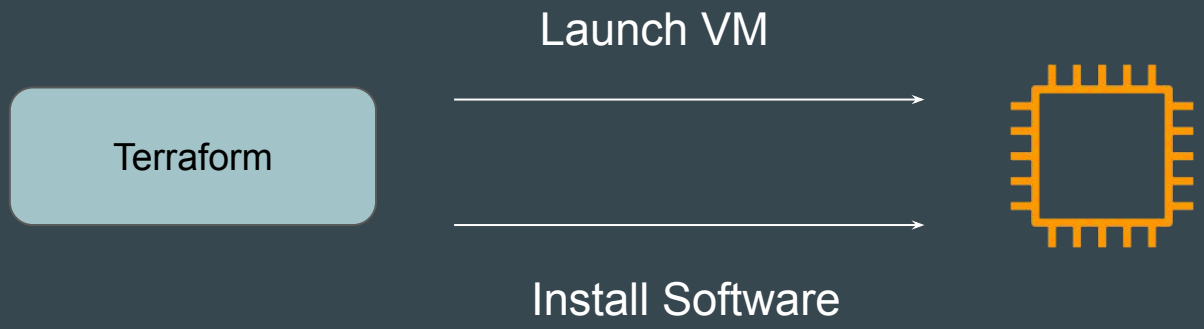
}
```

Creation-Time & Destroy-Time Provisioners

Basic of Creation-Time Provisioners

By default, provisioners run when the resource they are defined within is created.

Creation-time provisioners are only run during creation, not during updating or any other lifecycle.



Destroy-Time Provisioner

Destroy provisioners are run before the resource is destroyed.

Example:

Remove and De-Link Anti-Virus software before EC2 gets terminated.

Define destroy-time provisioner

```
resource "aws_instance" "web" {
  # ...

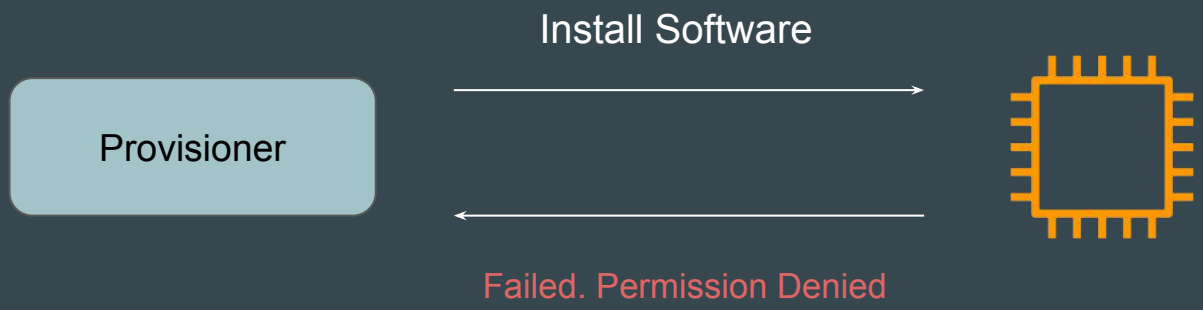
  provisioner "local-exec" {
    when    = destroy
    command = "echo 'Destroy-time provisioner'"
  }
}
```

Tainting Resource in Creation-Time Provisioners

If a creation-time provisioner fails, the resource is marked as tainted.

A tainted resource will be planned for destruction and recreation upon the next terraform apply.

Terraform does this because a failed provisioner can leave a resource in a semi-configured state.



Reference Screenshot - Resource Marked as Tainted

Following screenshot shows state file that has marked the resource as “tainted” because the provisioner had failed.

```
Error: local-exec provisioner error

  with aws_iam_user.lb,
  on test-prov.tf line 5, in resource "aws_iam_user" "lb":
   5:   provisioner "local-exec" {

Error running command 'echo1 local-exec provisioner is starting': exit status 1.
internal or external command,
operable program or batch file.
```



Failure Behaviour in Provisioners

Understanding the Challenge

By default, provisioners that fail will also cause the terraform apply itself to fail.

This will lead to resource being tainted and we have to re-create the resource.

```
  # aws_iam_user.lb will be created
  + resource "aws_iam_user" "lb" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "loadbalancer2"
      + path          = "/"
      + tags_all      = (known after apply)
      + unique_id     = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
aws_iam_user.lb: Creating...
aws_iam_user.lb: Provisioning with 'local-exec'...
aws_iam_user.lb (local-exec): Executing: ["cmd" "/C" "echo1 local-exec provisioner is starting"]
aws_iam_user.lb (local-exec): 'echo1' is not recognized as an internal or external command,
aws_iam_user.lb (local-exec): operable program or batch file.

Error: local-exec provisioner error

  with aws_iam_user.lb,
  on test-prov.tf line 5, in resource "aws_iam_user" "lb":
   5:   provisioner "local-exec" {

Error running command 'echo1 local-exec provisioner is starting': exit status 1. Output: 'echo1' is not recognized as an
internal or external command,
operable program or batch file.
```

Basics of On Failure Setting

The on_failure setting can be used to change the default behaviour.

Allowed Values	Description
continue	Ignore the error and continue with creation or destruction.
fail	Raise an error and stop applying (the default behavior). If this is a creation provisioner, taint the resource.

Reference Code - On-Failure

Following screenshot shows a reference code where on_failure is set to continue.

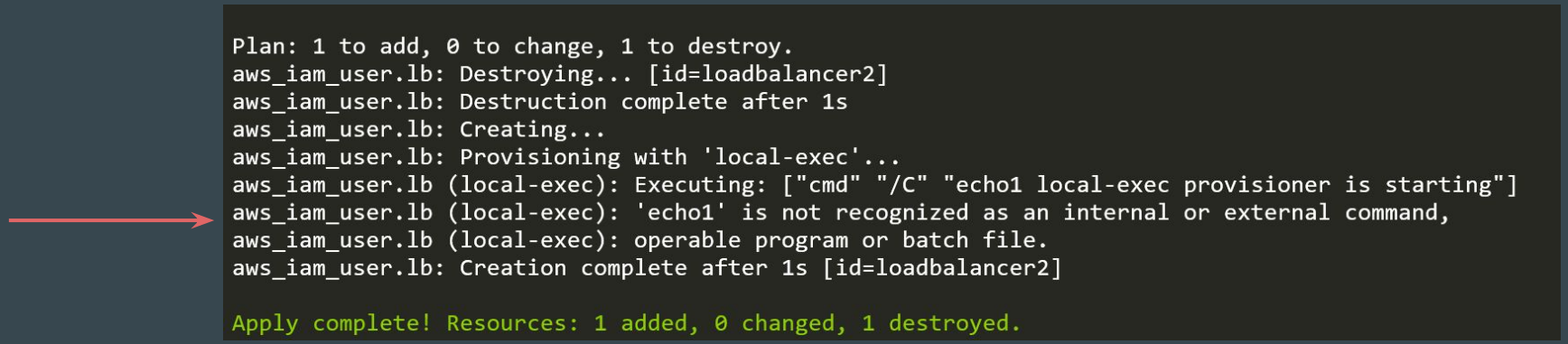
```
resource "aws_iam_user" "lb" {
  name = "loadbalancer"

  provisioner "local-exec" {
    command = "echo1 local-exec provisioner is starting"
    on_failure = continue
  }
}
```

Reference Screenshot - Failed Provisioner

Following screenshot shows that the provisioner has failed but still the apply has completed successfully.

This is an example of on_failure = continue



Join us in our Adventure

Be Awesome



kplabs.in/chat

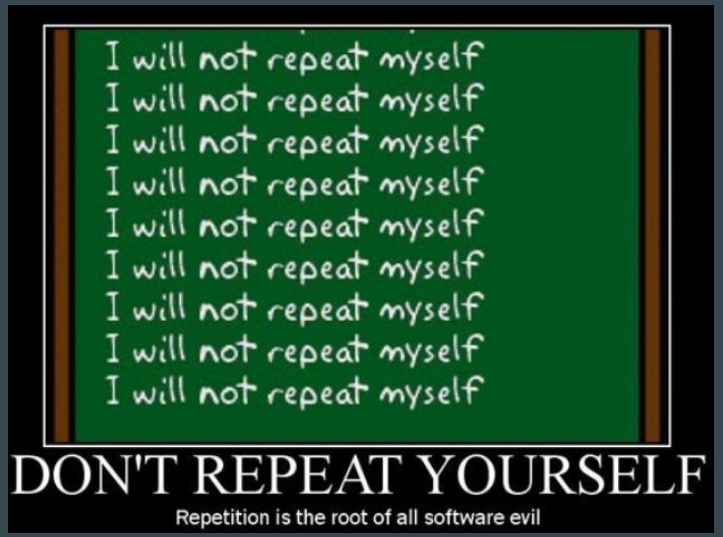


kplabs.in/linkedin

Terraform Modules

Understanding the Basic

In software engineering, don't repeat yourself (DRY) is a principle of software development aimed at reducing repetition of software patterns.



Understanding the Challenge

Let’s assume there are 10 teams in your organization using Terraform to create and manage EC2 instances.

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
}
```

Team1

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
}
```

Team 3

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
}
```

Team 5

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
}
```

Team 2

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
}
```

Team 4

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
}
```

Team 6

Challenge with the Previous Example

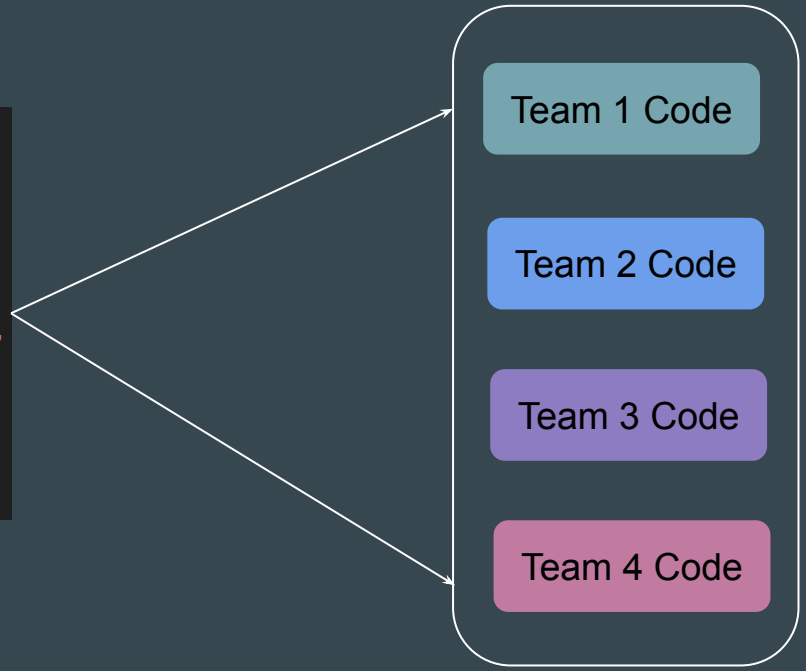
1. Repetition of Code.
2. Change in AWS Provider specific option will require change in EC2 code blocks of all the teams.
3. Lack of standardization.
4. Difficult to manage.
5. Difficult for developers to use.

Better Approach

In this approach, the DevOps Team has defined standard EC2 template in a central location that all can use.

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
  associate_public_ip_address = true
  instance_initiated_shutdown_behavior = "stop"
  ebs_optimized  = true
  source_dest_check = false
  hibernation = true
  disable_api_termination = true
}
```

Standard Template

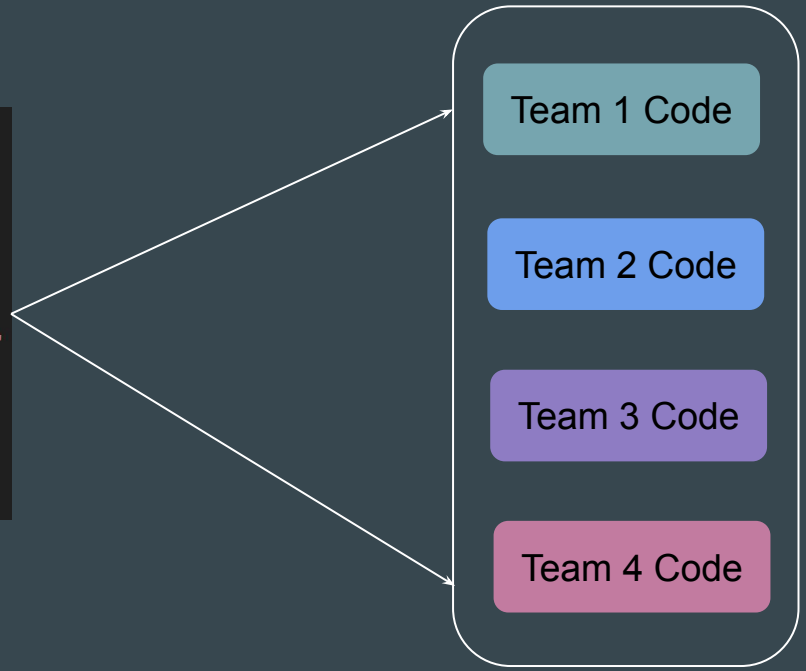


Introducing Terraform Modules

Terraform Modules allows us to centralize the resource configuration and it makes it easier for multiple projects to re-use the Terraform code for projects.

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
  associate_public_ip_address = true
  instance_initiated_shutdown_behavior = "stop"
  ebs_optimized  = true
  source_dest_check = false
  hibernation = true
  disable_api_termination = true
}
```

Terraform Module



Multiple Modules for a Single Project

Instead of writing code from scratch, we can use multiple ready-made modules available.

```
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
  associate_public_ip_address = true
  instance_initiated_shutdown_behavior = "stop"
  ebs_optimized  = true
  source_dest_check = false
  hibernation = true
  disable_api_termination = true
}
```

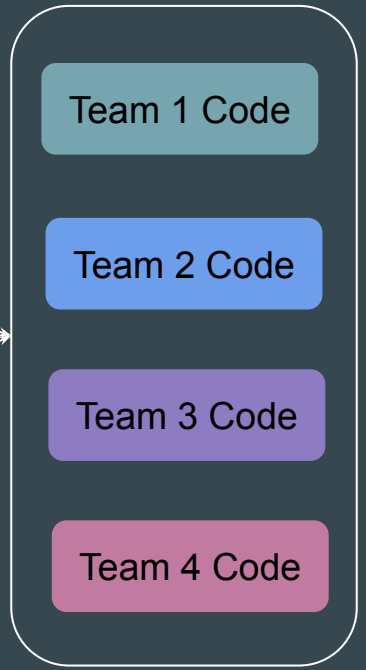
EC2 Module

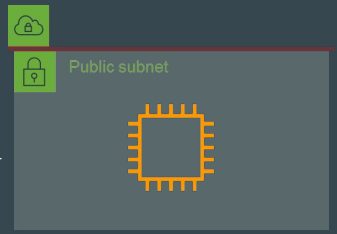
```
resource "aws_vpc" "this" {
  count = local.create_vpc ? 1 : 0

  cidr_block          = var.use_ipam_pool ? null : var.cidr
  ipv4_ipam_pool_id   = var.ipv4_ipam_pool_id
  ipv4_netmask_length = var.ipv4_netmask_length

  assign_generated_ipv6_cidr_block     = var.enable_ipv6 && !var.use_ipam_pool ? true : null
  ipv6_cidr_block                      = var.ipv6_cidr
  ipv6_ipam_pool_id                    = var.ipv6_ipam_pool_id
  ipv6_netmask_length                  = var.ipv6_netmask_length
  ipv6_cidr_block_network_border_group = var.ipv6_cidr_block_network_border_group
```

VPC Module





Infrastructure Created

Points to Note - Referencing Terraform Modules

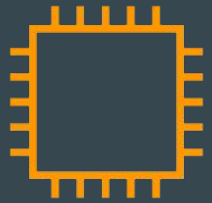
Understanding the Base

For some infrastructure resources, you can directly use the module calling code, and the entire infrastructure will be created for you.

ec2-module.tf > ...

```
module "ec2-instance" {
  source  = "terraform-aws-modules/ec2-instance/aws"
  version = "5.6.1"
}
```

terraform apply



Avoiding Confusion

Just by referencing any module, it is not always the case that the infrastructure resource will be created for you directly.

Some modules require specific inputs and values from the user side to be filled in before a resource gets created.

Example Module - AWS EKS

If you try to use an AWS EKS Module directly and run “terraform apply”, it will throw an error.

eks-module.tf > ...

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "20.11.1"
}
```

terraform apply

```
Plan: 21 to add, 0 to change, 0 to destroy.

Error: Error in function call

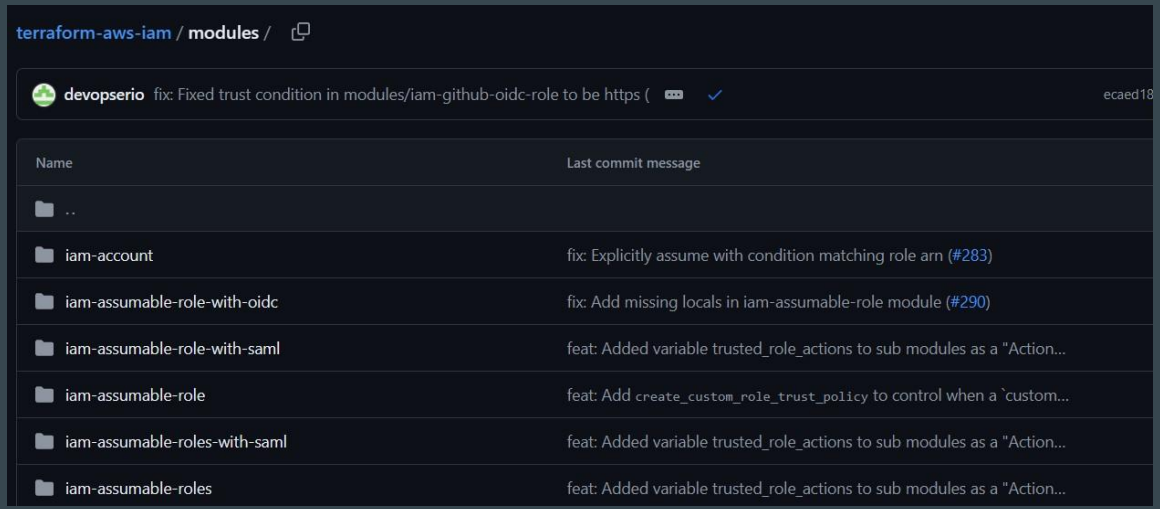
  on .terraform\modules\eks\main.tf line 48, in resource "aws_eks_cluster" "this":
  48:     subnet_ids              = coalescelist(var.control_plane_subnet_ids, var.subnet_ids)
    ├────────────────
    while calling coalescelist(vals...)
    var.control_plane_subnet_ids is empty list of string
    var.subnet_ids is empty list of string

Call to function "coalescelist" failed: no non-null arguments.
```

Module Structure Can be Different

Some module pages in GitHub can contain multiple sets of modules together for different features.

In such cases, you have to reference the exact sub-module required.



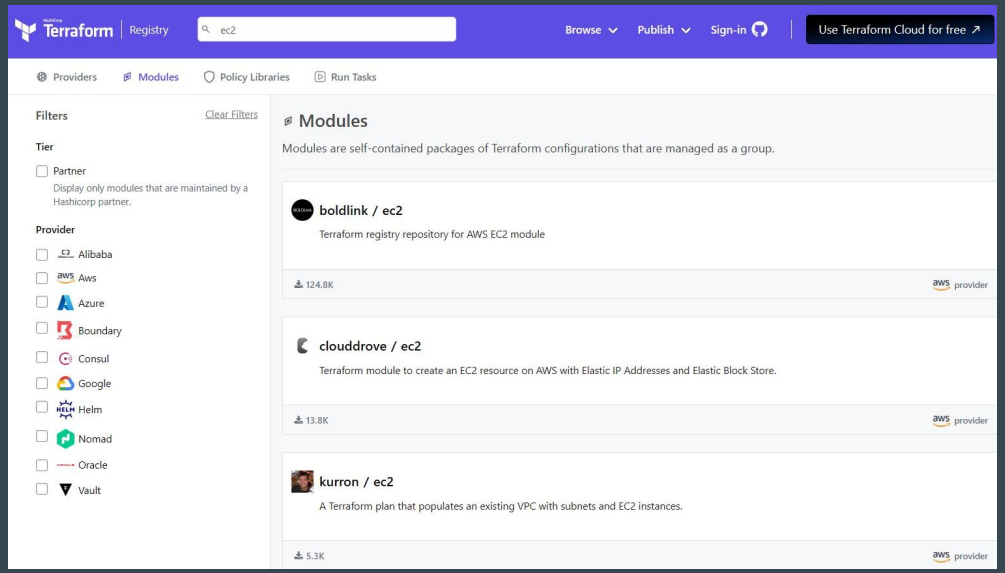
Learnings for Today's Video

Always read the Module Documentation to understand the overall structure, important information, and what is expected from the user side when creating a resource.

Choosing the Right Terraform Module

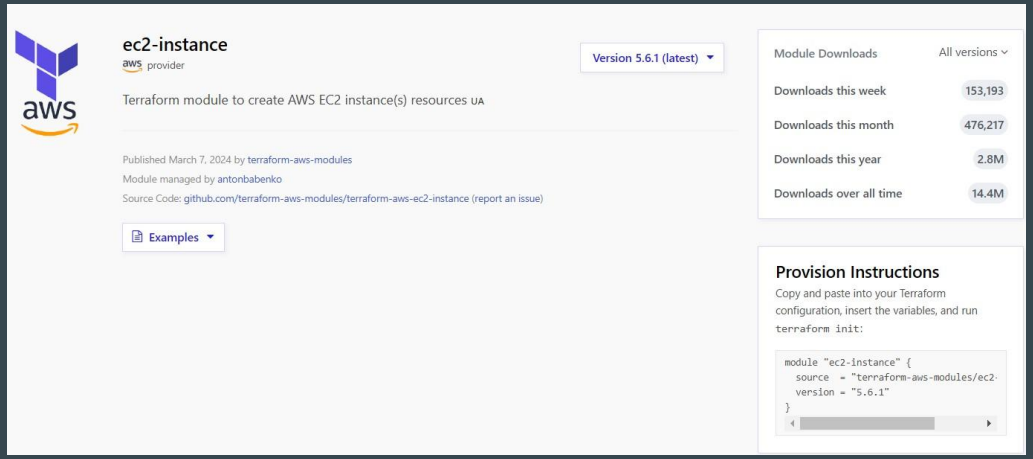
Understanding the Base

Terraform Registry can contain multiple modules for a specific infrastructure resource maintained by different users



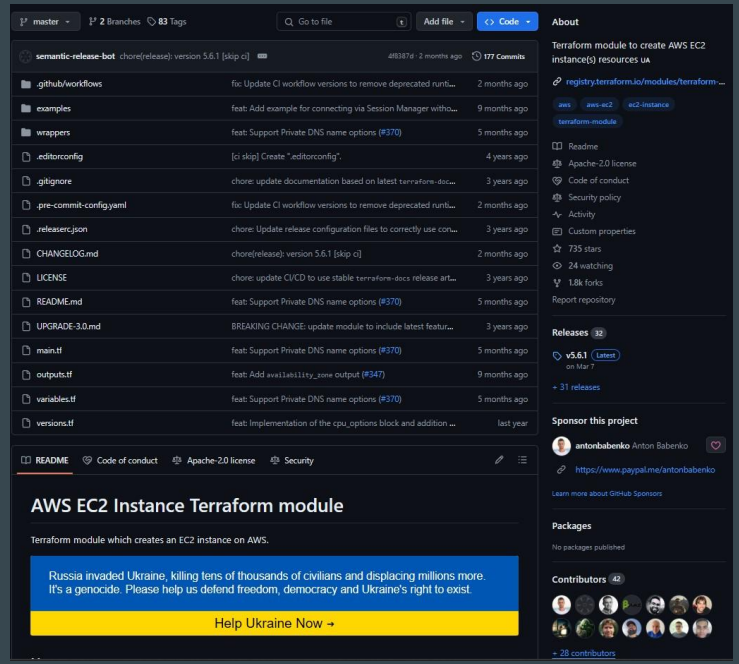
1 - Check Total Downloads

Module Downloads can provide early indication about level of acceptance by users in the Terraform community



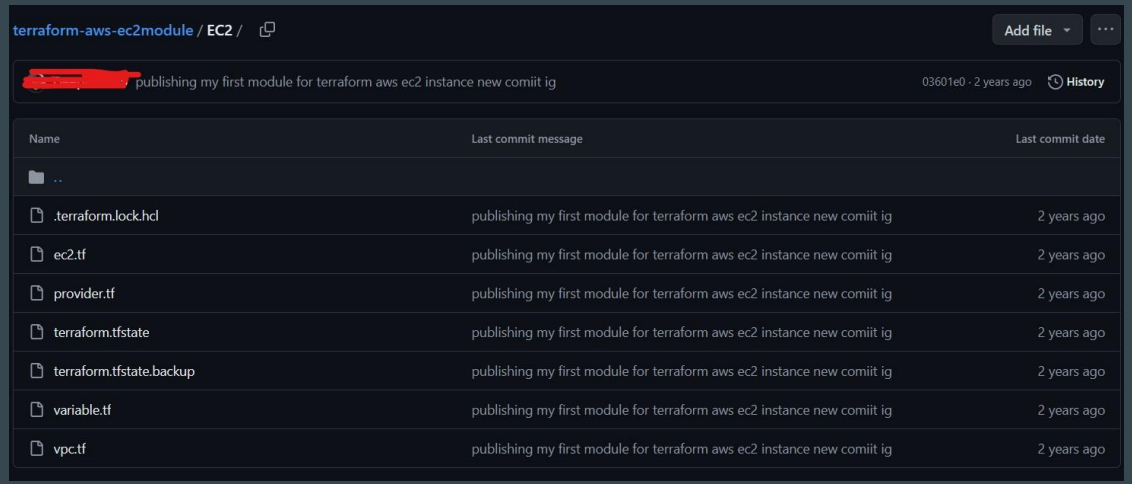
2 - Check GitHub Page of Module

GitHub page can provide important information related to the Contributors, Reported Issues and other data.



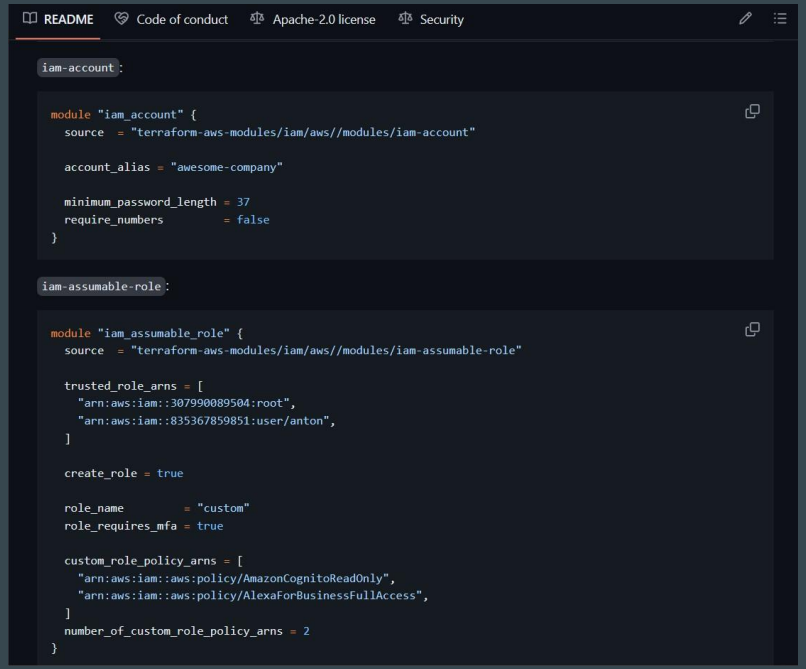
3 - Avoid Modules Written by Individual Participant

Avoid module that are maintained by a single contributor as regular updates, issues and other areas might not always be maintained.



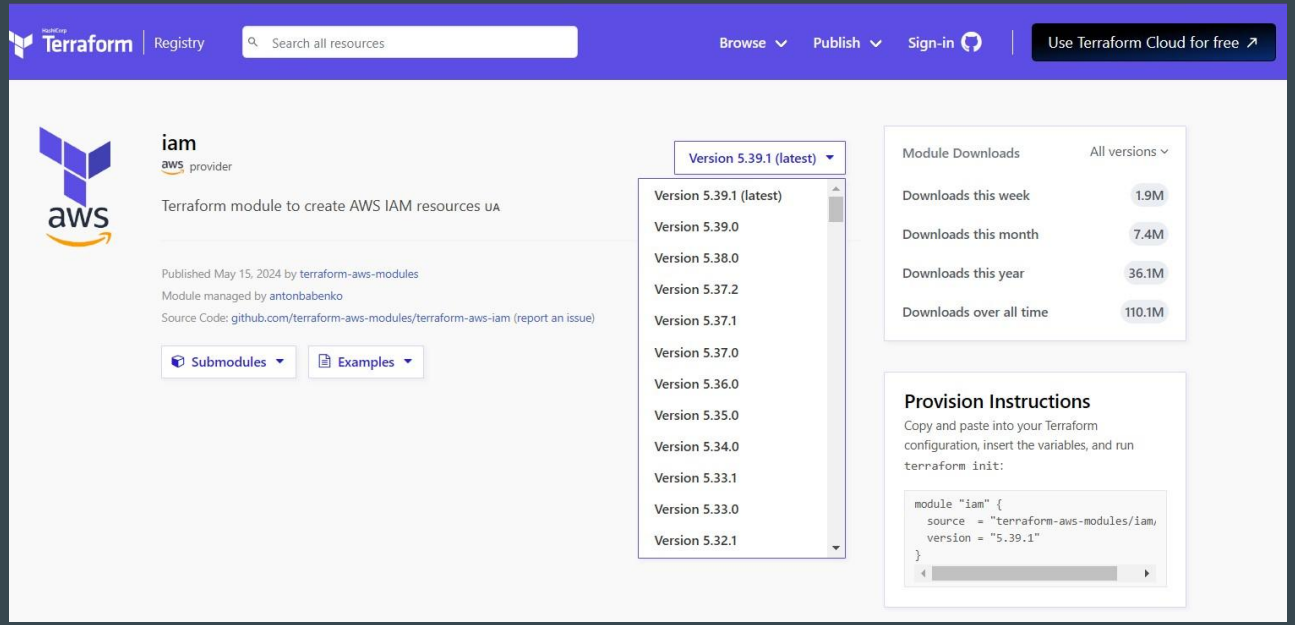
4 - Analyze Module Documentation

Good documentation should include an overview, usage instructions, input and output variables, and examples.



5 - Check Version History of Module

Look at the version history. Frequent updates and a clear versioning strategy suggest active maintenance.



6 - Analyze the Code

Inspect the module's source code on GitHub or another platform. Clean, well-structured code is a good sign.

```
resource "aws_instance" "this" {
  count = local.create && !var.ignore_ami_changes && !var.create_spot_instance ? 1 : 0

  ami                  = local.ami
  instance_type        = var.instance_type
  cpu_core_count       = var.cpu_core_count
  cpu_threads_per_core = var.cpu_threads_per_core
  hibernation          = var.hibernation

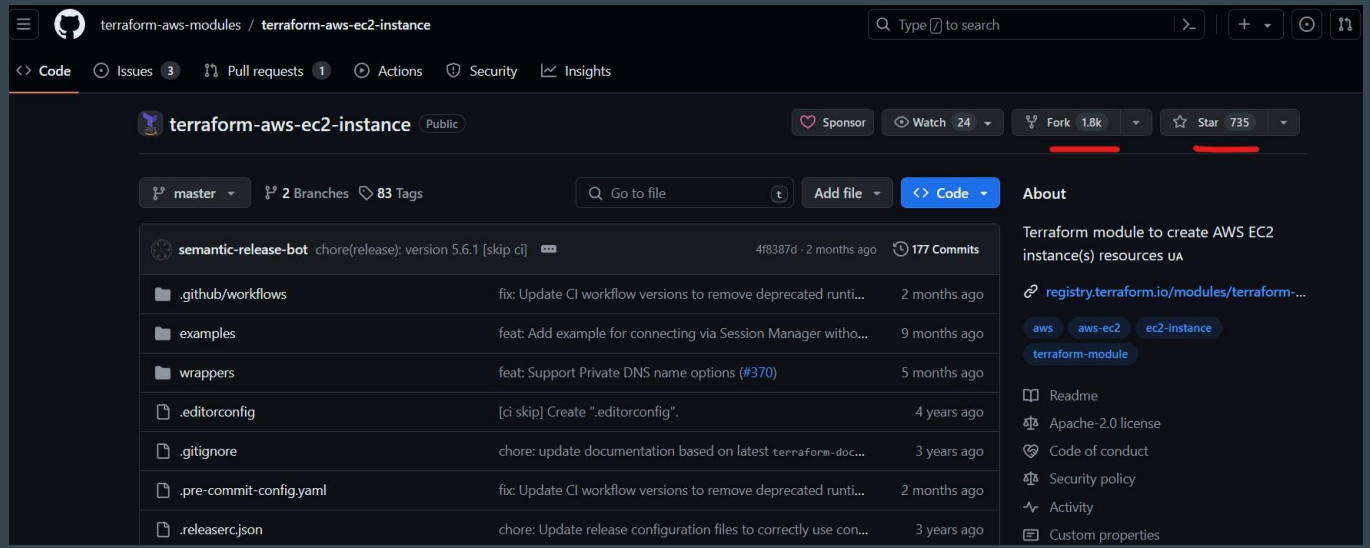
  user_data                   = var.user_data
  user_data_base64            = var.user_data_base64
  user_data_replace_on_change = var.user_data_replace_on_change

  availability_zone      = var.availability_zone
  subnet_id              = var.subnet_id
  vpc_security_group_ids = var.vpc_security_group_ids

  key_name             = var.key_name
  monitoring           = var.monitoring
  get_password_data    = var.get_password_data
  iam_instance_profile = var.create_iam_instance_profile ? aws_iam_instance_profile.this[0].name : var.iam_instance_profile
```

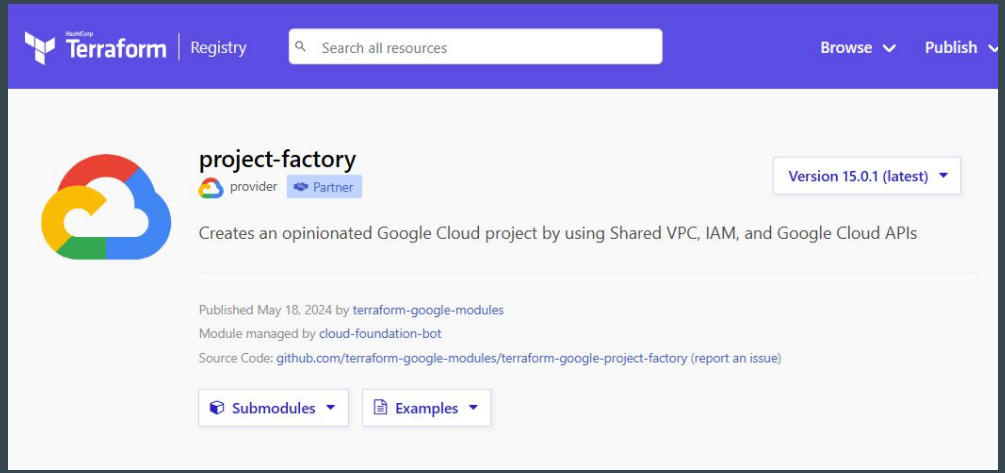
7 - Check the Community Feedback

The number of stars and forks on GitHub can indicate popularity and community interest.



8 - Modules Maintained by HashiCorp Partner

Search for modules that are maintained by HashiCorp Partners



Important Point to Note

Avoid directly trying any random Terraform module that is not actively maintained and looks shady (primarily by sole individual contributors)

An attacker can include malicious code in a module that sends information about your environment to the attacker.

Which Modules do Organizations Use?

In most of the scenarios, organizations maintain their own set of modules.

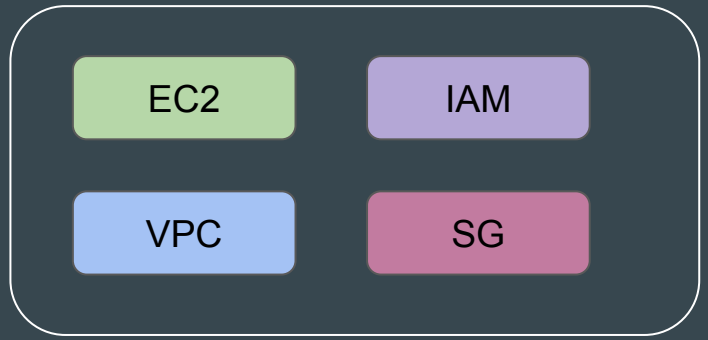
They might initially fork a module from the Terraform registry and modify it based on their use case.

Creating Base Module Structure

Understanding the Base Structure

A base “modules” folder.

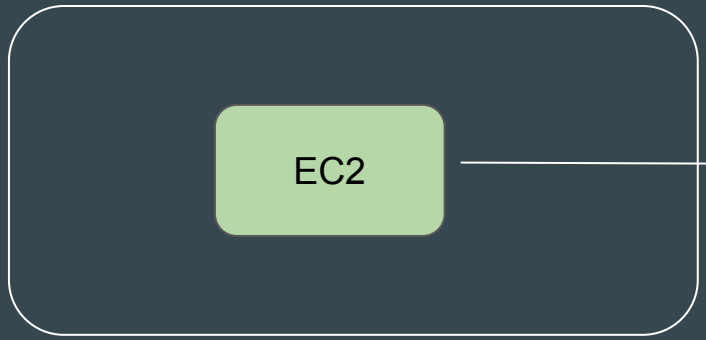
A sub-folder containing name for each modules that are available.



modules folder

What is Inside the Sub-Folders

Each module's sub-folder contains the actual module Terraform code that other projects can reference from.



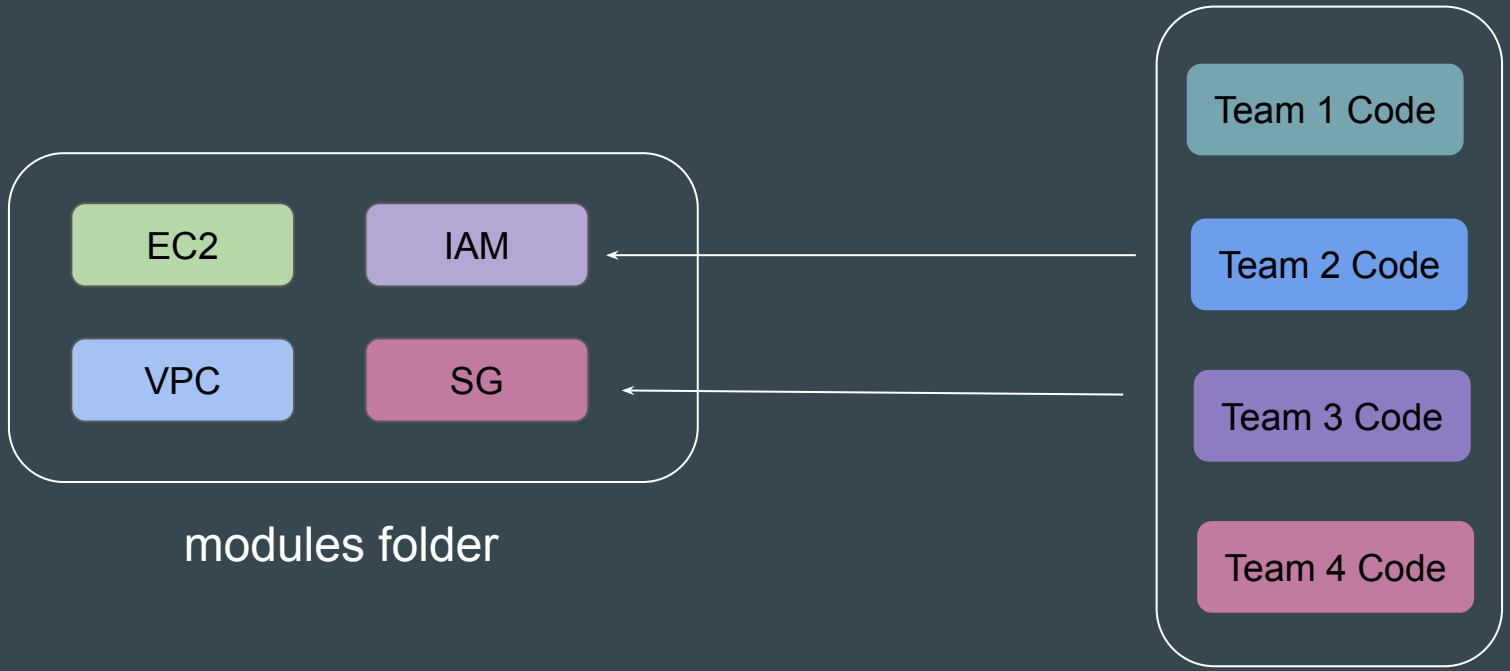
modules folder

```hcl
resource "aws_instance" "web" {
  ami           = "ami-1234"
  instance_type = "t3.micro"
  key_name      = "user1"
  monitoring    = true
  vpc_security_group_ids = ["sg-12345678"]
  associate_public_ip_address = true
  instance_initiated_shutdown_behavior = "stop"
  ebs_optimized  = true
  source_dest_check = false
  hibernation = true
  disable_api_termination = true
}
```

main.tf

Calling the Module

Each Team can call various set of modules that are available in the modules folder based on their requirements.

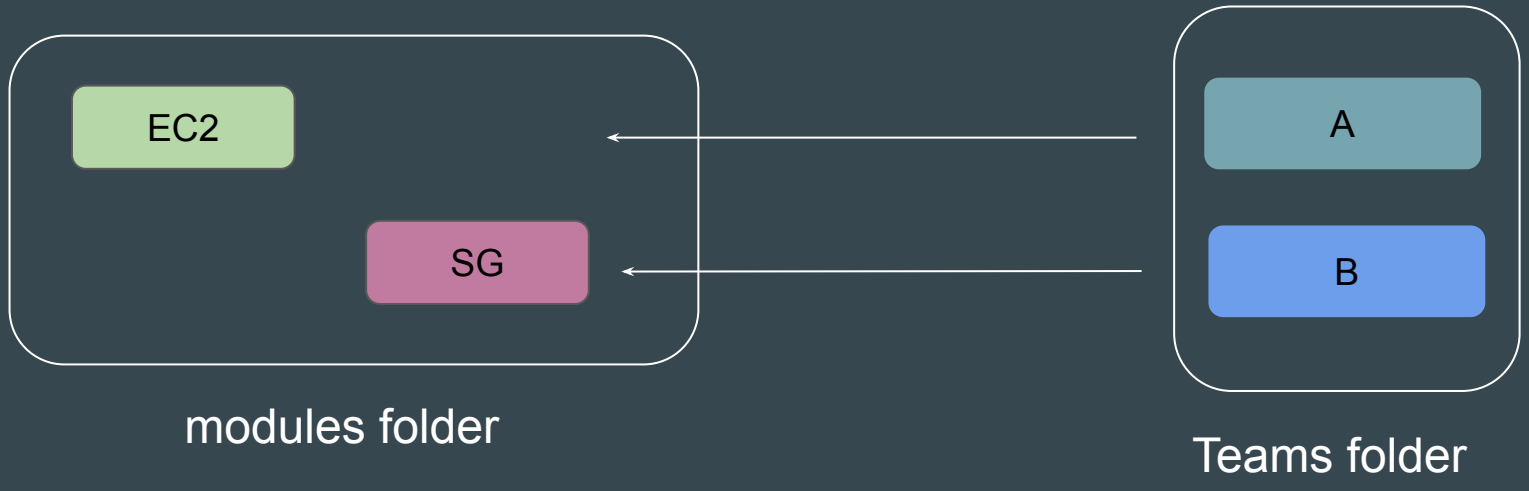


Our Practical Structure

Our practical structure will include two main folders (modules and teams).

Modules sub-folder will contain sub-folders of modules that are available.

Teams sub-folder will contain list of teams that we want to be made available.



Module Sources - Calling a Module

Understanding the Base

Module source code can be present in wide variety of locations.

These includes:

1. GitHub
2. HTTP URLs
3. S3 Buckets
4. Terraform Registry
5. Local paths

Base - Calling the Module

In order to reference to a module, you need to make use of module block

The module block must contain source argument that contains location to the referenced module.

```
module "ec2" {
  source = "https://example.com/vpc-module.zip"
}
```

Example 1 - Local Paths

Local paths are used to reference to module that is available in local filesystem.

A local path must begin with either ./ or ../ to indicate that a local path

```
module "ec2" {
  source = "../modules/ec2"
}
```

Example 2 - Generic Git Repository

Arbitrary Git repositories can be used by prefixing the address with the special git:: prefix.

```
module "vpc" {
  source = "git::https://example.com/vpc.git"
}
```

Module Version

A specific module can have multiple versions.

You can reference to specific version of module with the version block

```
module "eks" {
  source  = "terraform-aws-modules/eks/aws"
  version = "20.11.1"
}
```

Improvements in Custom Module Code

Our Simple Module

We had created a very simple module that allows developers to launch an EC2 instance when calling the module.

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "myec2" {
    ami = "ami-0bb84b8ffd87024d8"
    instance_type = "t2.micro"
}
```

Need to Analyze Shortcomings

Being a simplistic and a basic module code, there is a good room of improvements.

In today's video, we will be discussing about some of the important shortcomings with the code.



Challenge 1 - Hardcoded Values

The values are hardcoded as part of the module.

If developer is calling the module, he will have to stick with same values.

Developer will not be able to override the hardcoded values of the module.



Challenge 2 - Provider Improvements

Avoid hard-coding region in the Module code as much as possible.

A required_provider block with version constraints for module to work is important.

```
provider "aws" {
  region = "us-east-1"
}

resource "aws_instance" "myec2" {
    ami = "ami-0bb84b8ffd87024d8"
    instance_type = "t2.micro"
}
```



```
terraform {

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 5.5"
    }
  }
}

resource "aws_instance" "myec2" {
    ami = "ami-0bb84b8ffd87024d8"
    instance_type = "t2.micro"
}
```

Variables in Terraform Modules

Point to Note

As much as possible, avoid hardcoding values as part of the Modules.

This will make the module less flexible.



Convert Hard Coded Values to Variables

For modules, it is especially recommended to convert hard-coded values to variables so that it can be overridden based on user requirements.

```
resource "aws_instance" "myec2" {
    ami = "ami-0bb84b8ffd87024d8"
    instance_type = "t2.micro"
}
```

Bad Approach



```
resource "aws_instance" "myec2" {
    ami = var.ami
    instance_type = var.instance_type
}
```

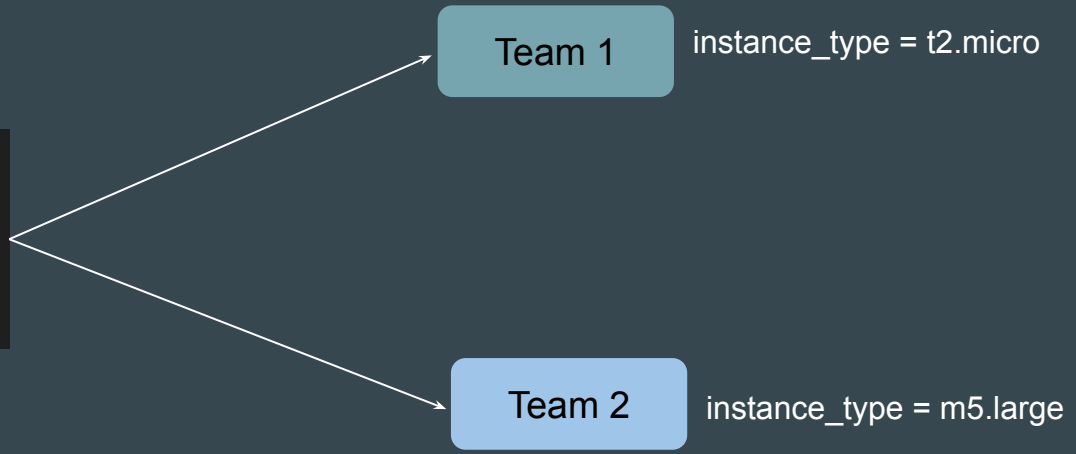
Good Approach

Advantages of Variables in Module Code

Variable based approach will allow the teams to override the values.

```
resource "aws_instance" "myec2" {
    ami = var.ami
    instance_type = var.instance_type
}
```

Main Module



Reviewing Professional EC2 Module Code

Reviewing an EC2 Module code that is professionally written, we see that the values associated with arguments are not hardcoded and variables are used extensively.

```
resource "aws_instance" "this" {
  count = local.create && !var.ignore_ami_changes && !var.create_spot_instance ? 1 : 0

  ami                  = local.ami
  instance_type        = var.instance_type
  cpu_core_count       = var.cpu_core_count
  cpu_threads_per_core = var.cpu_threads_per_core
  hibernation          = var.hibernation

  user_data                   = var.user_data
  user_data_base64            = var.user_data_base64
  user_data_replace_on_change = var.user_data_replace_on_change

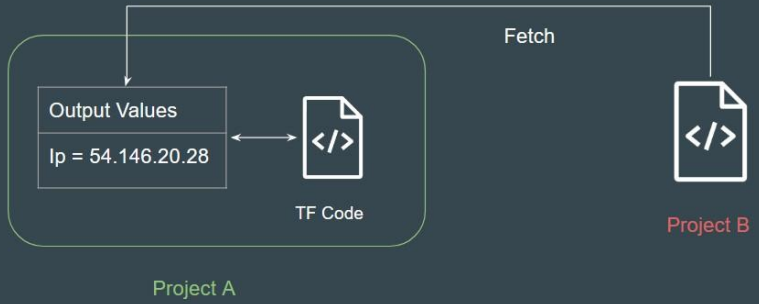
  availability_zone      = var.availability_zone
  subnet_id              = var.subnet_id
  vpc_security_group_ids = var.vpc_security_group_ids

  key_name             = var.key_name
  monitoring           = var.monitoring
  get_password_data    = var.get_password_data
  iam_instance_profile = var.create_iam_instance_profile ? aws_iam_instance_profile.this[0].name : var.iam_instance_profile
```

Module Outputs

Revising Output Values

Output values make information about your infrastructure available on the command line, and can expose information for other Terraform configurations to use.




```
resource "aws_eip" "lb" {
  domain   = "vpc"
}

output "public-ip" {
  value = aws_eip.lb.public_ip
}
```

Understanding the Challenge

If you want to create a resource that has a dependency on an infrastructure created through a module, you won't be able to implicitly call it without output values.

```
resource "aws_instance" "myec2" {
    ami = "ami-123"
    instance_type = "t2.micro"
}
```



```
module "ec2" {
  source = "../../modules/ec2"
}

resource "aws_eip" "lb" {
  instance = module.ec2.instance_id
  domain   = "vpc"
}
```

Accessing Child Module Outputs

Ensure to include output values in the module code for better flexibility and integration with other resources and projects.

Format: module.<MODULE NAME>.<OUTPUT NAME>

```
resource "aws_instance" "myec2" {
    ami = var.ami
    instance_type = var.instance_type
}

output "instance_id" {
  value = aws_instance.myec2.id
}
```



```
module "ec2" {
  source = "../../modules/ec2"
}

resource "aws_eip" "lb" {
  instance = module.ec2.instance_id
  domain   = "vpc"
}
```

Root and Child Modules

Root Module

Root Module resides in the main working directory of your Terraform configuration. This is the entry point for your infrastructure definition

```
module "ec2" {
  source = "../../modules/ec2"
}
```

Root Module

Child Module

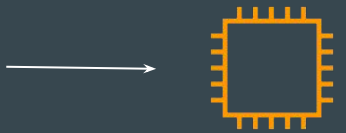
A module that has been called by another module is often referred to as a child module.

```
resource "aws_instance" "myec2" {
    ami = var.ami
    instance_type = var.instance_type
}
```

Child Module

```
module "ec2" {
  source = "../../modules/ec2"
}
```

Root Module



Standard Module Structure

Setting the Base

At this stage, we have been keeping the overall module structure very simple to understand the concepts.

In production environments, it is important to follow recommendations and best-practices set by HashiCorp.

Basic of Standard Module Structure

The standard module structure is a file and directory layout HashiCorp recommends for reusable modules.

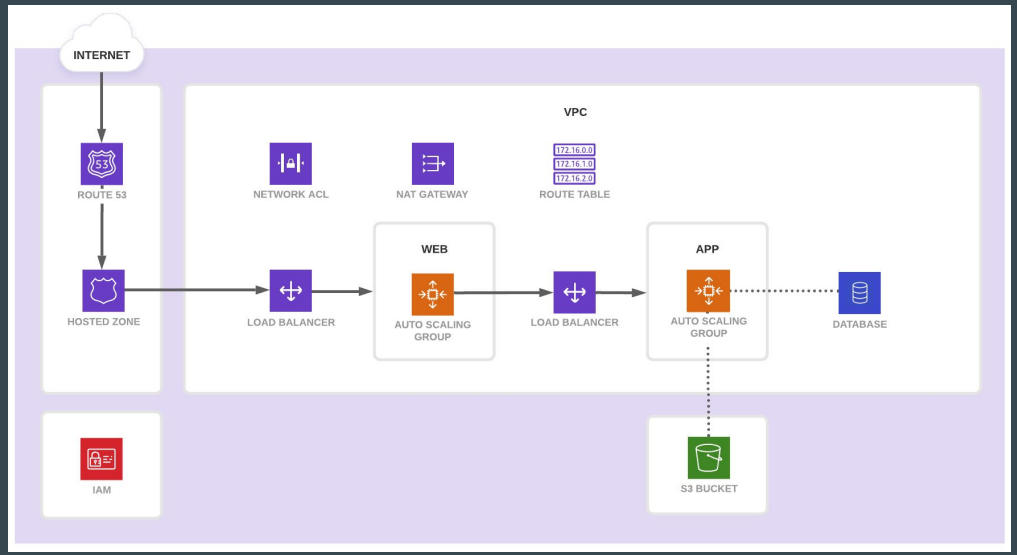
A minimal recommended module following the standard structure is shown below

```
$ tree minimal-module/
.
├── README.md
├── main.tf
├── variables.tf
└── outputs.tf
```

Scope the Requirements for Module Creation

A team wants to provision their infrastructure using Terraform.

The following architecture diagram depicts the desired outcome.



Planning a Module Structure

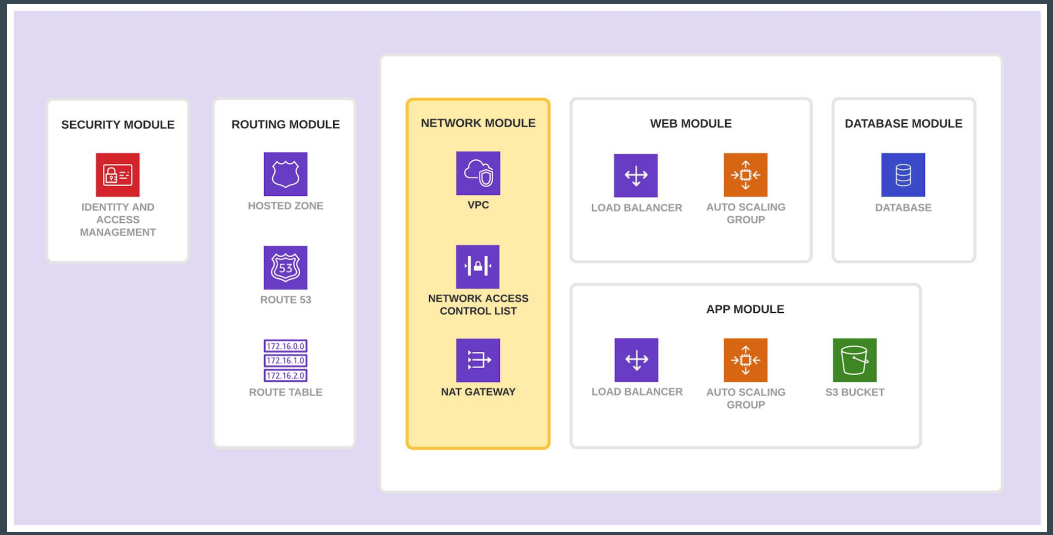
In this scenario, a team of Terraform producers, who write Terraform code from scratch, will build a collection of modules to provision the infrastructure and applications.

The members of the team in charge of the application will consume these modules to provision the infrastructure they need.

Final Module Output

After reviewing the consumer team's requirements, the producer team has broken up the application infrastructure into the following modules:

Network, Web, App, Database, Routing, and Security.



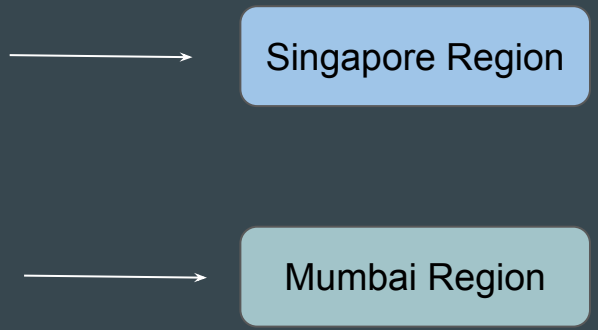
Multiple Provider Configuration

Understanding the Requirement

There can be a requirement that multiple resource types in the same TF file need to be deployed in separate regions.

```
resource "aws_instance" "myec2" {
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}

resource "aws_security_group" "allow_tls" {
  name        = "staging_firewall"
}
```



Setting the Base

At this stage, we have been dealing with single-provider configuration.

In the below code, both resources will be created in Singapore region.

```
provider "aws" {
    region = "ap-southeast-1"
}

resource "aws_instance" "myec2" {
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}

resource "aws_security_group" "allow_tls" {
  name        = "staging_firewall"
}
```

Alias Meta-Argument

Each provider can have one default configuration, and any number of alternate configurations that include an extra name segment (or "alias").

```
resource "aws_instance" "myec2" {
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}
```



```
resource "aws_security_group" "allow_tls" {
  name        = "staging_firewall"
}
```



```
provider "aws" {
    region = "ap-southeast-1"
}

provider "aws" {
    alias  = "mumbai"
    region = "ap-south-1"
}

provider "aws" {
    alias = "usa"
    region = "us-east-1"
}
```

Final Output Using Alias

By using the provider meta-argument, you can select an alternate provider configuration for a resource.

```
provider "aws" {
    region = "ap-southeast-1"
}

provider "aws" {
    alias  = "mumbai"
    region = "ap-south-1"
}

provider "aws" {
    alias = "usa"
    region = "us-east-1"
}
```

```
resource "aws_instance" "myec2" {
  provider = aws.mumbai
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}
```

```
resource "aws_security_group" "allow_tls" {
  provider    =  aws.usa
  name        = "staging_firewall"
}
```

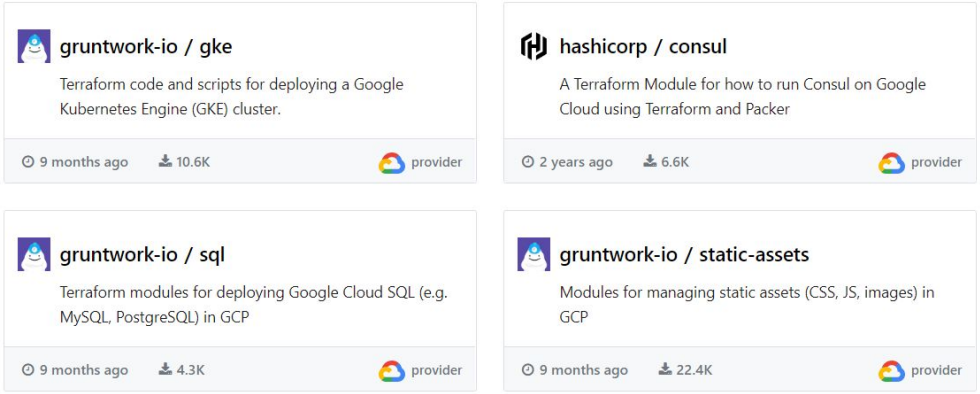
Publishing Modules

Publish Modules to Terraform Registry

Overview of Publishing Modules

Anyone can publish and share modules on the Terraform Registry.

Published modules support versioning, automatically generate documentation, allow browsing version histories, show examples and READMEs, and more.



Requirements for Publishing Module

Requirement	Description
GitHub	The module must be on GitHub and must be a public repo. This is only a requirement for the public registry.
Named	Module repositories must use this three-part name format terraform-<PROVIDER>-<NAME>
Repository description	The GitHub repository description is used to populate the short description of the module.
Standard module structure	The module must adhere to the standard module structure.
x.y.z tags for releases	The registry uses tags to identify module versions. Release tag names must be a semantic version, which can optionally be prefixed with a v. For example, v1.0.4 and 0.9.2

Standard Module Structure

The standard module structure is a file and directory layout that is recommend for reusable modules distributed in separate repositories

```
$ tree minimal-module/
.
├── README.md
├── main.tf
├── variables.tf
└── outputs.tf
```

```
$ tree complete-module/
.
├── README.md
├── main.tf
├── variables.tf
├── outputs.tf
├── ...
├── modules/
│   ├── nestedA/
│   │   ├── README.md
│   │   ├── variables.tf
│   │   ├── main.tf
│   │   └── outputs.tf
│   ├── nestedB/
│   └── .../
├── examples/
│   ├── exampleA/
│   │   └── main.tf
│   ├── exampleB/
│   └── .../
```

Terraform Workspace

Setting the Base

An infrastructure created through Terraform is tied to the underlying Terraform configuration and a state file.

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
}
```



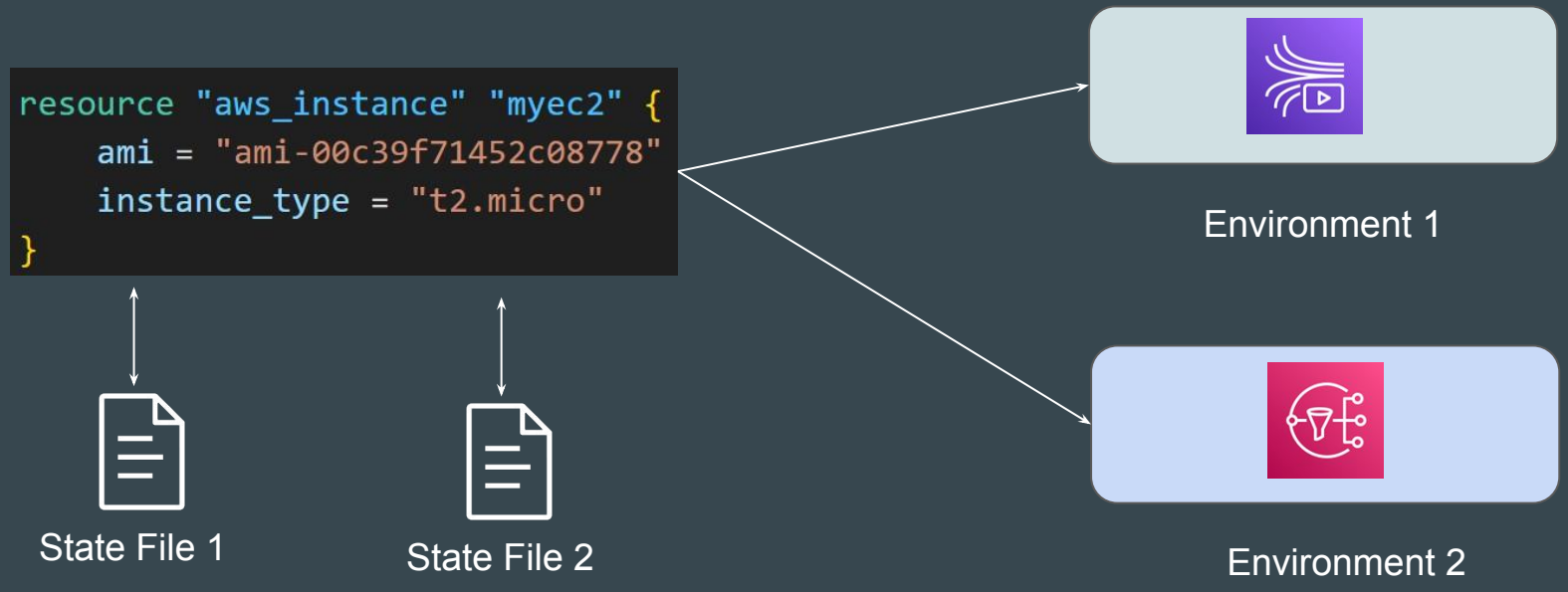
EC2 Instance

terraform.tfstate

What If?

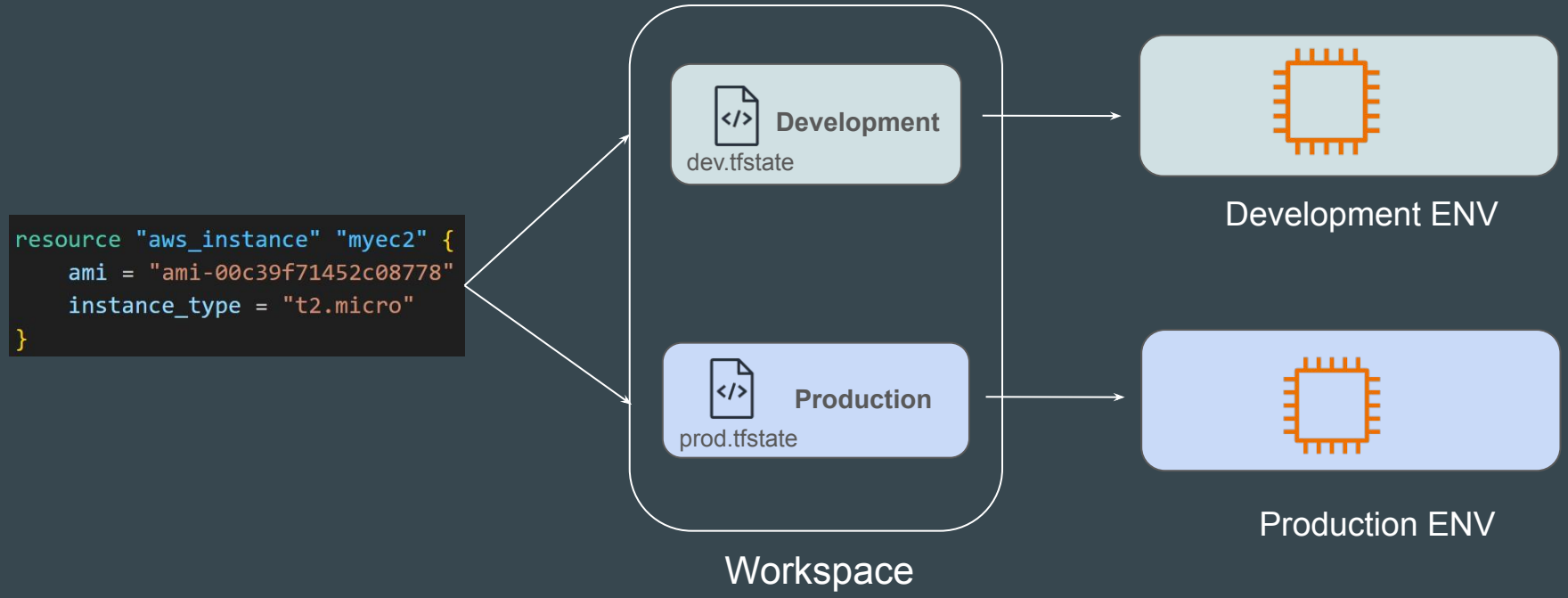
What if we have multiple state file for single Terraform configuration?

Can we manage different env's through it separately?



Introducing Terraform Workspace

Terraform workspaces enable us to manage multiple set of deployments from the same sets of configuration file.

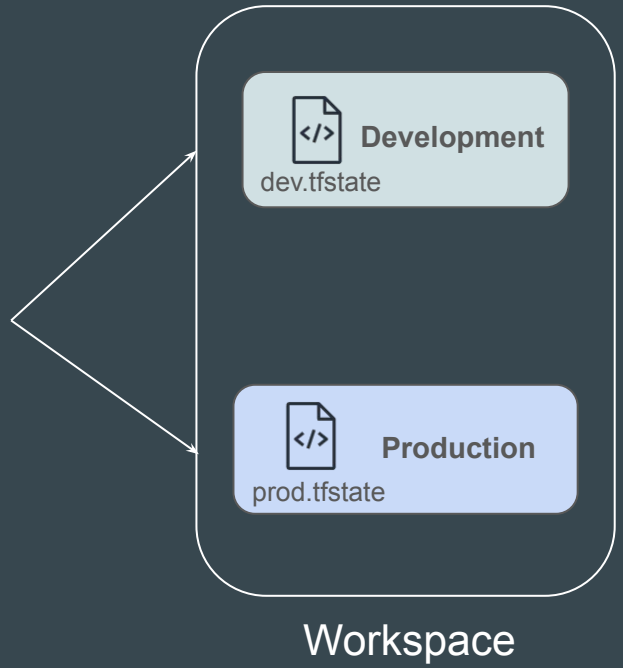


Flexibility with Workspace

Depending on the workspace being used, the value to a specific argument in your Terraform code can also change.

```
resource "aws_instance" "myec2" {
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = local.instance_types[terraform.workspace]
}
```

Environment	instance_type
Development	t2.micro
Production	m5.large



Team Collaboration

Local Changes are not always good

Until now, we've been working with Terraform code stored on our local machines.



Disadvantages of a Local-Only Approach

1. If your laptop fails, you risk losing both your Terraform code and, critically, the state file.

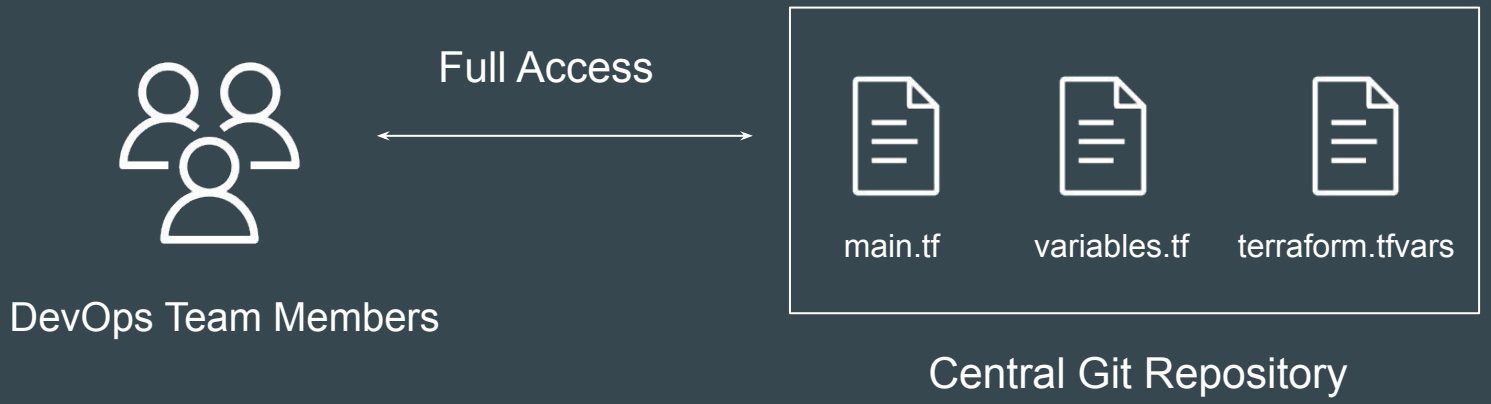
2. Team collaboration is difficult—other members can't access or update the infrastructure code.

3. No versioning of changes is visible.

Integration with Git Repository - Essential

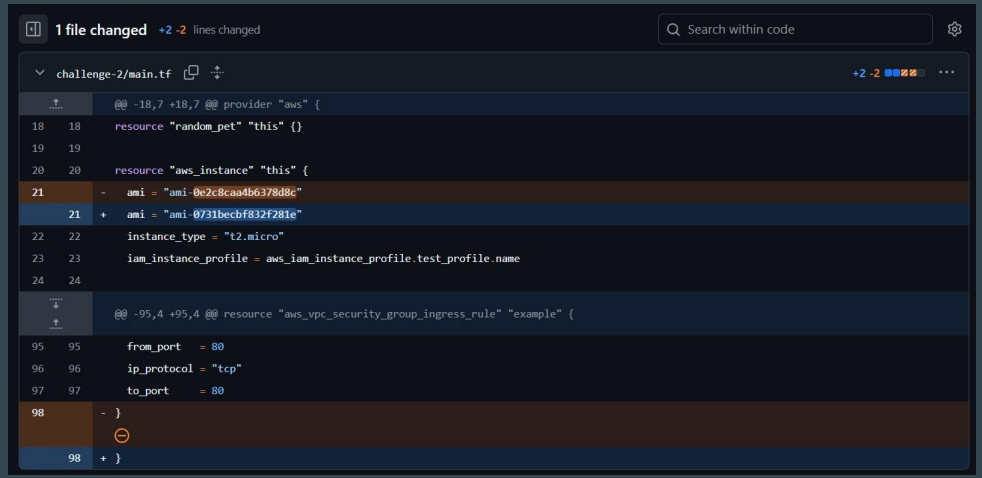
It is very important that all of your Terraform code is stored in a Git repository.

All DevOps Team members can have access to Git repository.



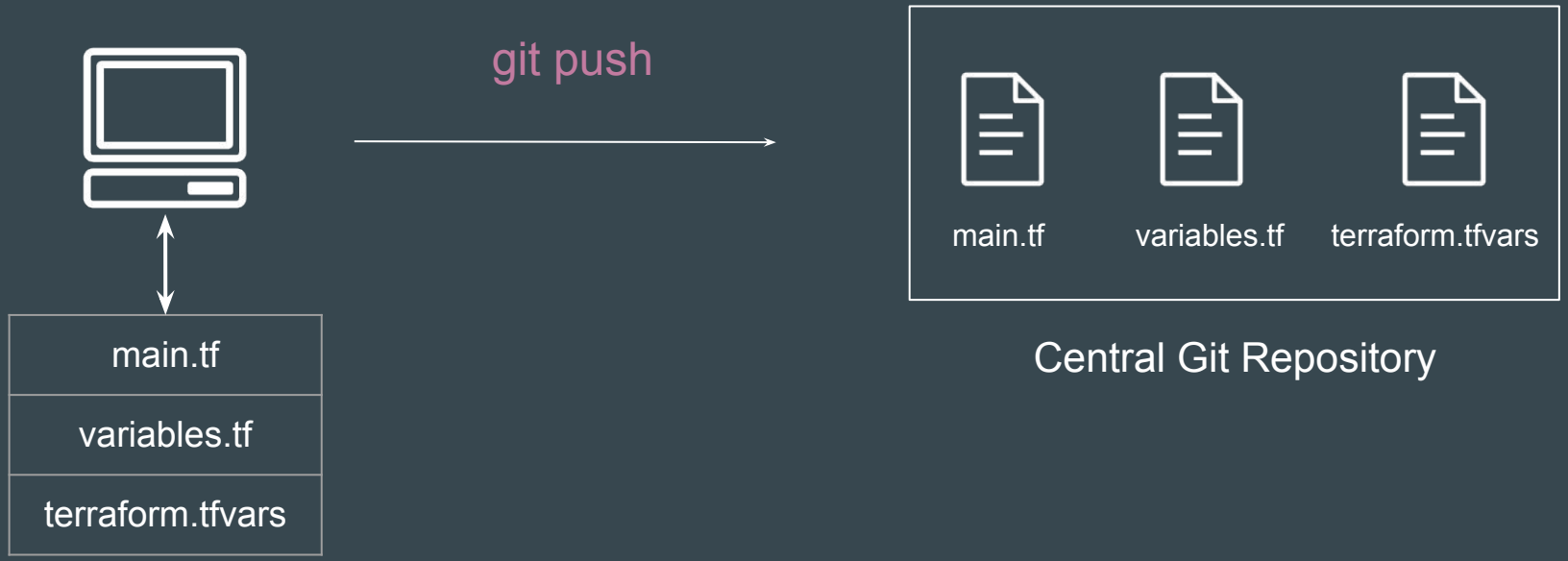
Benefits of Git Repository

1. Centralized access for all team members
2. Full version history and change tracking
3. Code review and approval workflows
4. Integration with CI/CD for validation and plans



Important Step To-Do

Always commit and push your changes to the Git repository so all team members see the latest updates.



Point to Note - To be Discussed in Next Lectures

While it's good practice to commit your Terraform code files to Git, you shouldn't commit every file.

terraform.tfstate file should NOT be added to your Git repository.

.terraform folder should NOT be added to your Git repository.



Central Git Repository

Relax and Have a Meme Before Proceeding

me: i'll do it at 6

time: 6:05

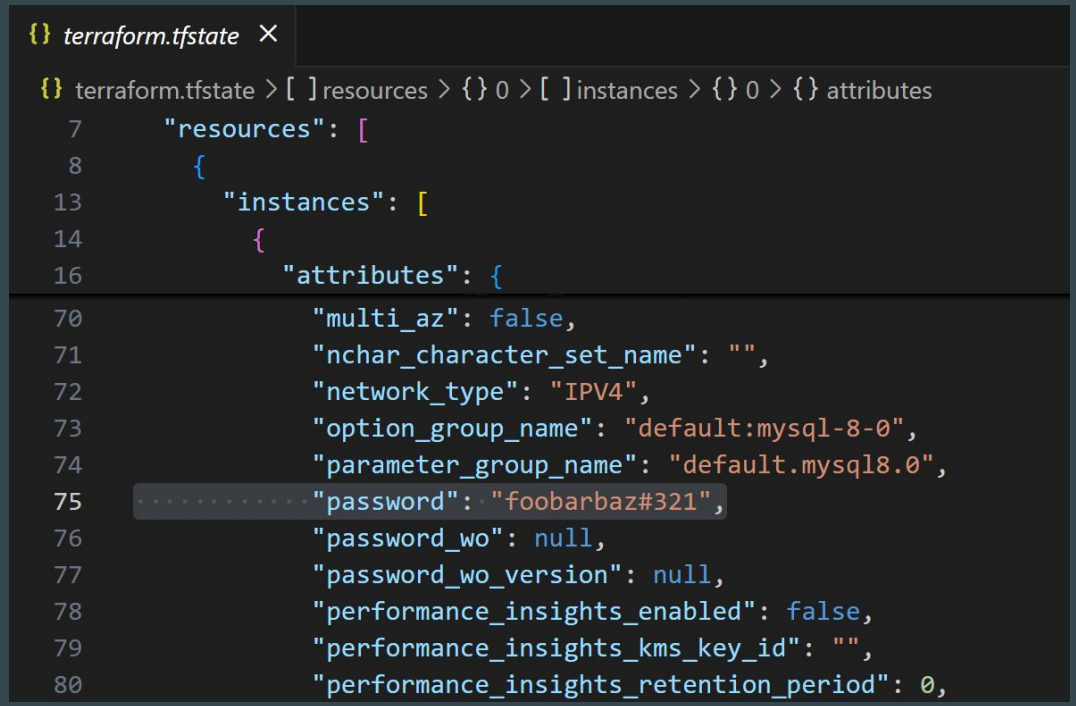
me: wow looks like i gotta wait til 7 now



Security Risks of Storing Terraform State Files in Git

Setting the Base

Terraform State File (terraform.tfstate) can include secrets in plain text (e.g., passwords, tokens, etc)



Why Not Commit to Git

Committing terraform.tfstate file to Git risks accidental exposure if the repository is public, shared, or compromised, leading to data breaches.



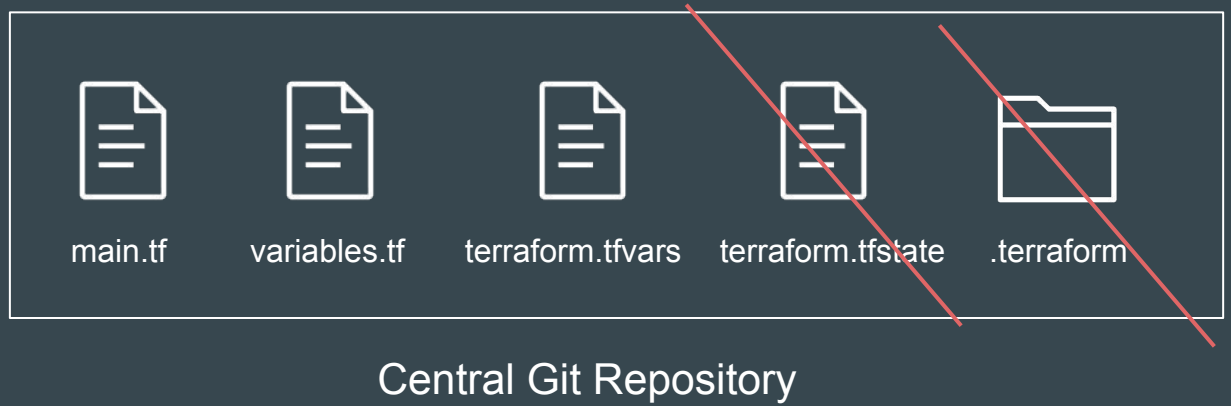
Terraform and .gitignore

Revising A Point Discussed in Earlier Video

While it's good practice to commit your Terraform code files to Git, you shouldn't commit every file.

terraform.tfstate file should NOT be added to your Git repository.

.terraform folder should NOT be added to your Git repository.



Central Git Repository

Files You Should Avoid Publishing to Git Repository

Do Not Commit	Description
.terraform	Your .terraform directory, where Terraform downloads providers and child modules.
terraform.tfvars	Any .tfvars files that contain sensitive information.
Terraform.tfstate and terraform.tfstate.backup	Avoid committing terraform.tfstate and backup file.
crash.log	If terraform crashes, the logs are stored to a file named crash.log
terraform.tfstate.lock.info	Terraform creates and deletes this file automatically when you run a terraform apply command and contains info about your state lock
Saved Plan Files	Saved plan files that you create when you include the -out flag when you run terraform plan.

Overview of gitignore

The .gitignore file is a text file that tells Git which files or folders to ignore in a project.

.gitignore
terraform.tfstate
.terraform/
crash.log



Always Commit the Following

Always commit:

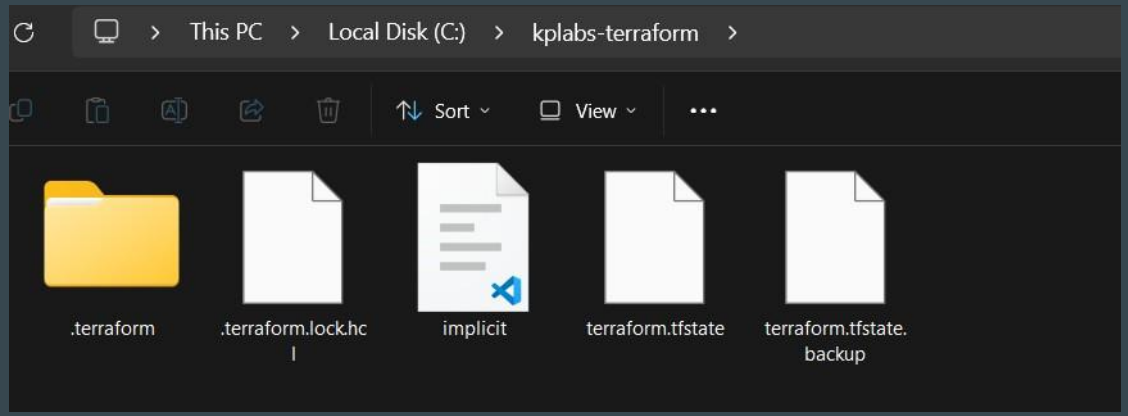
1. All Terraform code files
2. Your .terraform.lock.hcl dependency lock file
3. A .gitignore file that excludes the files
4. A README.md to describe the code, input variables, and outputs

Terraform Backends

Setting the Base

Backends primarily determine where Terraform stores its state file.

When you do not specify a backend block in your Terraform configuration, Terraform uses the default local backend that stores terraform.tfstate in the same project folder.



Challenges with Local Backend

Nowadays, Terraform projects are often handled collaboratively by entire teams.

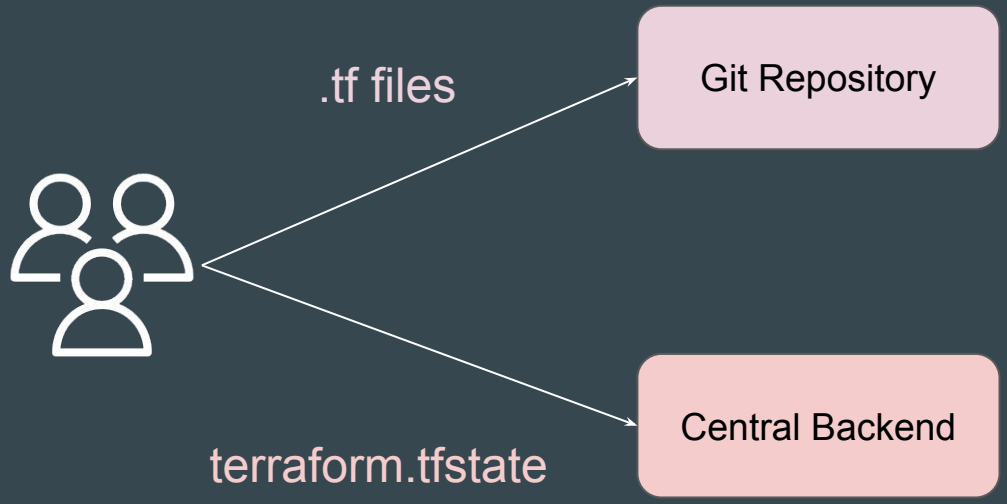
Storing the state file locally (e.g., on a developer's laptop) hinders effective team collaboration



Recommended Architecture

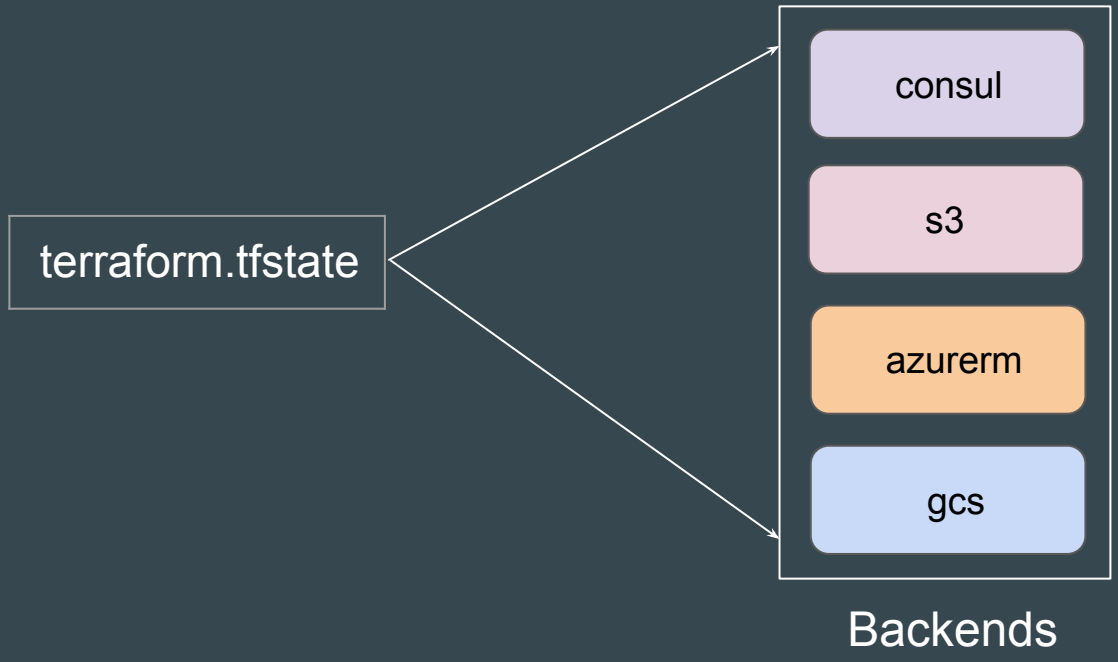
1. The Terraform Code is stored in a Git Repository.

2. The State file is stored in a Central backend.



Supported Backends in Terraform

Terraform supports wide variety of remote backends to store state file information.



Explicit Local Backend Configuration

The local backend stores state on the local filesystem, locks that state using system APIs, and performs operations locally.

```
terraform {
  backend "local" {
    path = "relative/path/to/terraform.tfstate"
  }
}
```

Important Note - Remote Backends

Accessing state in a remote service generally requires some kind of access credentials

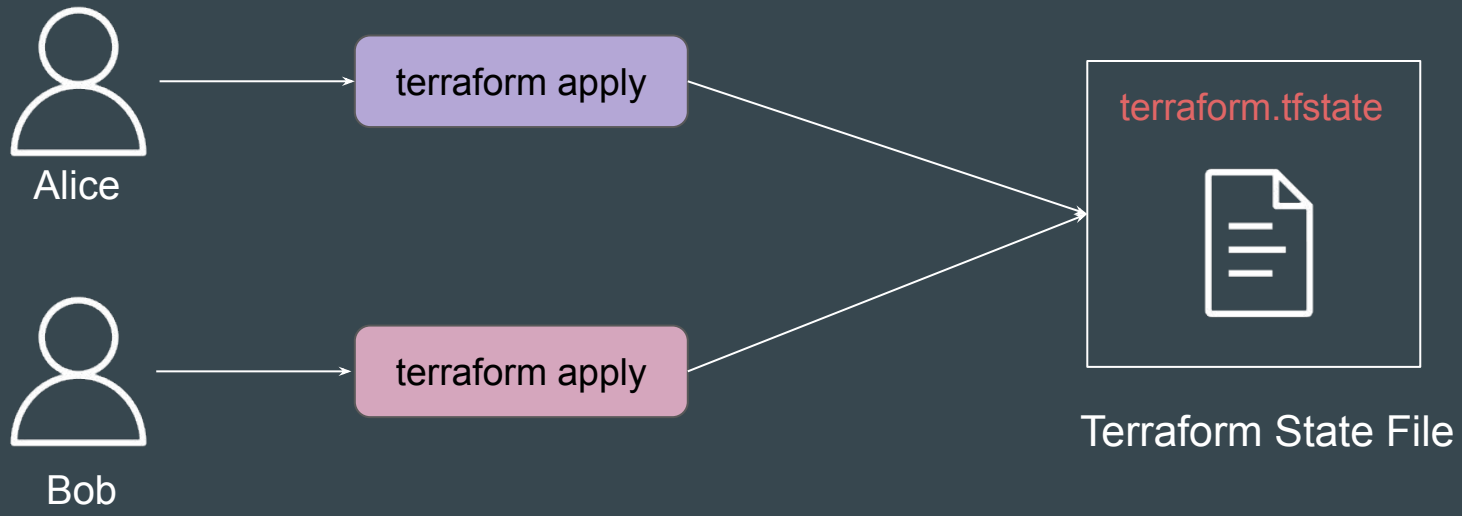
Some backends act like plain "remote disks" for state files; others support state locking functionality.



State Locking

Setting the Base

Running "terraform apply" simultaneously by multiple team members on same project can cause Terraform to make concurrent changes to the state file. This can result in state file corruption and inconsistencies.



Introducing State Locking

State locking is a mechanism that prevents multiple operations from making concurrent changes to your infrastructure state file, which could lead to corruption or inconsistent state.



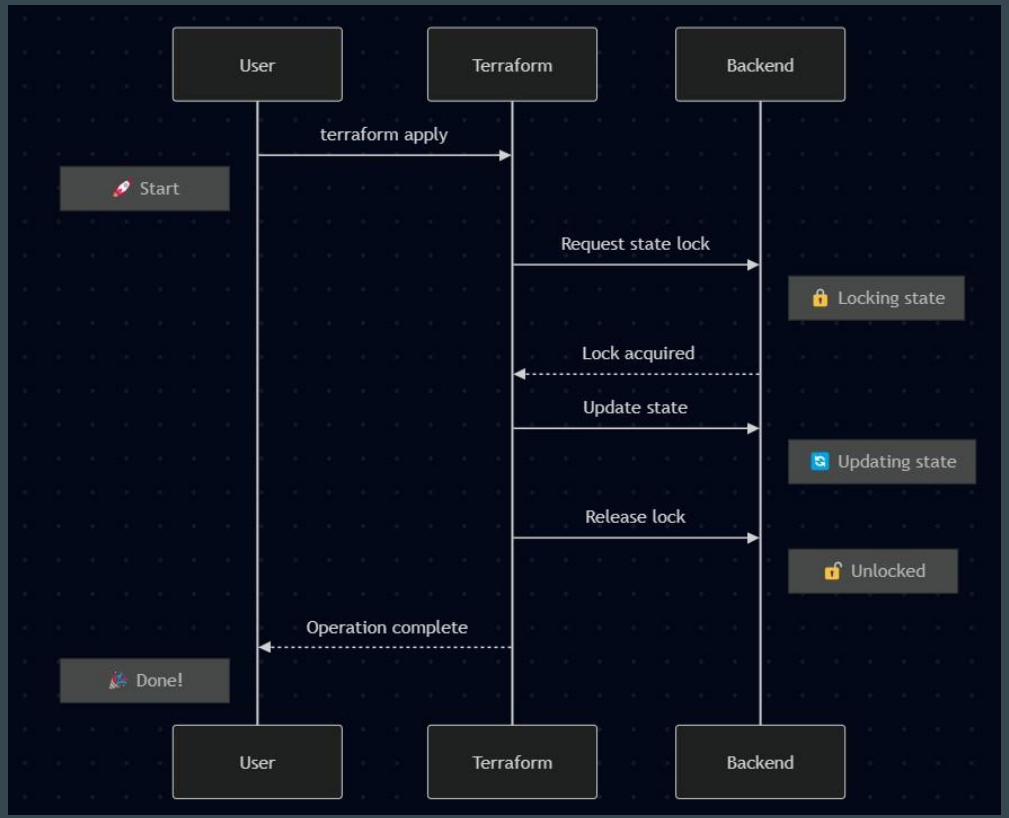
State File Locked

Process of State Locking

1. Before performing any write operation, Terraform attempts to acquire a "lock" on the state file.

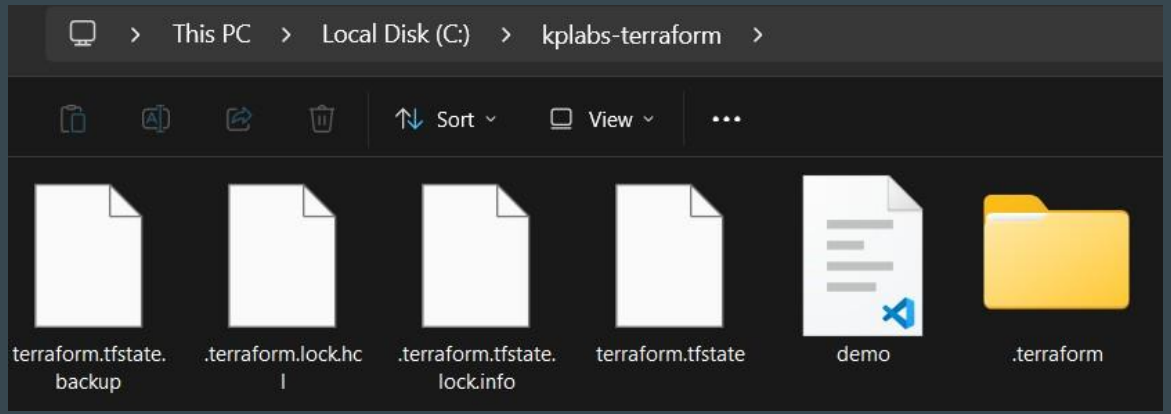
2. If the lock is successfully acquired, Terraform proceeds with the operation.

3. Once the operation is complete , Terraform releases the lock, allowing other processes to acquire it.


User
Terraform
Backend
terraform apply
Start
Request state lock
Locking state
Lock acquired
Update state
Updating state
Release lock
Unlocked
Operation complete
Done!
User
Terraform
Backend

State Locking with Local Backend

If using local state (local backend), Terraform uses a .lock file in the working directory for locking.



What Happens if state file is Locked?

When a state lock is held, any attempt by another process to operate will fail with an error.

```
C:\kplabs-terraform>terraform plan

  Error: Error acquiring the state lock

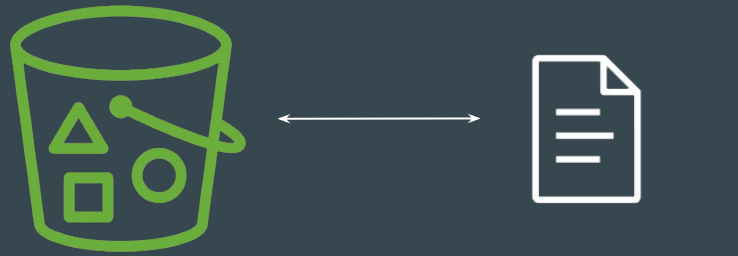
  Error message: Failed to read state file: The state file could not be read: read terraform.tfstate: The process cannot
  access the file because another process has locked a portion of the file.

  Terraform acquires a state lock to protect the state from being written
  by multiple users at the same time. Please resolve the issue above and try
  again. For most commands, you can disable locking with the "-lock=false"
  flag, but this is not recommended.
```

S3 Backend

Setting the Base

The S3 backend allows Terraform to store state file in a specific S3 bucket.



Example Configuration

Note: Terraform should have required access to the S3 Bucket to be able to push and pull state file to/from S3 bucket.

s3-backend.tf > ...

```
terraform {
  backend "s3" {
    bucket = "kplabs-tf-bucket"
    key    = "terraform.tfstate"
    region = "us-east-1"
    use_lockfile = true
  }
}
```

Additional Recommendation - Bucket Versioning

It is highly recommended that you enable Bucket Versioning on the S3 bucket to allow for state recovery in the case of accidental deletions and human error.

Terraform State Management

Setting the Base

As your Terraform usage becomes more advanced, there are some cases where you may need to modify the Terraform state.

It is NOT recommended to modify the state file manually.



State Management

The terraform state command is used for advanced state management

Sub-Commands	Description
list	List resources within terraform state file.
mv	Moves item with terraform state.
pull	Manually download and output the state from remote state.
push	Manually upload a local state file to remote state.
rm	Remove items from the Terraform state
show	Show the attributes of a single resource in the state.
replace-provider	Used to replace the provider for resources in a Terraform state.

Sub-Command 1 - List

The terraform state list command is used to list resources within a Terraform state.

Useful if you want to quickly view all resources managed by Terraform.

```
C:\kplabs-terraform>terraform state list
aws_iam_user.dev
aws_security_group.dev
```

Sub-Command 2 - Show

The terraform state show command is used to show the attributes of a single resource in the state.

Useful for debugging and understanding the current attributes of a resource.

```
C:\kplabs-terraform>terraform state show aws_iam_user.dev
# aws_iam_user.dev:
resource "aws_iam_user" "dev" {
    arn                  = "arn:aws:iam::042025557788:user/kplabs-user-01"
    force_destroy        = false
    id                   = "kplabs-user-01"
    name                 = "kplabs-user-01"
    path                 = "/"
    permissions_boundary = null
    tags                 = {}
    tags_all             = {}
    unique_id            = "AIDAQTSHLD4OALWEWB2AW"
}
```

Sub-Command 3 - pull

The terraform state pull command is used to pull the state from a remote backend and output it to stdout.

Useful to view or backup the current state stored in a remote backend.

```
C:\kplabs-terraform>terraform state pull
{
  "version": 4,
  "terraform_version": "1.9.1",
  "serial": 6,
  "lineage": "78f3ac79-1cb9-e724-964b-37bb3dc649a8",
  "outputs": {},
  "resources": [
    {
      "mode": "managed",
      "type": "aws_iam_user",
      "name": "dev",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        {
          "schema_version": 0,
          "attributes": {
            "arn": "arn:aws:iam::042025557788:user/kplabs-user-01",
            "force_destroy": false,
            "id": "kplabs-user-01",
            "name": "kplabs-user-01",
            "path": "/",
            "permissions_boundary": "",
```

Sub-Command 4 - rm

The terraform state rm command is used to remove items from the state.

Use this when you need to remove a resource from Terraform's state management without destroying it.

```
C:\kplabs-terraform>terraform state rm aws_security_group.dev
Removed aws_security_group.dev
Successfully removed 1 resource instance(s).
```

Sub-Command 5 - mv

The terraform state mv command is used to move an item in the state to a different address.

```
C:\kplabs-terraform>terraform state mv aws_security_group.dev  aws_security_group.prod
Move "aws_security_group.dev" to "aws_security_group.prod"
Successfully moved 1 object(s).
```

Sub-Command 6 - replace-provider

The terraform state replace-provider command is used to replace the provider for resources in a Terraform state.

```
C:\kplabs-terraform>terraform state replace-provider hashicorp/local kplabs.in/internal/local
Terraform will perform the following actions:

  ~ Updating provider:
    - registry.terraform.io/hashicorp/local
    + kplabs.in/internal/local

Changing 1 resources:

  local_file.foo

Do you want to make these changes?
Only 'yes' will be accepted to continue.

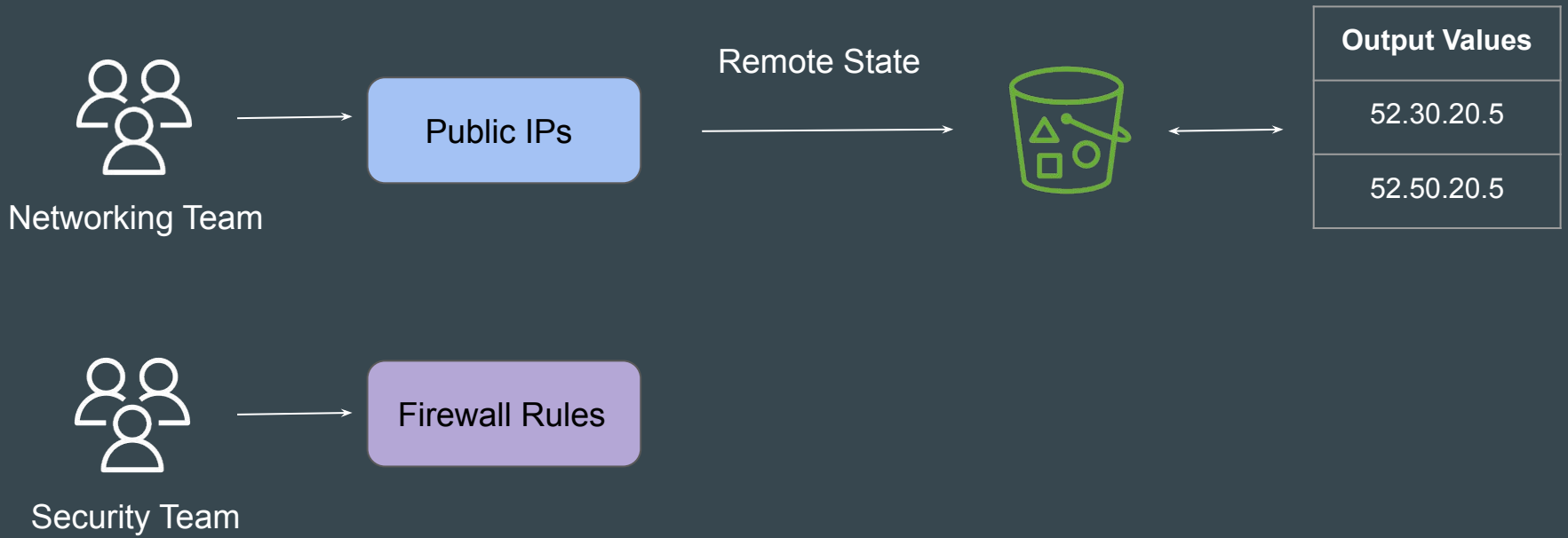
Enter a value: yes

Successfully replaced provider for 1 resources.
```

Remote State Data Source

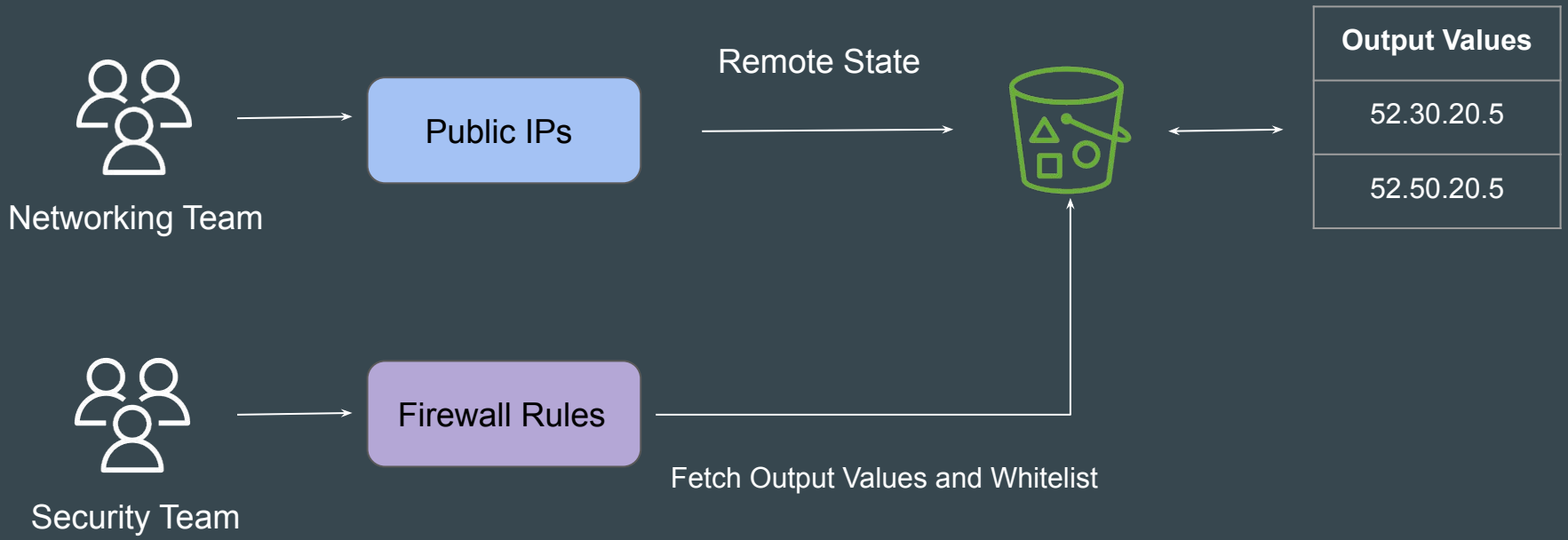
Setting up the Base

In larger enterprises, there can be multiple different teams working on different aspects of a infrastructure resource



Understanding the Challenge

Security Team wants that all the IP addresses added as part of Output Values in tfstate file of Networking Team project should be whitelisted in Firewall.



What Needs to be Achieved

1. The code from Security Team project should connect to the terraform.tfstate file managed by the Networking team.

2. The code should fetch all the IP addresses mentioned in the output values in the state file.

3. The code should whitelist these IP addresses in Firewall rules.

Practical Workflow Steps

1. Create two folders for networking-team and security-team

2. Create Elastic IP resource in Networking Team and Store the State file in S3 bucket. Output values should have information of EIP.

3. In Security Team, use Terraform Remote State data source to connect to the tfstate file of Networking Team.

4. Use the Remote State to fetch EIP and whitelist it in Security Group rule.

Introducing Remote State Data Source

The terraform_remote_state data source allows us to fetch output values from a specific state backend

```
data "terraform_remote_state" "eip" {
  backend = "s3"
  config = {
    bucket = "kplabs-team-networking-bucket"
    key    = "eip.tfstate"
    region = "us-east-1"
  }
}
```

Step 1 - Define Remote State Source

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "${data.terraform_remote_state.eip.outputs.eip_addr}/32"
  from_port         = 443
  ip_protocol       = "tcp"
  to_port           = 443
}
```

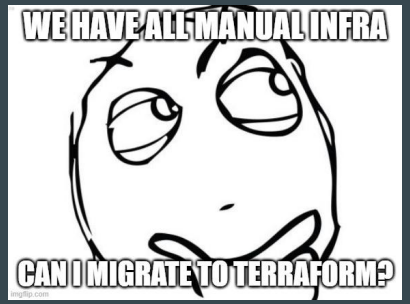
Step 2 - Define Data to Fetch

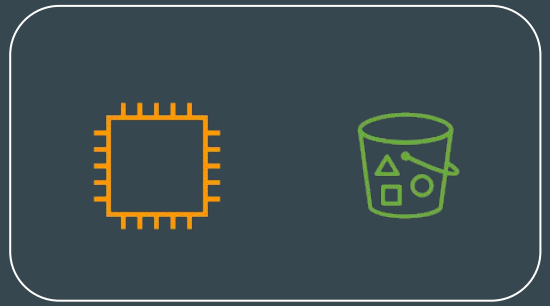
Terraform Import

Typical Challenge

It can happen that all the resources in an organization are created manually.

Organization now wants to start using Terraform and manage these resources via Terraform.



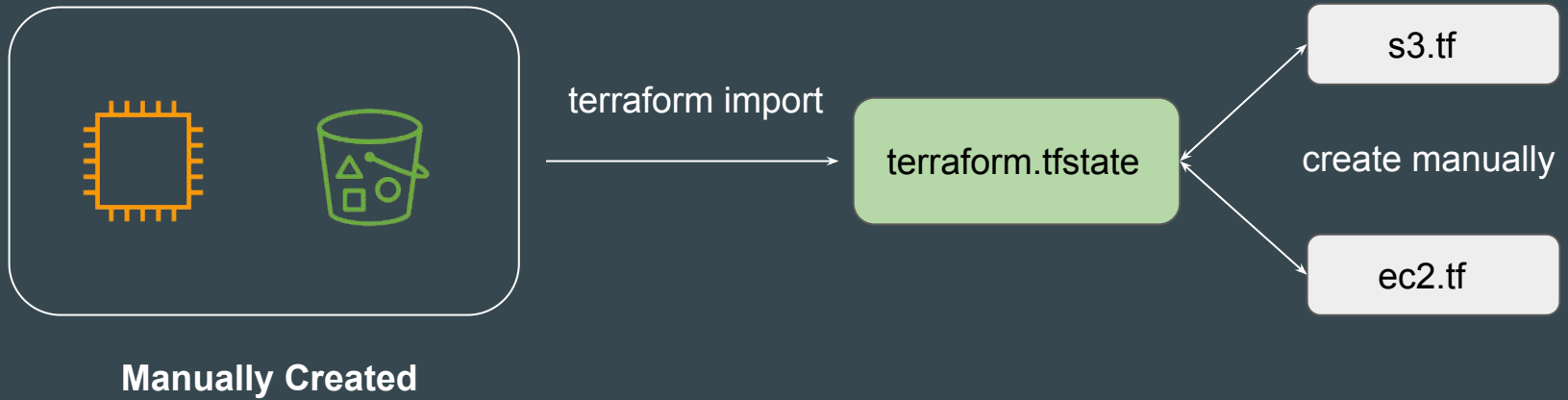


Manually Created

Earlier Approach

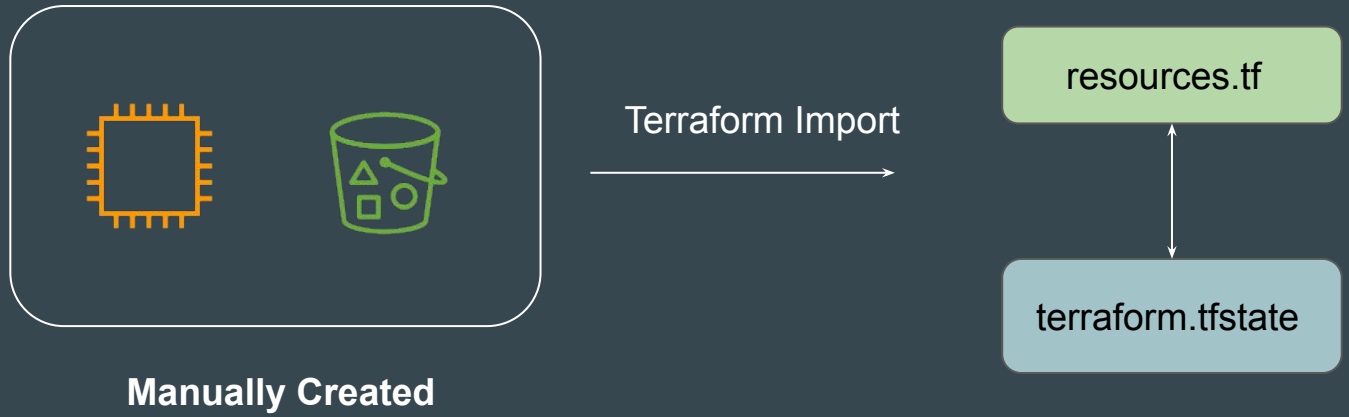
In the older approach, Terraform import would create the state file associated with the resources running in your environment.

Users still had to write the tf files from scratch.



Newer Approach

In the newer approach, terraform import can automatically create the terraform configuration files for the resources you want to import.



Point to Note

Terraform 1.5 introduces automatic code generation for imported resources.

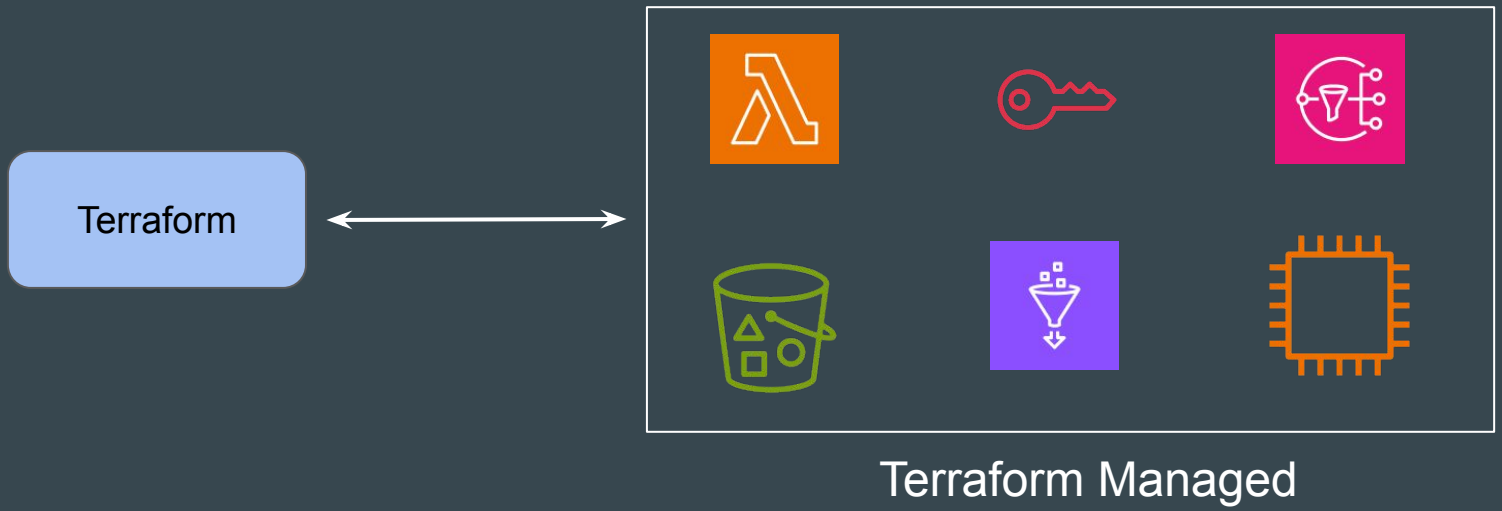
This dramatically reduces the amount of time you need to spend writing code to match the imported

This feature is not available in the older version of Terraform.

Removed Block

Setting the Base

Imagine you have an EC2 instance defined in your Terraform code. You decide you still want the EC2 instance to exist in AWS, but you no longer want Terraform to manage it (perhaps another team is taking over)



The Old Workflow

Step 1	You would delete the resource "aws_instance" "example" { ... } block from your .tf file.
Step 2	You had to run a CLI command: terraform state rm aws_instance.example

Disadvantage of Old Workflow

Problems	Description
Imperative, not Declarative	This is a manual command run by a human. It is not recorded in your code (Git).
CI/CD Issues	Hard to automate safely as part of CI/CD pipeline

Introducing Removed Block

The removed block allows you to declare your intention to remove a resource from the Terraform state.

It turns the imperative terraform state rm command into a declarative configuration block.

```
/*
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
}
*/

removed {
  from = aws_instance.myec2

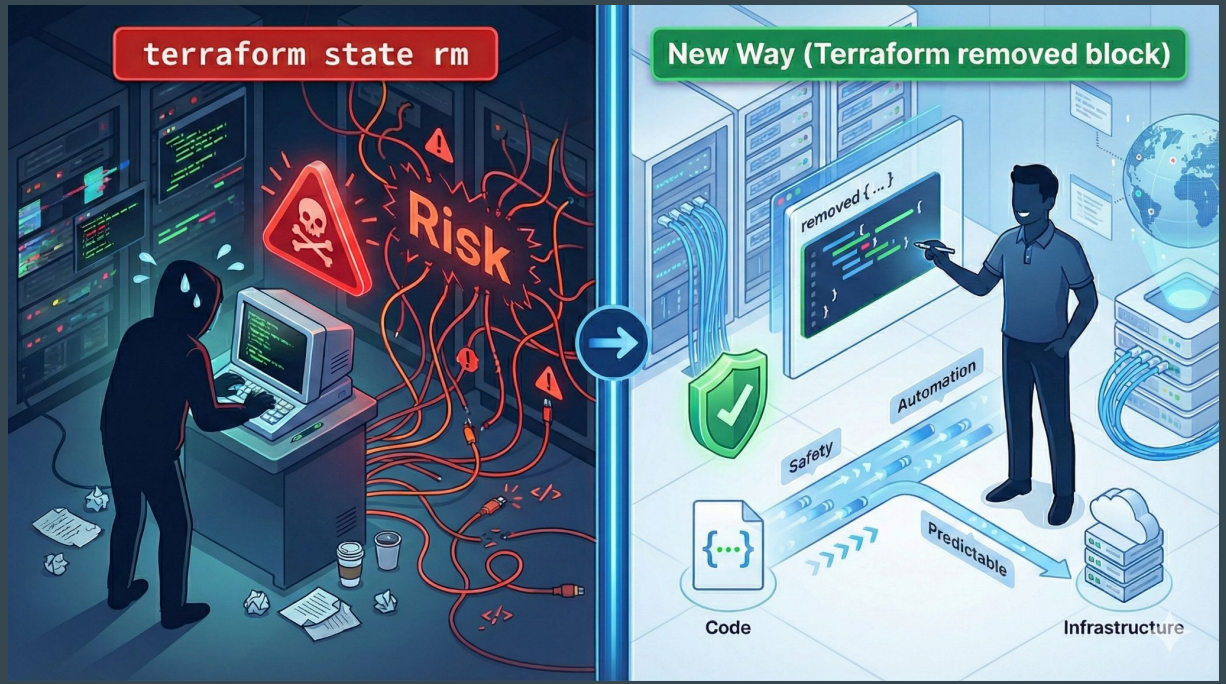
  lifecycle {
    destroy = false
  }
}
```

The Workflow

1. When you run terraform plan, Terraform notices the removed block with destroy=false

2. Terraform will plan a ‘forget’ action, and on apply it will remove the object from state.

3. The actual EC2 instance in AWS remains untouched.


terraform state rm
Risk
New Way (Terraform removed block)
removed { ... }
Automation
Safety
Predictable
Code
Infrastructure

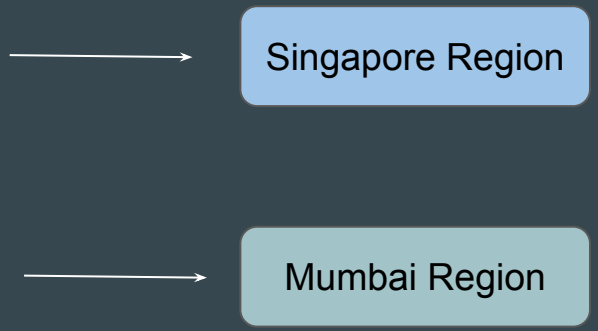
Multiple Provider Configuration

Understanding the Requirement

There can be multiple resource types in same project and you want to deploy them in different set of AWS regions.

```
resource "aws_instance" "myec2" {
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}

resource "aws_security_group" "allow_tls" {
  name        = "staging_firewall"
}
```



Setting the Base

At this stage, we have been dealing with single provider configuration.

In below code, both resources will be created in Singapore region.

```
provider "aws" {
    region = "ap-southeast-1"
}

resource "aws_instance" "myec2" {
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}

resource "aws_security_group" "allow_tls" {
  name        = "staging_firewall"
}
```

Alias Meta-Argument

Each provider can have one default configuration, and any number of alternate configurations that include an extra name segment (or "alias").

```
resource "aws_instance" "myec2" {
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}
```



```
resource "aws_security_group" "allow_tls" {
  name        = "staging_firewall"
}
```



```
provider "aws" {
    region = "ap-southeast-1"
}

provider "aws" {
    alias  = "mumbai"
    region = "ap-south-1"
}

provider "aws" {
    alias = "usa"
    region = "us-east-1"
}
```

Final Output Using Alias

By using the provider meta-argument, you can select an alternate provider configuration for a resource.

```
provider "aws" {
    region = "ap-southeast-1"
}

provider "aws" {
    alias  = "mumbai"
    region = "ap-south-1"
}

provider "aws" {
    alias = "usa"
    region = "us-east-1"
}
```

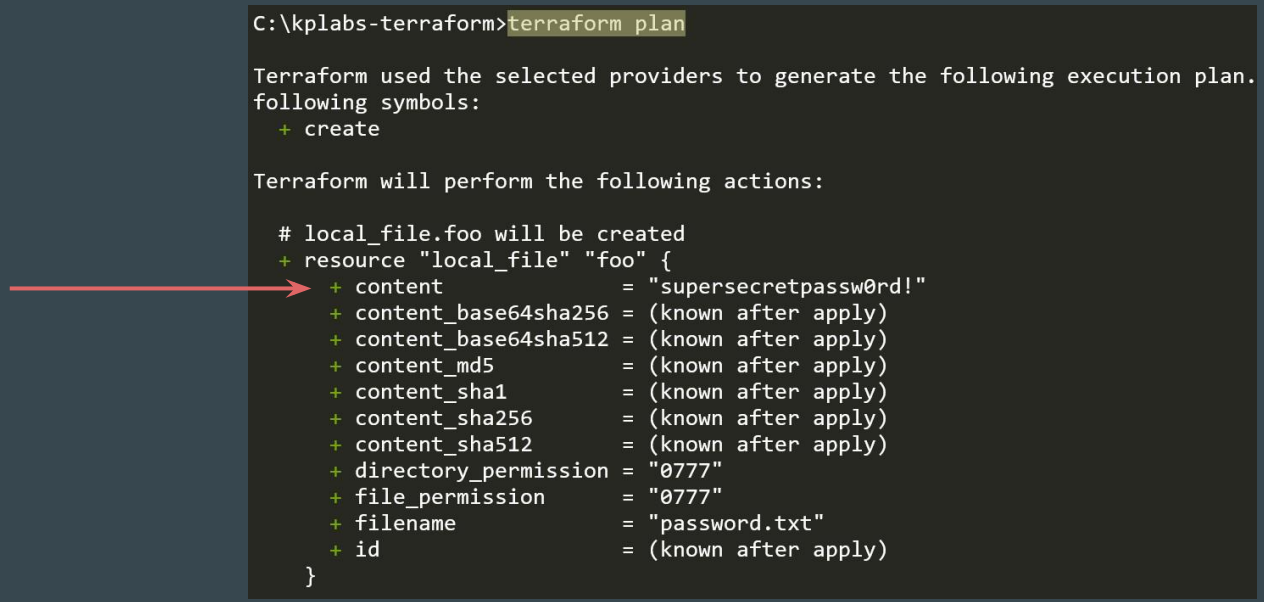
```
resource "aws_instance" "myec2" {
  provider = aws.mumbai
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}
```

```
resource "aws_security_group" "allow_tls" {
  provider    =  aws.usa
  name        = "staging_firewall"
}
```

Sensitive Parameter

Setting the Base

By default, Terraform will show the values associated with defined attributes in the CLI output during plan, apply operations for most of the resources.



What to Expect

We should design our Terraform code in such way that no sensitive information is available and shown out of the box in CLI Output, Logs, etc.



Basics of Sensitive Parameter

Adding sensitive parameter ensures that you do not accidentally expose this data in CLI output, log output

```
variable "sensitive_content" {
  sensitive = true
  default = "supersecretpassw0rd"
}

resource "local_file" "foo" {
  content  = var.sensitive_content
  filename = "password.txt"
}
```



```
Terraform will perform the following actions:

  # local_file.foo will be created
  + resource "local_file" "foo" {
      + content              = (sensitive value)
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "password.txt"
      + id                   = (known after apply)
    }
```

Sensitive Values AND Output Values

If you try to reference sensitive value in output values, Terraform will immediately give you an error.

```
variable "sensitive_content" {
  sensitive = true
  default = "supersecretpassw0rd"
}

resource "local_file" "foo" {
  content  = var.sensitive_content
  filename = "password.txt"
}

output "content" {
    value = local_file.foo.content
}
```



```
Error: Output refers to sensitive values

  on local_file.tf line 12:
  12: output "content" {

To reduce the risk of accidentally exporting sensitive data that
any root module output containing sensitive data be explicitly m

If you do intend to export this data, annotate the output value
    sensitive = true
```

Sensitive Values AND Output Values

If you still want sensitive content to be available in “output” of state file but should not be visible in CLI Output, Logs, following approach can be used.

```
variable "sensitive_content" {
  sensitive = true
  default = "supersecretpassw0rd"
}

resource "local_file" "foo" {
  content  = var.sensitive_content
  filename = "password.txt"
}

output "content" {
    value = local_file.foo.content
    sensitive = "true"
}
```



```
Changes to Outputs:
  + content = (sensitive value)

You can apply this plan to save these new output values to the Terraform state

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

content = <sensitive>
```

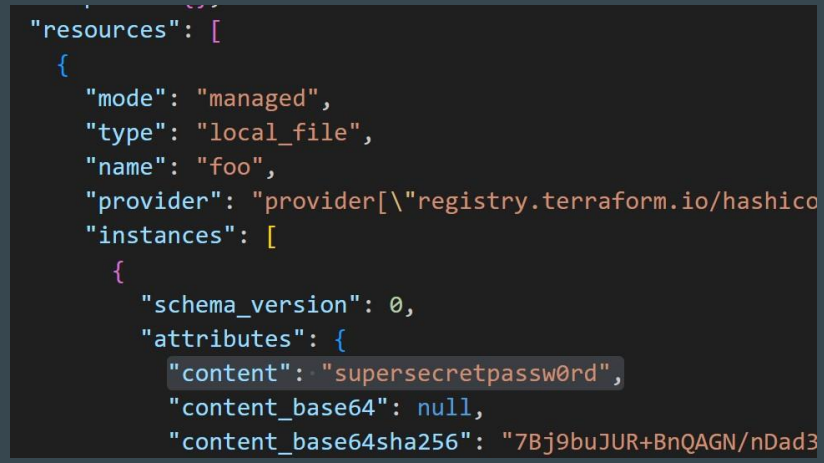
Important Point to Note

Sensitive parameter will NOT protect / redact information from State file.

```
variable "sensitive_content" {
  sensitive = true
  default = "supersecretpassw0rd"
}

resource "local_file" "foo" {
  content  = var.sensitive_content
  filename = "password.txt"
}
```

Configuration File



State File

Benefits of Mature Providers

Various providers like AWS will automatically considers the password argument for any database instance as sensitive and will redact it as a sensitive value

```
resource "aws_db_instance" "default" {
  allocated_storage    = 10
  db_name              = "mydb"
  engine               = "mysql"
  engine_version       = "8.0"
  instance_class       = "db.t3.micro"
  username             = "foo"
  password             = "foobarbaz"
  parameter_group_name = "default.mysql8.0"
  skip_final_snapshot  = true
}
```



```
C:\kplabs-terraform>terraform apply -auto-approve

Terraform will perform the following actions:

  # aws_db_instance.default will be created
  + resource "aws_db_instance" "default" {
      + address                               = (known after apply)
      + allocated_storage                     = 10
      + password                              = (sensitive value)
      + performance_insights_enabled          = false
      + performance_insights_kms_key_id       = (known after apply)
      + performance_insights_retention_period = (known after apply)
      + port                                  = (known after apply)
```

Overview of Vault

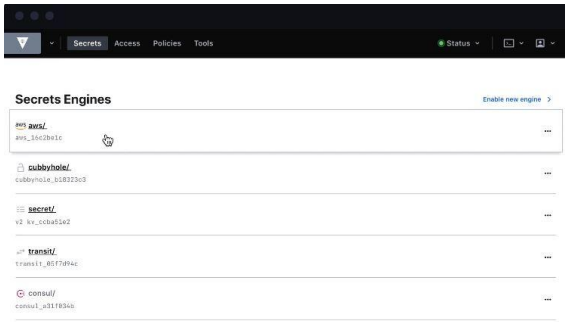
HashiCorp Certified: Vault Associate

Let’s get started

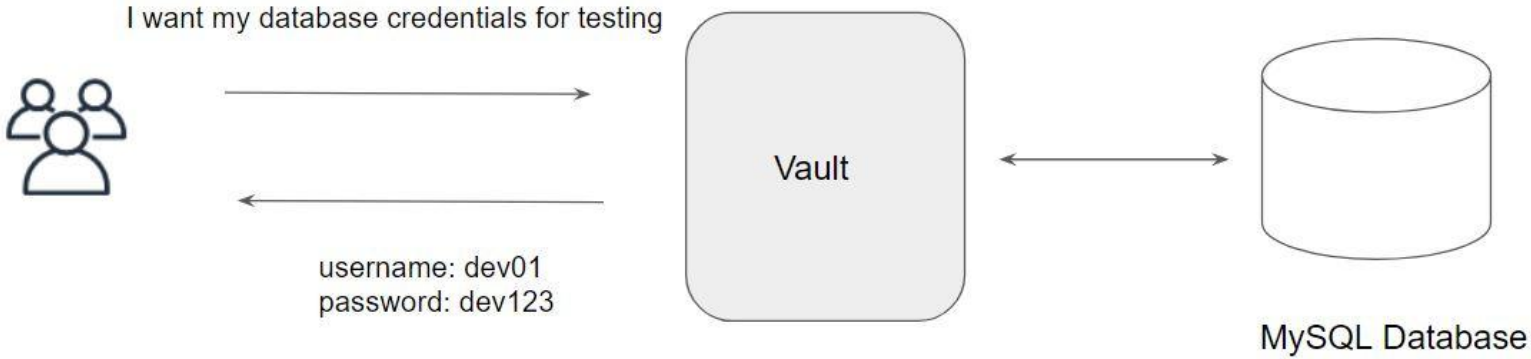
HashiCorp Vault allows organizations to securely store secrets like tokens, passwords, certificates along with access management for protecting secrets.

One of the common challenges nowadays in an organization is “Secrets Management”

Secrets can include, database passwords, AWS access/secret keys, API Tokens, encryption keys and others.



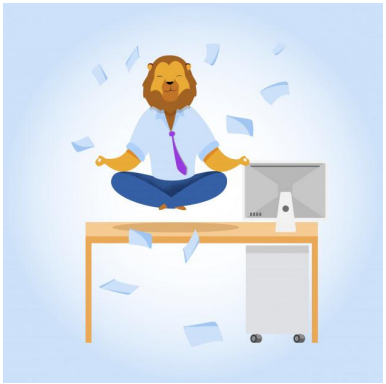
Dynamic Secrets



Life Becomes Easier

Once Vault is integrated with multiple backends, your life will become much easier and you can focus more on the right work.

Major aspect related to Access Management can be taken over by vault.

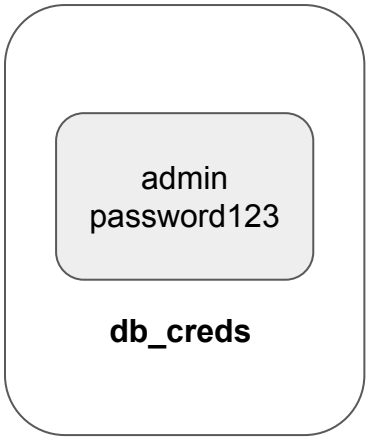


Vault Provider

Back to Providers

Vault Provider

The Vault provider allows Terraform to read from, write to, and configure HashiCorp Vault.



Inject in Terraform

```
provider "vault" {
  address = "http://127.0.0.1:8200"
}

data "vault_generic_secret" "demo" {
  path = "secret/db-creds"
}
```

Important Note

Interacting with Vault from Terraform causes any secrets that you read and write to be persisted in both Terraform's state file.

Dependency Lock File

Revising the Basics

Provider Plugins and Terraform are managed independently and have different release cycle.



Understanding the Challenge

The AWS code written in Terraform is working perfectly well with AWS Plugin v1

It can happen that same code might have some issues with newer AWS plugins.



Version Dependencies

Version constraints within the configuration itself determine which versions of dependencies are potentially compatible.

After selecting a specific version of each dependency Terraform remembers the decisions it made in a dependency lock file so that it can (by default) make the same decisions again in future.

.terraform.lock.hcl

```
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version     = "4.62.0"
  constraints = "~> 4.0"
  hashes = [
    "h1:TBuxeLRMrChrzgDhAyil3HITJhuNrL+X58QRC+FTmFQ=",
    "zh:12059dc2b639797b9facb6397ac6aec563891634be8e5aadf3a4
    "zh:1b3515d70b6998359d0a6d3b3c287940ab2e5c59cd02f95c7d9d
```



demo.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 4.0"
    }
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "us-east-1"
}
```

Default Behaviour

What happens if you update the TF file with version that does not match the terraform.lock.hcl?

.terraform.lock.hcl

```
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version     = "4.62.0"
  constraints = "~> 4.0"
  hashes = [
    "h1:TBuxeLRMrChrzgDhAyil3HITJhuNrL+X58QRC+FTmFQ=",
    "zh:12059dc2b639797b9facb6397ac6aec563891634be8e5aadf3a4
    "zh:1b3515d70b6998359d0a6d3b3c287940ab2e5c59cd02f95c7d9d
```

demo.tf > ...

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "4.60"
    }
  }
}

# Configure the AWS Provider
provider "aws" {
  region = "us-east-1"
}
```

```
C:\Users\zealv\Desktop\kplabs-terraform>terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file

Error: Failed to query available provider packages

Could not retrieve the list of available versions for provider hashicorp/aws: l
configured version constraint 4.60.0; must use terraform init -upgrade to allow
```

Upgrading Option

If there is a requirement to use newer or downgrade a provider, can override that behavior by adding the -upgrade option when you run terraform init, in which case Terraform will disregard the existing selections

```
C:\Users\zealv\Desktop\kplabs-terraform>terraform init -upgrade

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "4.60.0"...
- Installing hashicorp/aws v4.60.0...
- Installed hashicorp/aws v4.60.0 (signed by HashiCorp)

Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.
```

Points to Note

When installing a particular provider for the first time, Terraform will pre-populate the hashes value with any checksums that are covered by the provider developer's cryptographic signature, which usually covers all of the available packages for that provider version across all supported platforms.

.terraform.lock.hcl

```
# This file is maintained automatically by "terraform init".
# Manual edits may be lost in future updates.

provider "registry.terraform.io/hashicorp/aws" {
  version     = "4.60.0"
  constraints = "4.60.0"
  hashes = [
    "h1:M9Oxusbiz/HW7zF+jLddXqdpzsfZ38Fa2S+Yaquad2g=",
    "zh:1853d6bc89e289ac36c13485e8ff877c1be8485e22f545bb32c7a30f1d1856e8",
    "zh:4321d145969e3b7ede62fe51bee248a15fe398643f21df9541eef85526bf3641",
    "zh:4c01189cc6963abfe724e6b289a7c06d2de9c395011d8d54efa8fe1aac444e2e",
    "zh:5934db7baa2eec0f9acb9c7f1c3dd3b3fe1e67e23dd4a49e9fe327832967b32b",
```

Points to Note

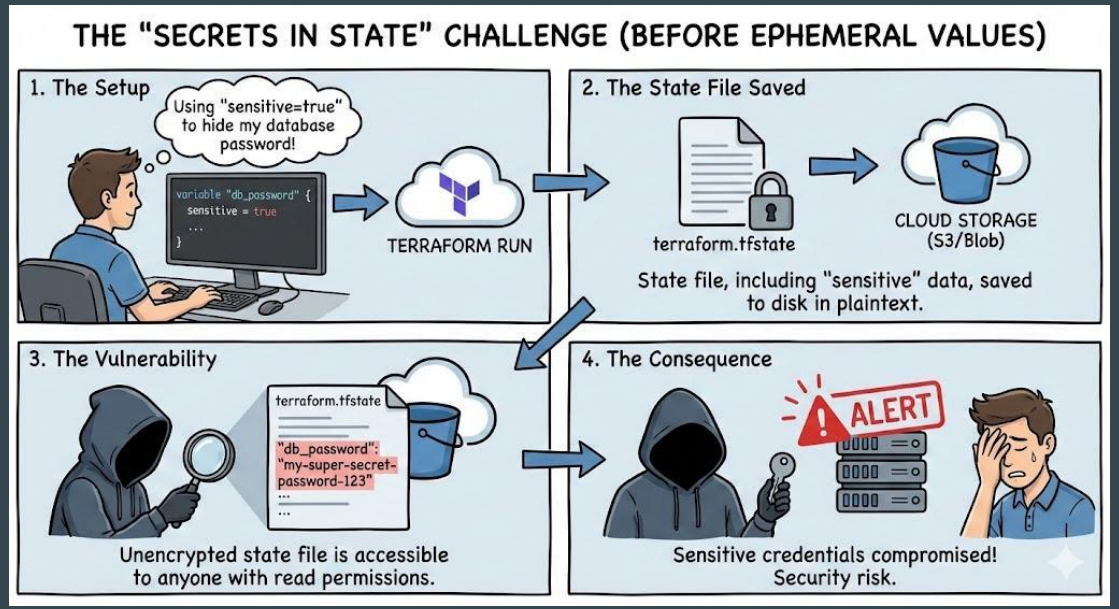
At present, the dependency lock file tracks only provider dependencies.

Terraform does not remember version selections for remote modules, and so Terraform will always select the newest available module version that meets the specified version constraints.

Ephemeral Values and Write-Only Arguments

Understanding the Challenge

By adding sensitive = true for sensitive data like passwords only redacted the value from the CLI output. The actual value was still written in plaintext (JSON) inside the terraform.tfstate file



Ephemeral Blocks

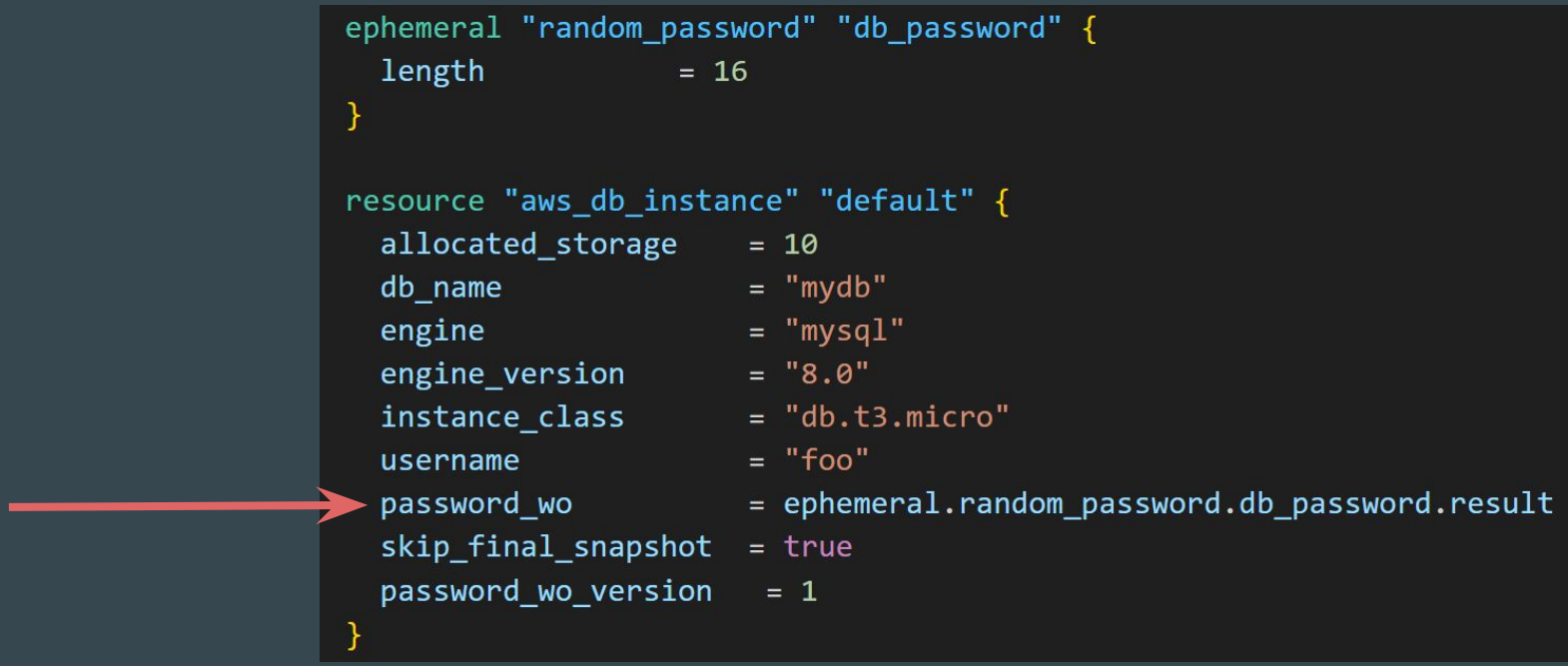
The ephemeral block defines resources that are essentially temporary.

Ephemeral resources have a unique lifecycle, and Terraform does not store information about ephemeral resources in state or plan files

```
ephemeral "random_password" "db_password" {
  length           = 16
}
```

Write-only arguments

Write-only arguments let you securely pass temporary values to Terraform's managed resources during an operation without persisting those values to state or plan files.



```
ephemeral "random_password" "db_password" {
  length           = 16
}

resource "aws_db_instance" "default" {
  allocated_storage    = 10
  db_name              = "mydb"
  engine               = "mysql"
  engine_version       = "8.0"
  instance_class       = "db.t3.micro"
  username             = "foo"
  password_wo          = ephemeral.random_password.db_password.result
  skip_final_snapshot  = true
  password_wo_version   = 1
}
```

Points to Note - Part 1

To use write-only arguments, you must use Terraform v.1.11 or later and use a resource that supports write-only arguments.

write-only arguments accept both ephemeral and non-ephemeral values.

Points to Note - Part 2

Terraform does not store write-only arguments in state files, so Terraform has no way of knowing if a write-only argument value has changed.

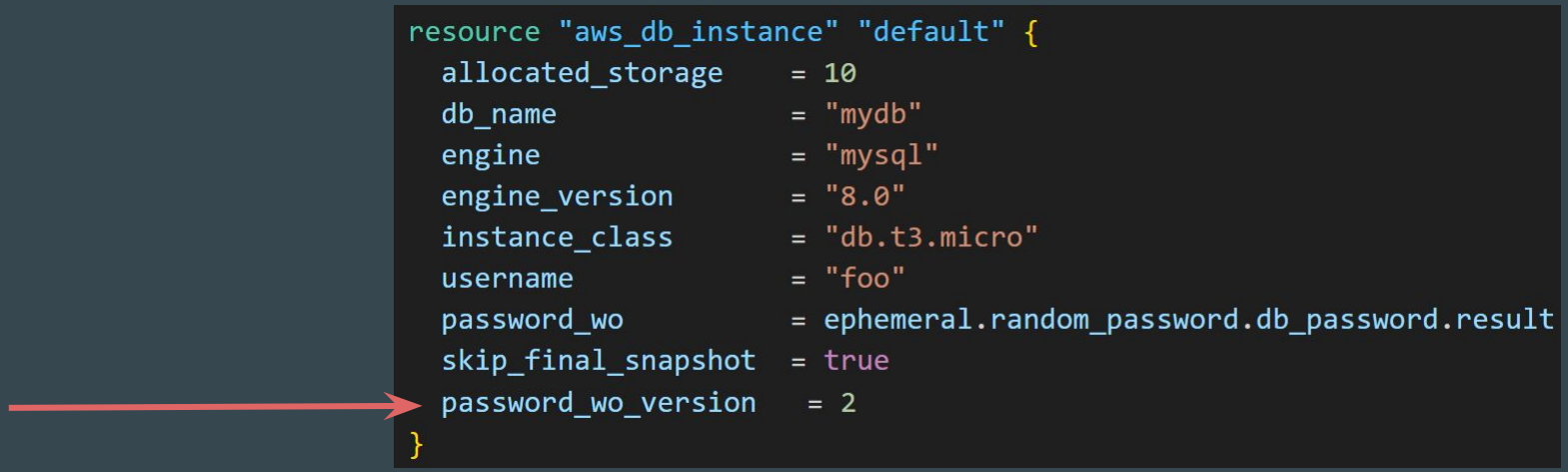
Terraform also cannot create plan diffs for write-only arguments because it does not store those values in plan files.

Providers typically include version arguments alongside write-only arguments. Terraform stores version arguments in state, and can track if a version argument changes.

Trigger an Update of Write-only Argument

To trigger an update of a write-only argument, increment the version argument's value in your configuration:

When you increment the password_wo_version argument, Terraform notices that change in its plan and notifies the aws provider. The aws provider then uses the new password_wo value to update the aws_db_instance resource.

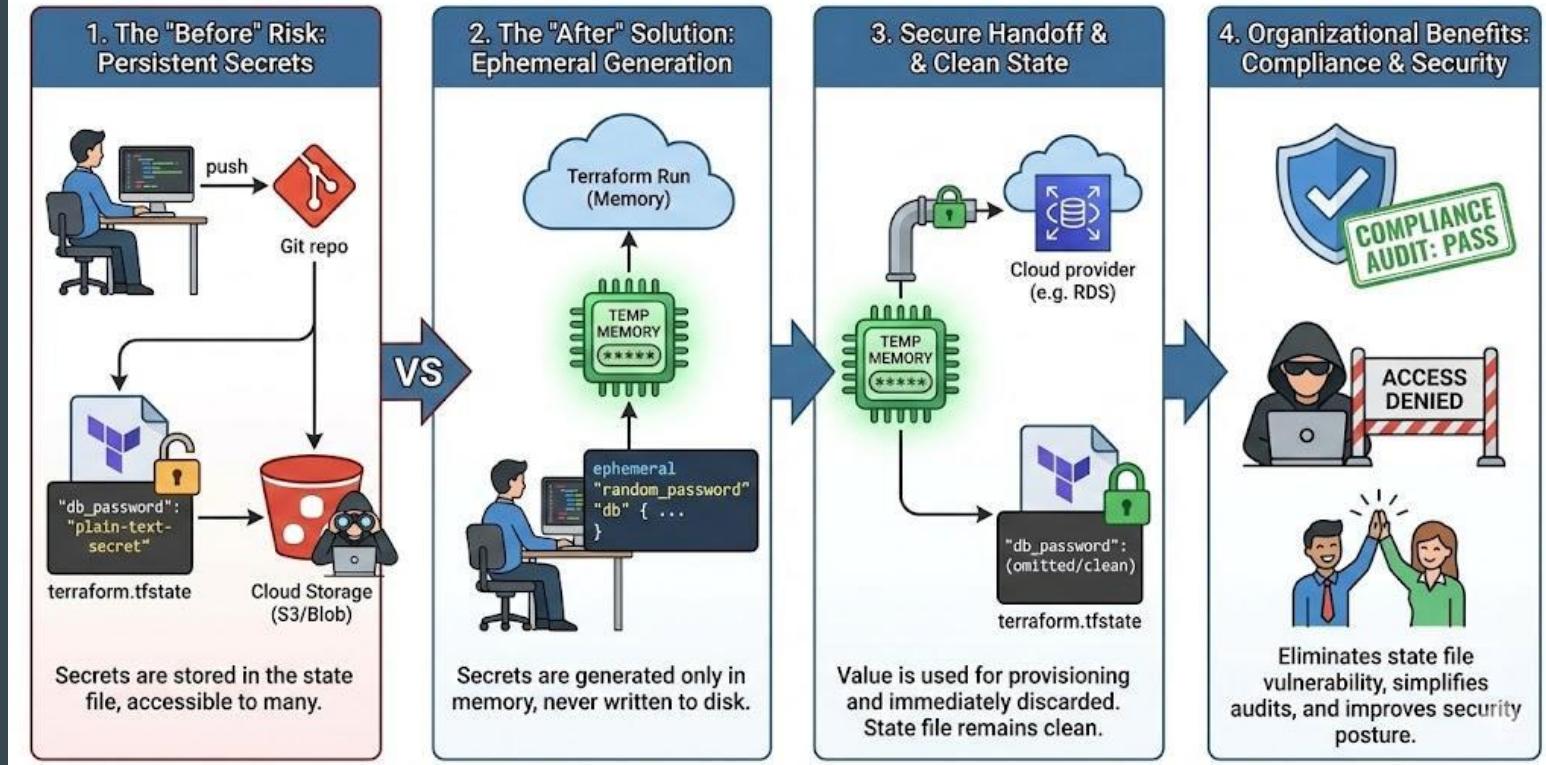


```
resource "aws_db_instance" "default" {
  allocated_storage    = 10
  db_name              = "mydb"
  engine               = "mysql"
  engine_version       = "8.0"
  instance_class       = "db.t3.micro"
  username             = "foo"
  password_wo          = ephemeral.random_password.db_password.result
  skip_final_snapshot  = true
  password_wo_version   = 2
}
```

Points to Note

The implementation of write-only arguments and their version arguments is provider-specific, so consult the Registry for more details about your specific provider.

HOW EPHEMERAL VALUES SECURE YOUR INFRASTRUCTURE (ORGANIZATIONAL BENEFITS)



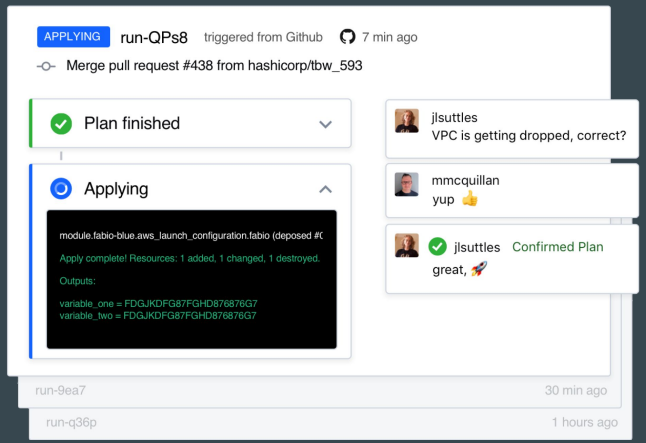
HashiCorp Cloud Platform - Terraform

Setting the Base

Till now, we have been managing Terraform through CLI

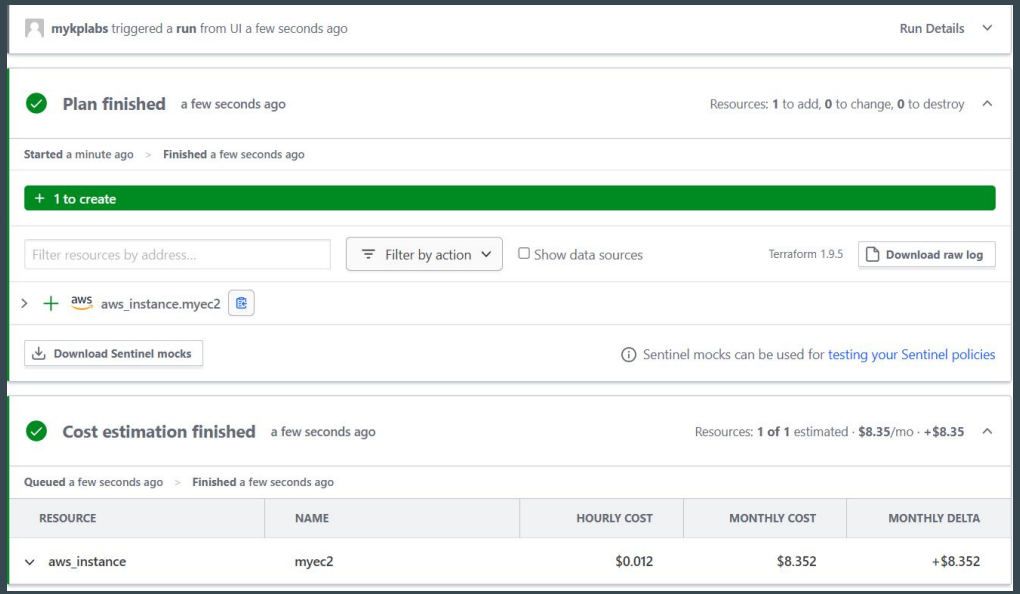
Although the CLI approach works well, it's important to remember that there is also a GUI-based alternative also available through HCP Terraform.

```
root@test-kplabs:~# terraform plan
2024-08-24T05:01:48.844Z [INFO]  Terraform version: 1.9.5
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/go-tfe v1.58.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/hcl/v2 v2.20.0
2024-08-24T05:01:48.845Z [DEBUG] using github.com/hashicorp/terraform-svchost v0.1.1
2024-08-24T05:01:48.845Z [DEBUG] using github.com/zclconf/go-cty v1.14.4
2024-08-24T05:01:48.845Z [INFO]  Go runtime version: go1.22.5
2024-08-24T05:01:48.845Z [INFO]  CLI args: []string{"terraform", "plan"}
2024-08-24T05:01:48.845Z [TRACE] Stdout is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stderr is a terminal of width 125
2024-08-24T05:01:48.845Z [TRACE] Stdin is a terminal
2024-08-24T05:01:48.845Z [DEBUG] Attempting to open CLI config file: /root/.terraformrc
2024-08-24T05:01:48.845Z [DEBUG] File doesn't exist, but doesn't need to. Ignoring.
2024-08-24T05:01:48.845Z [DEBUG] ignoring non-existing provider search directory terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.terraform.d/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /root/.local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/local/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /usr/share/terraform/plugins
2024-08-24T05:01:48.846Z [DEBUG] ignoring non-existing provider search directory /var/lib/snapd/desktop/terraform/plugins
2024-08-24T05:01:48.848Z [INFO]  CLI command args: []string{"plan"}
```



HCP Terraform

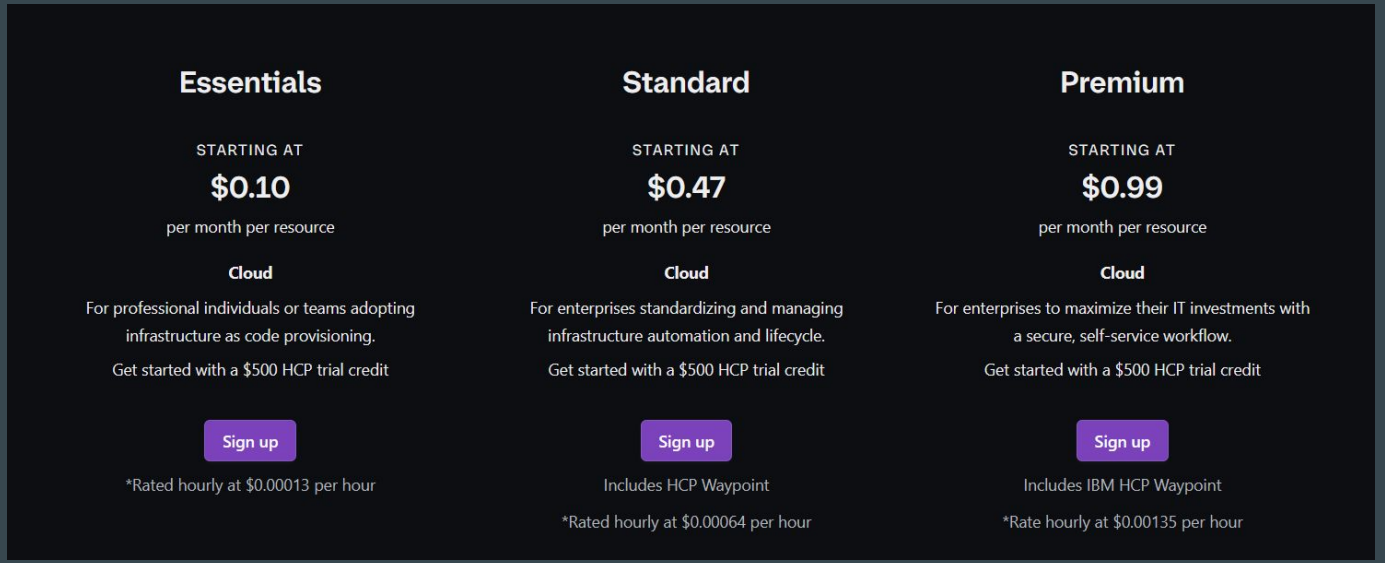
HCP Terraform manages Terraform runs in a consistent and reliable environment with various features like access controls, private registry for sharing modules, policy controls and others.



HCP Terraform - Pricing

Not Everything is Free

HCP Terraform is not entirely free. Depending on the usage and features needed, there are multiple pricing plans that are available.



Important Features Availability

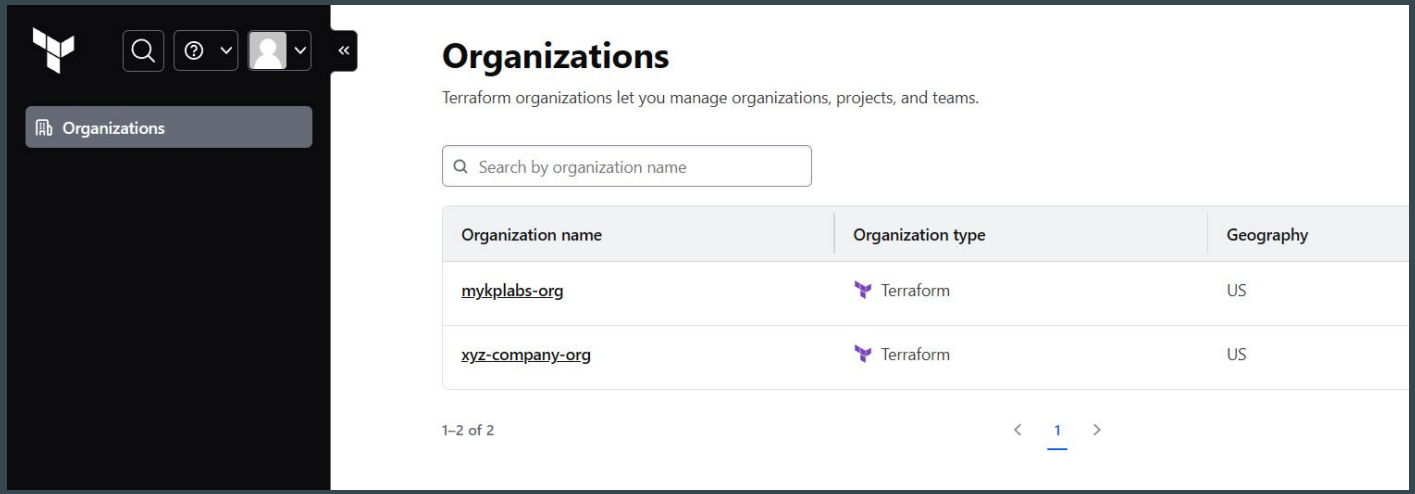
Feature	Essential	Standard	Premium	Enterprise
Audit Logging	No	Yes	Yes	Yes
Drift Detection	No	Yes	Yes	Yes
Continuous validation	No	Yes	Yes	Yes
Policy as code (Sentinel and OPA)	1 Policy set 5 Policies	Unlimited	Unlimited	Yes
Air Gap Installation	No	No	No	Yes
Audit Trails API	No	Yes	Yes	No

Pay-as-You-Go (PAYG)	On-demand, usage-based billing with no long-term commitment
Flex	Single and multi-year plans available with preferred pricing for maximum ROI
Enterprise Self Managed	Fully customizable plans for maximum control

HCP Terraform - Basic Structure

1 - Organizations

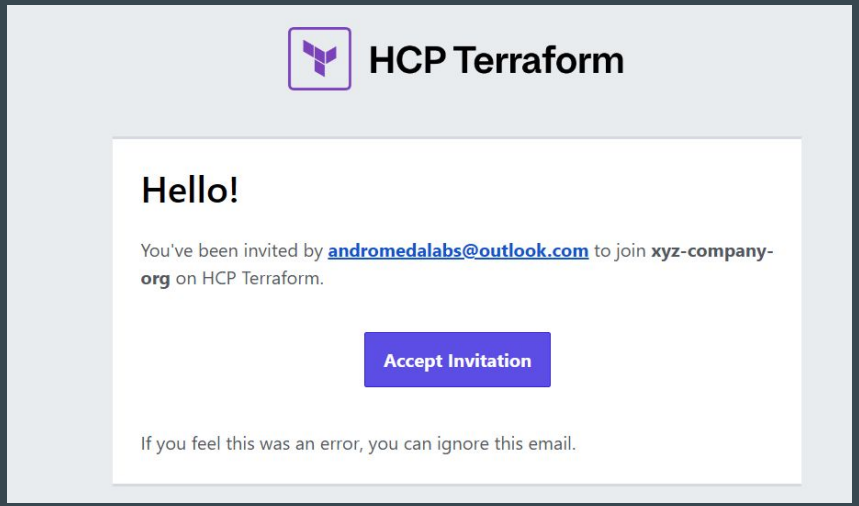
Organizations are a shared space for one or more teams to collaborate on workspaces.



Points to Note - Organizations

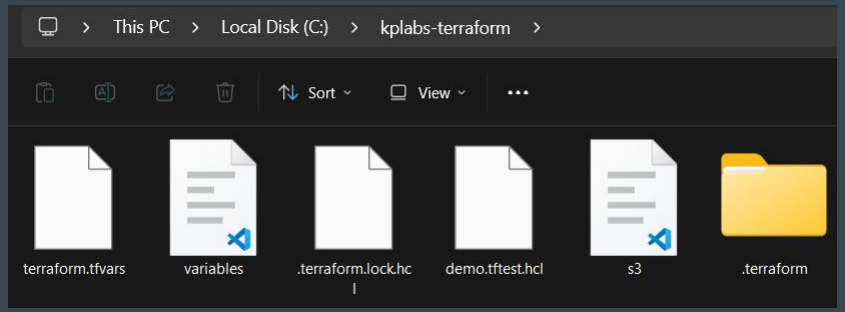
HCP Terraform manages plans and billing at the organization level.

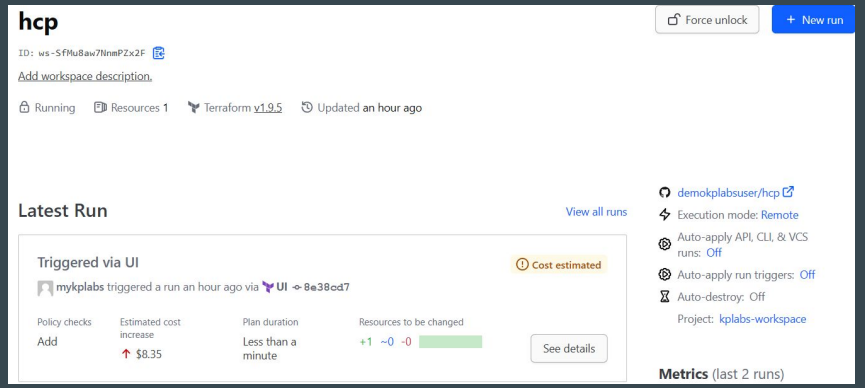
Each HCP Terraform user can belong to multiple organizations, which might subscribe to different billing plans.



2 - Workspace

HCP Terraform manages infrastructure collections with workspaces instead of directories



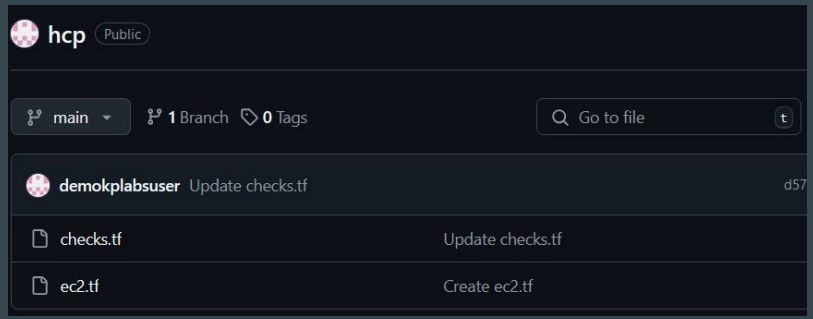


Workspace & Configuration Files

The Terraform configuration file (sample.tf) is not directly uploaded to a workspace.

Instead, workspace is connected to GitHub repository where it can fetch code from.

demo-workspace



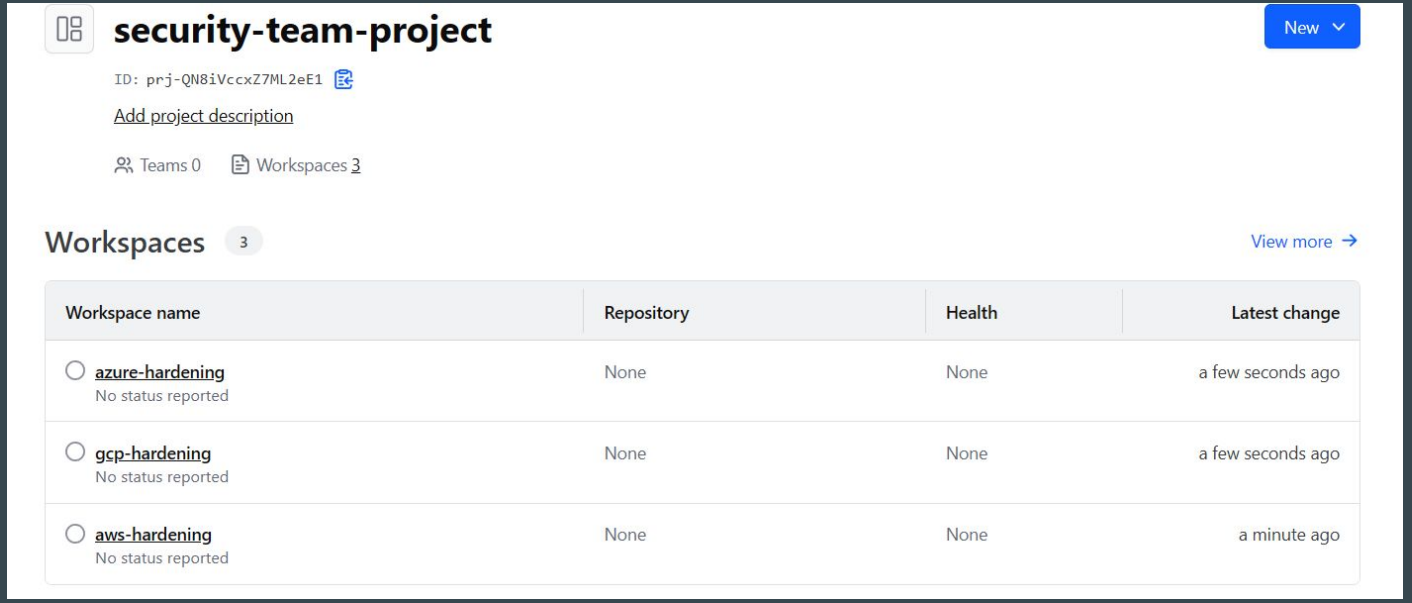
GitHub Repo

Workspace vs Directories

Component	Local Terraform	HCP Terraform
Terraform configuration	On disk	In linked version control repository, or periodically uploaded via API/CLI
Variable values	As `.tfvars` files, as CLI arguments, or in shell environment	In workspace
State	On disk or in remote backend	In workspace
Credentials and secrets	In shell environment or entered at prompts	In workspace, stored as sensitive variables

3 - Projects

HCP Terraform projects let you organize your workspaces into groups.



Point to Note - Projects

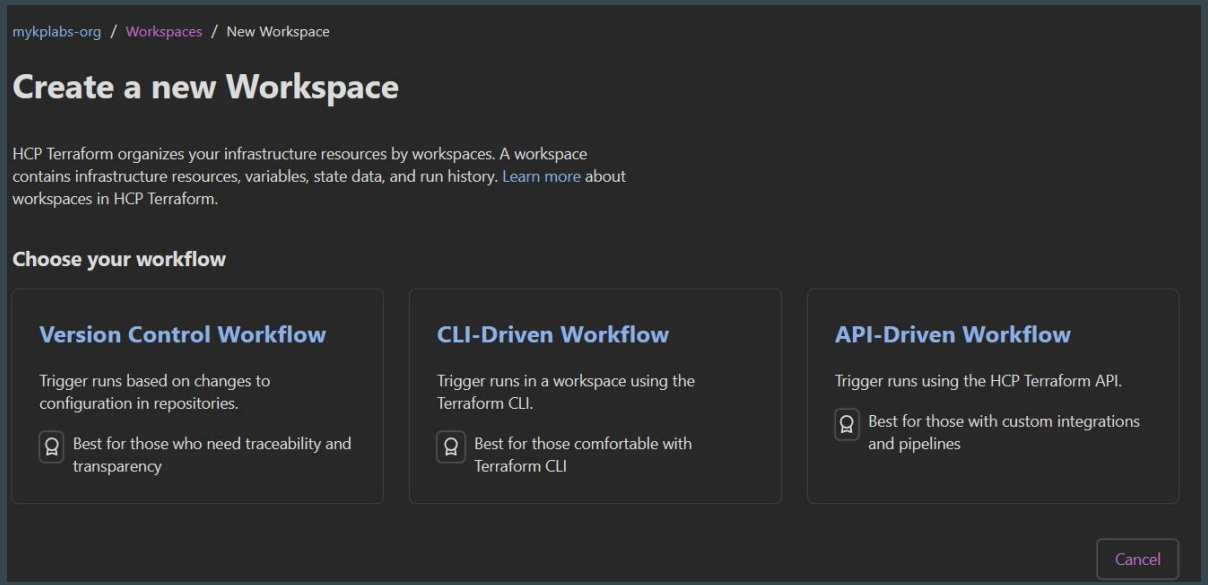
You can structure your projects based on your organization's resource usage and ownership patterns, such as teams, business units, or services.

With HCP Terraform Standard Edition, you can give teams access to groups of workspaces using projects.

The CLI-driven Run Workflow

Setting the Base

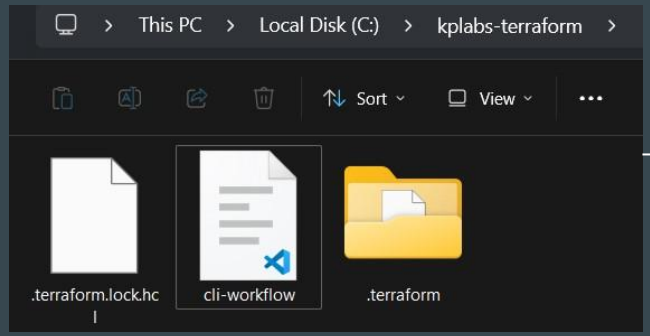
Whenever we create a new Workspace in HCP, following are the 3 types of workflow modes that are available.

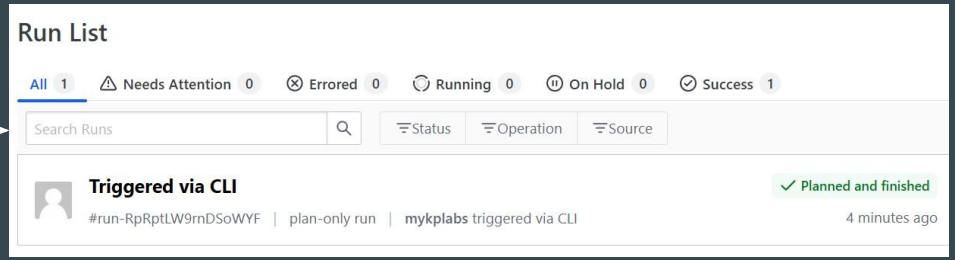


Basics of CLI Driven Workflow

In this approach, the working directory on your workstation is linked with HCP Workspace.

The code file can be present in your laptop, and plan/apply operations can also be initiated from local workstation.

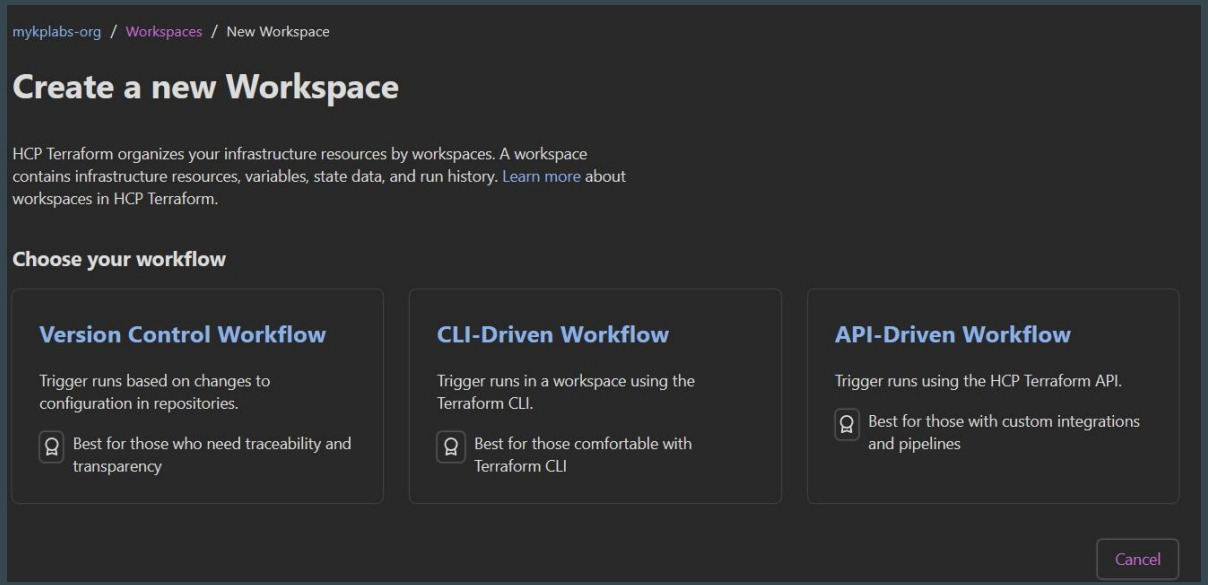




The CLI-driven Run Workflow - Practical

Setting the Base

Whenever we create a new Workspace in HCP, following are the 3 types of workflow modes that are available.



Step 1 - Setup Cloud Integration

You have to add code block within your .tf file to setup cloud integration.

This code will contain details about your HCP organization and workspace name.

```
terraform {
  cloud {

    organization = "mykplabs-org"

    workspaces {
      name = "remote-operation-workspace"
    }
  }
}
```

Step 2 - Terraform Login

Once your cloud integration code block has been added, next step is to run the terraform login command.

```
C:\kplabs-terraform>terraform login
Terraform will request an API token for app.terraform.io using your browser.

If login is successful, Terraform will store the token in plain text in
the following file for use by subsequent commands:
    C:\Users\zealv\AppData\Roaming\terraform.d\credentials.tfrc.json

Do you want to proceed?
  Only 'yes' will be accepted to confirm.

  Enter a value: yes


---------------------------------------------------------------------------

Terraform must now open a web browser to the tokens page for app.terraform.io.

If a browser does not open this automatically, open the following URL to proceed:
    https://app.terraform.io/app/settings/tokens?source=terraform-login
```

Step 3 - Initialize

Run the terraform init command to initialize

```
C:\kplabs-terraform>terraform init
Initializing HCP Terraform...
Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v5.70.0...
- Installed hashicorp/aws v5.70.0 (signed by HashiCorp)
Terraform has made some changes to the provider dependency selections recorded
in the .terraform.lock.hcl file. Review those changes and commit them to your
version control system if they represent changes you intended to make.

HCP Terraform has been successfully initialized!

You may now begin working with HCP Terraform. Try running "terraform plan" to
see any changes that are required for your infrastructure.

If you ever set or change modules or Terraform Settings, run "terraform init"
again to reinitialize your working directory.
```

Step 4 - Run the Plan / Apply Operations

Once initialized, the terraform “plan”, and “apply” commands when entered through CLI will run in HCP Terraform with output streamlined in terminal.

```
C:\kplabs-terraform>terraform plan
Running plan in HCP Terraform. Output will stream here. Pressing Ctrl-C
will stop streaming the logs, but will not stop the plan running remotely.

Preparing the remote plan...

To view this run in a browser, visit:
https://app.terraform.io/app/mykplabs-org/remote-operation-workspace/runs/run-RpRptLW9rnDSoWYF

Waiting for the plan to start...

Terraform v1.9.7
on linux_amd64
Initializing plugins and modules...

Terraform used the selected providers to generate the following execution plan. Resource actions
following symbols:
  + create
```

Sentinel

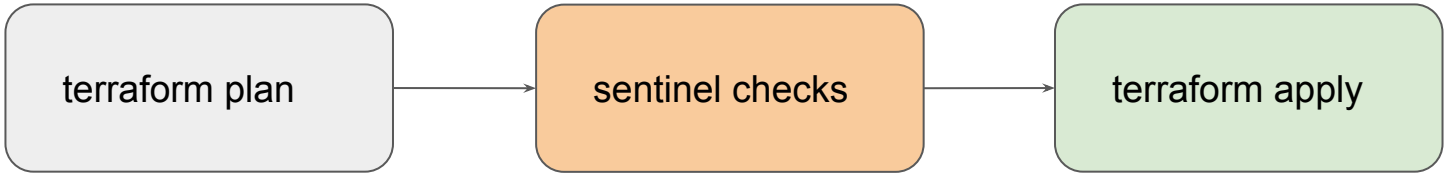
Terraform Cloud In Detail

Overview of the Sentinel

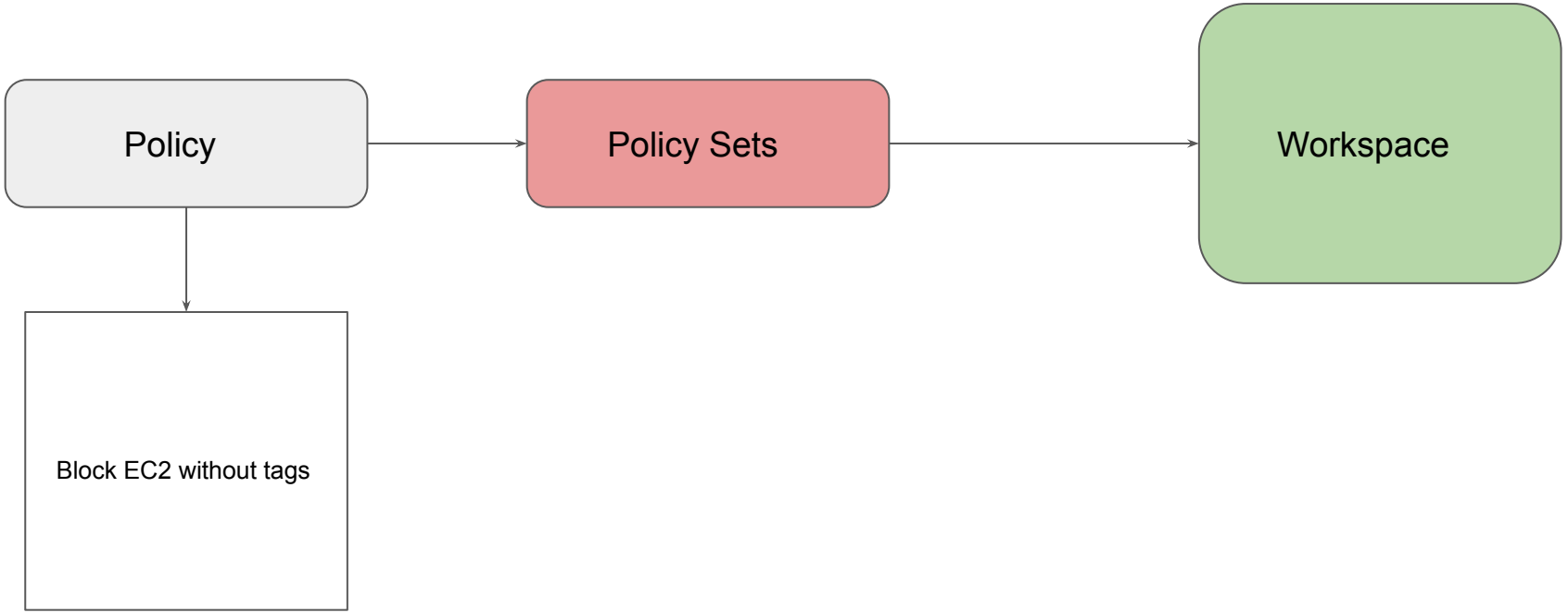
Sentinel is a policy-as-code framework integrated with the HashiCorp Enterprise products.

It enables fine-grained, logic-based policy decisions, and can be extended to use information from external sources.

Note: Sentinel policies are paid feature



High Level Structure

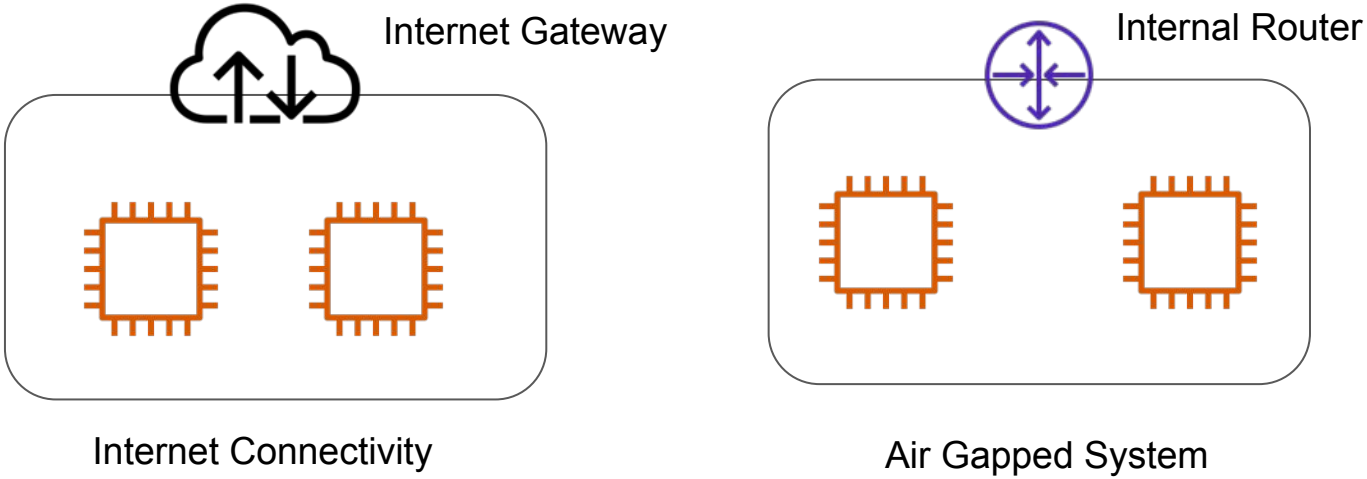


Air Gapped Environments

Installation Methods

Understanding Concept of Air Gap

An air gap is a network security measure employed to ensure that a secure computer network is physically isolated from unsecured networks, such as the public Internet.



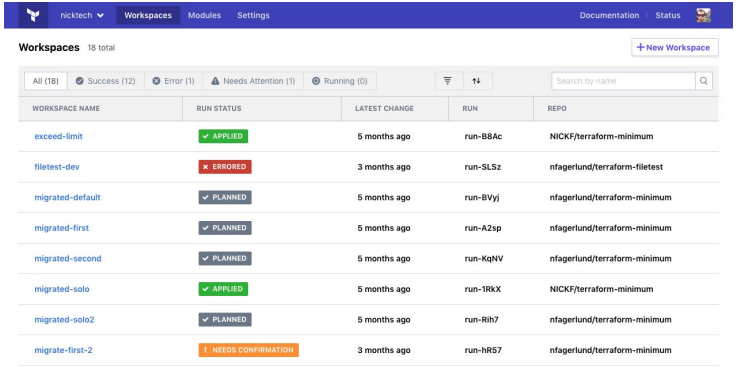
Usage of Air Gapped Systems

Air Gapped Environments are used in various areas. Some of these include:

- Military/governmental computer networks/systems
- Financial computer systems, such as stock exchanges
- Industrial control systems, such as SCADA in Oil & Gas fields

Terraform Enterprise Installation Methods

Terraform Enterprise installs using either an online or air gapped method and as the names infer, one requires internet connectivity, the other does not



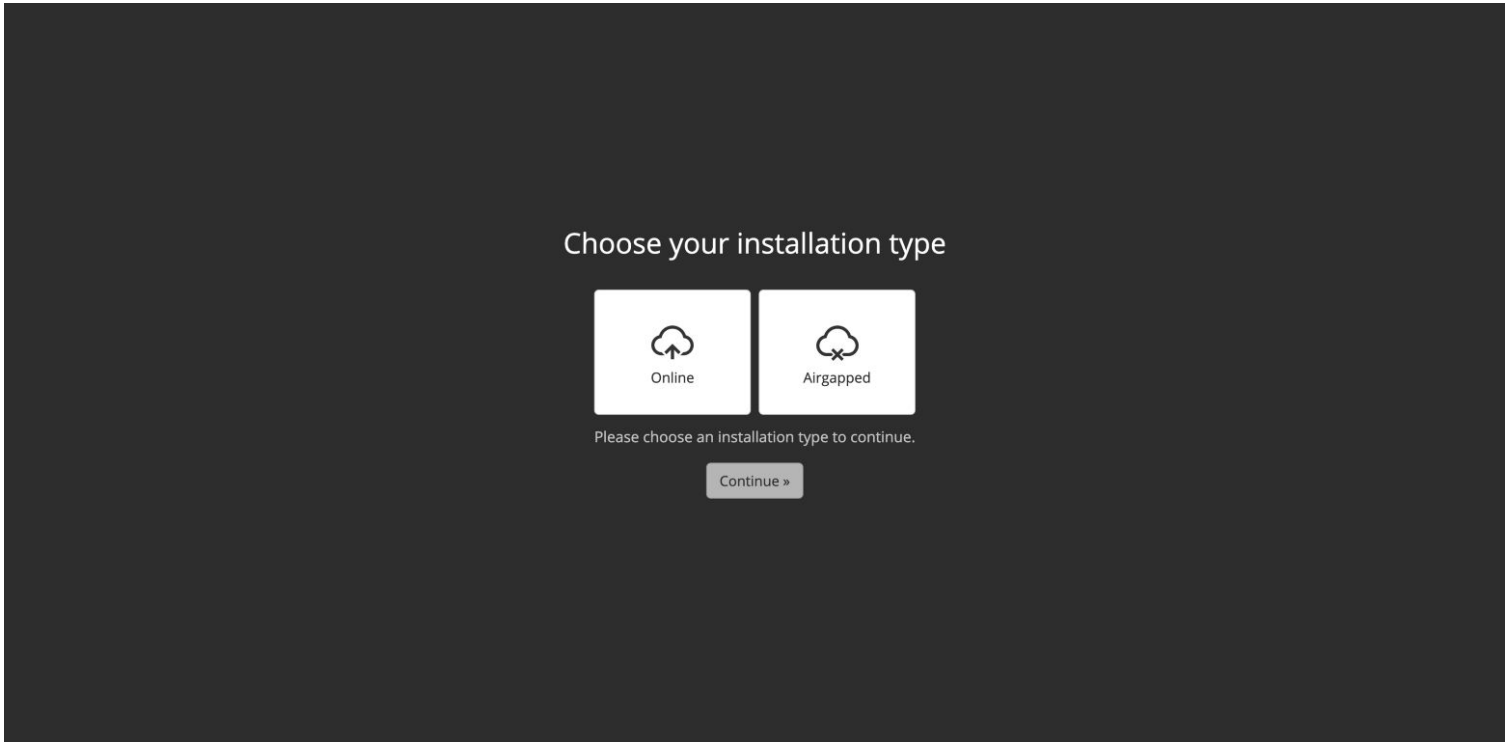
Terraform Enterprise

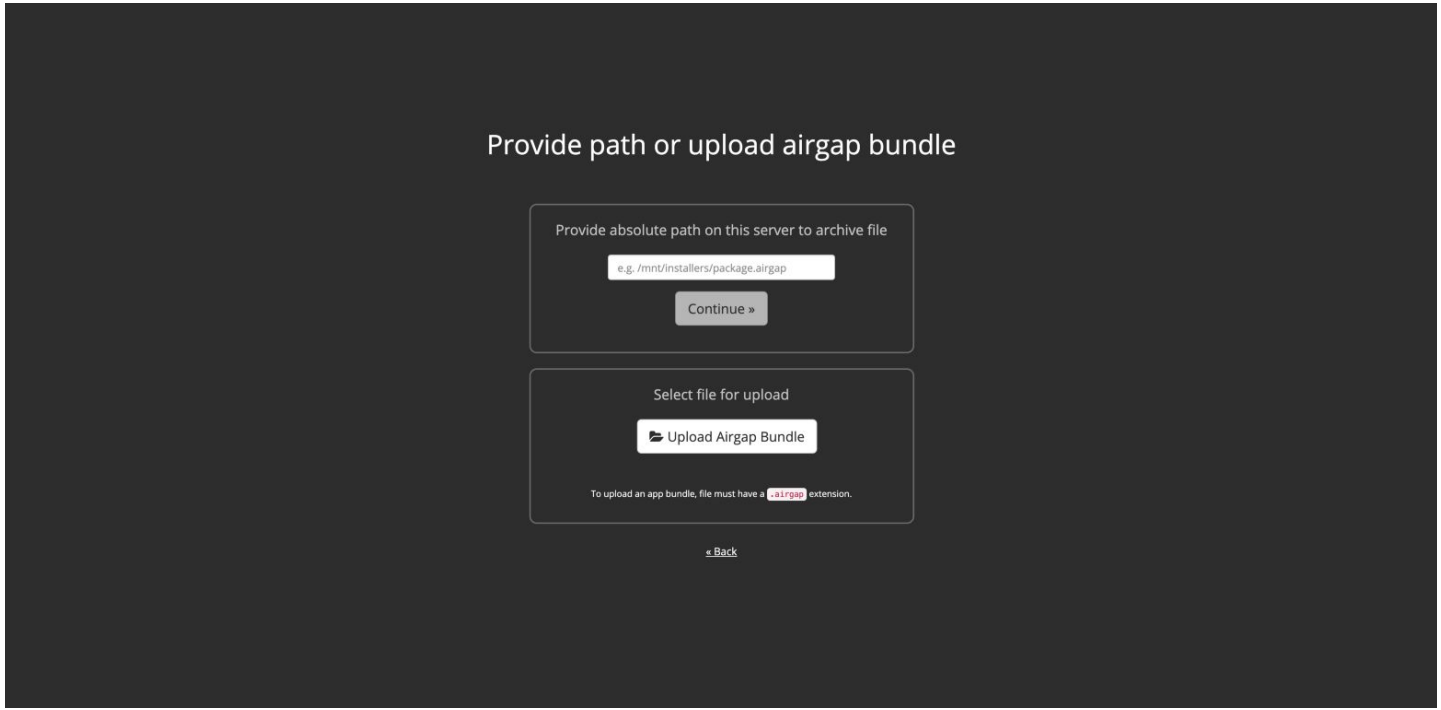
Air Gap Install





Isolated Server


Choose your installation type
Online
Airgapped
Please choose an installation type to continue.
Continue »


Provide path or upload airgap bundle
Provide absolute path on this server to archive file
e.g. /mnt/installers/package.airgap
Continue »
Select file for upload
Upload Airgap Bundle
To upload an app bundle, file must have a .airgap extension.
« Back

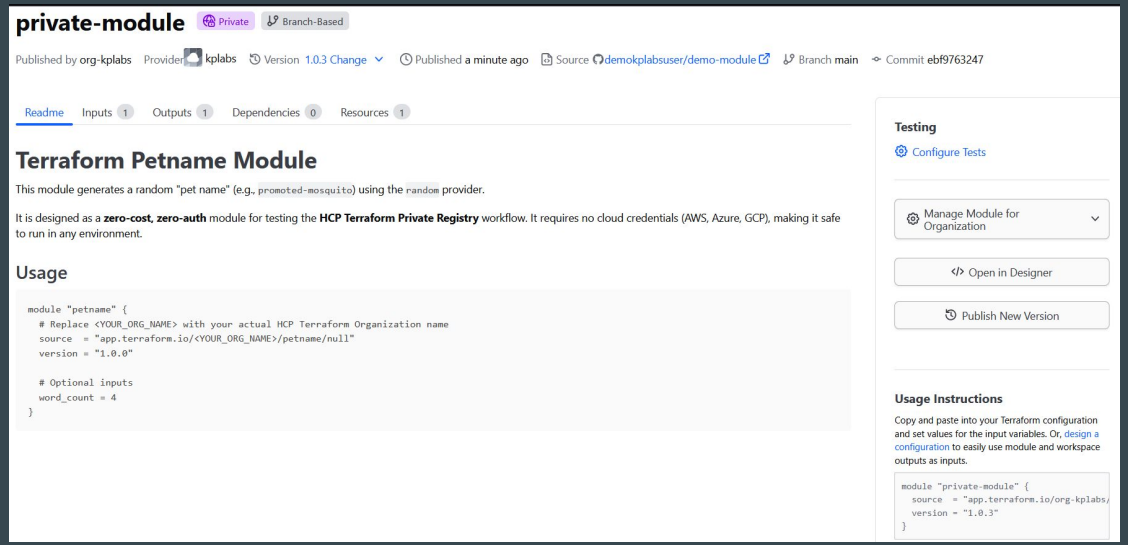
Relax and Have a Meme Before Proceeding



HCP Terraform - Private Registry

Setting the Base

HCP Terraform's private registry works similarly to the public Terraform Registry and helps you share Terraform providers and Terraform modules across your organization.



Calling the Private Module

To call the Private Registry Module from your workspace, you have to specify the correct private registry module source URL.

main.tf > ...

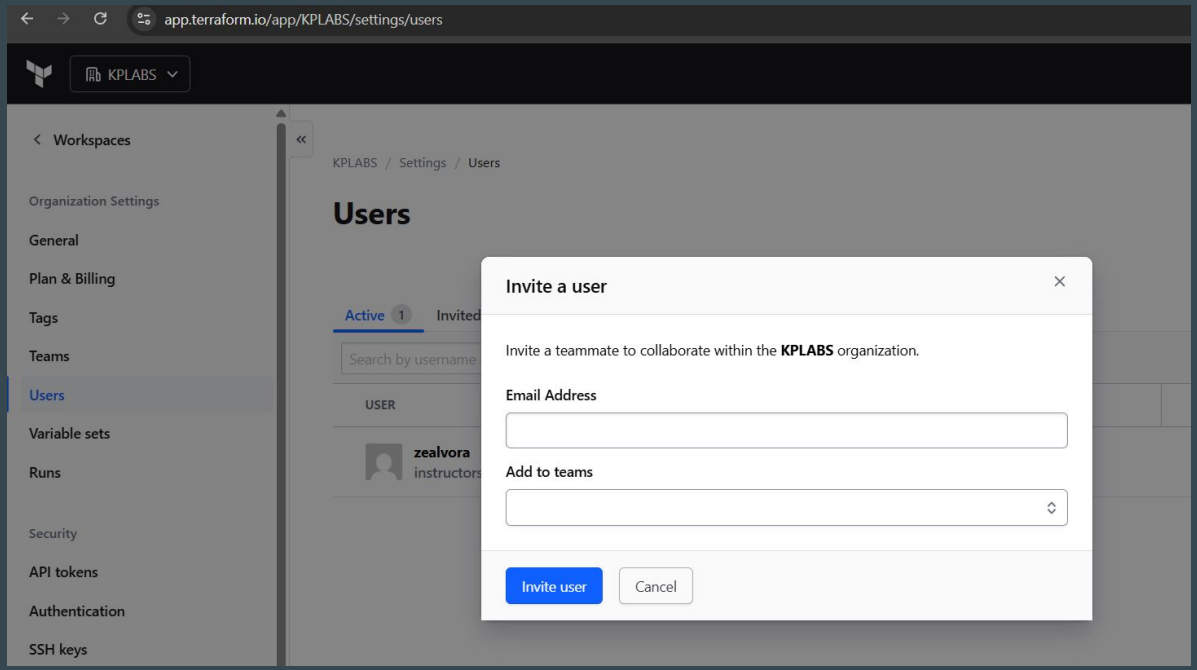
```
module "private-module" {
  source  = "app.terraform.io/org-kplabs/private-module/kplabs"
  version = "1.0.0"
  word_count = 4
}

output "final_pet_name" {
  value = module.private-module
}
```

HCP Terraform - Teams

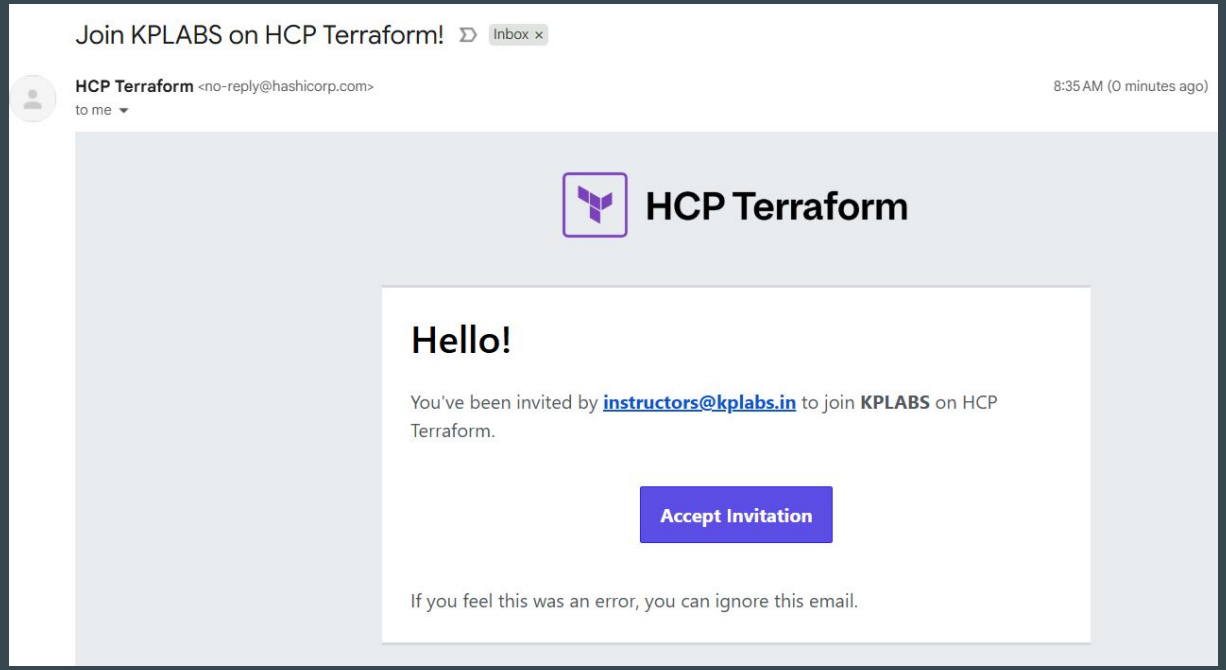
HCP Terraform - Users

You can invite new users to HCP Terraform Organization to collaborate on projects.



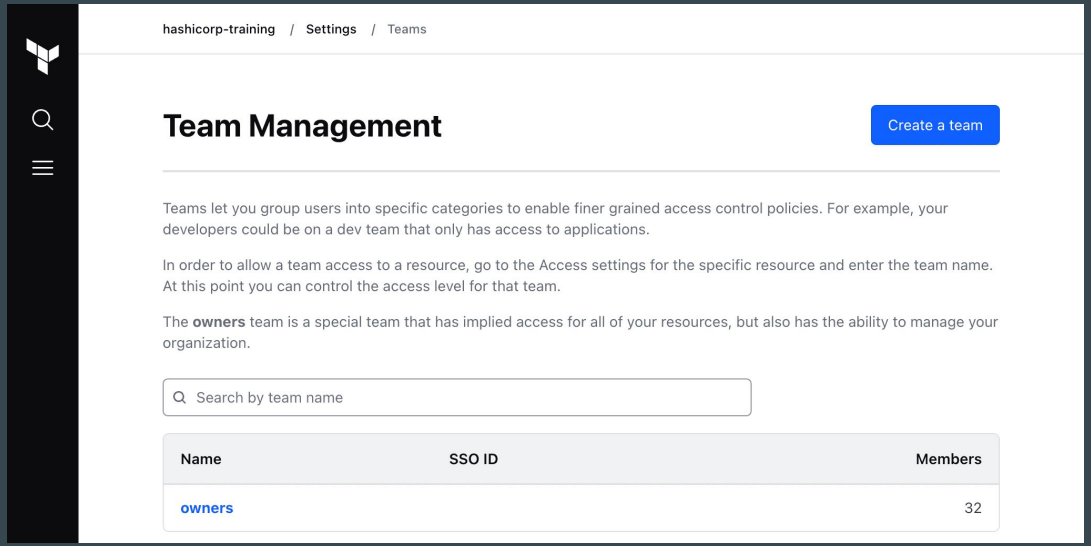
Reference Screenshot

Reference screenshot of user receiving the invitation to join.



HCP Terraform - Teams

Teams are groups of HCP Terraform users within an organization.



Point to Note

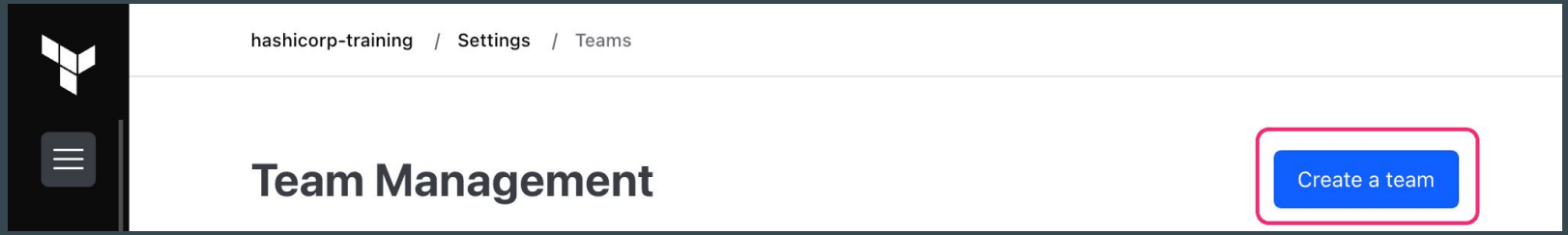
The owners team is the default team of an HCP Terraform organization.

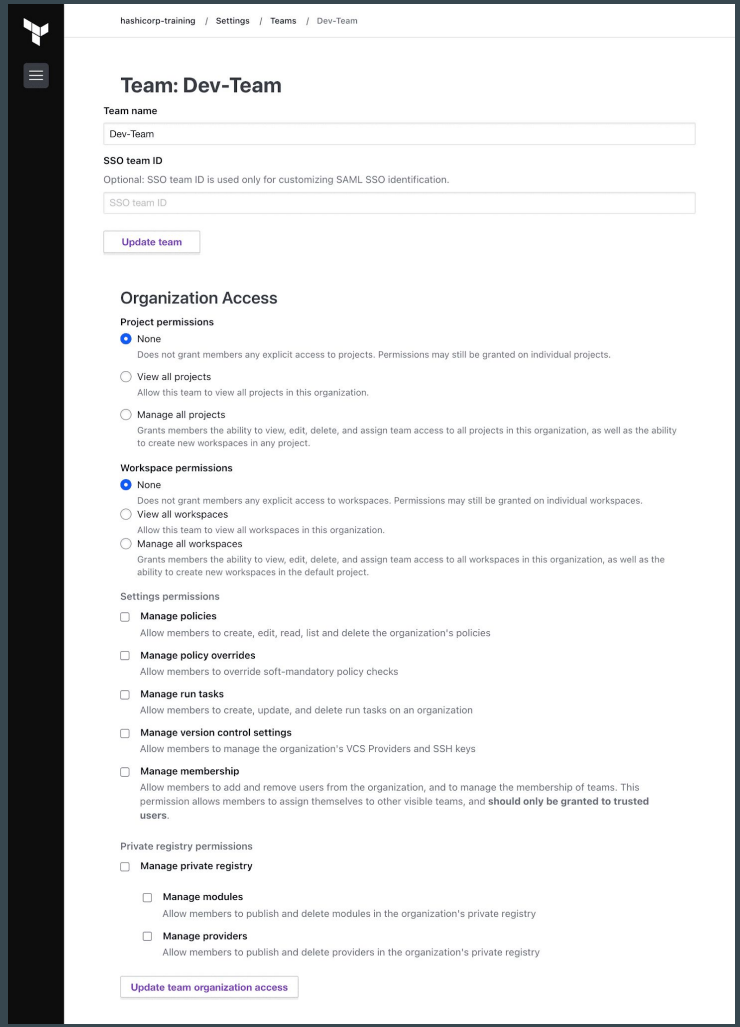
Manage permissions in HCP Terraform

Create a new team

The owners team is the default team of an HCP Terraform organization.

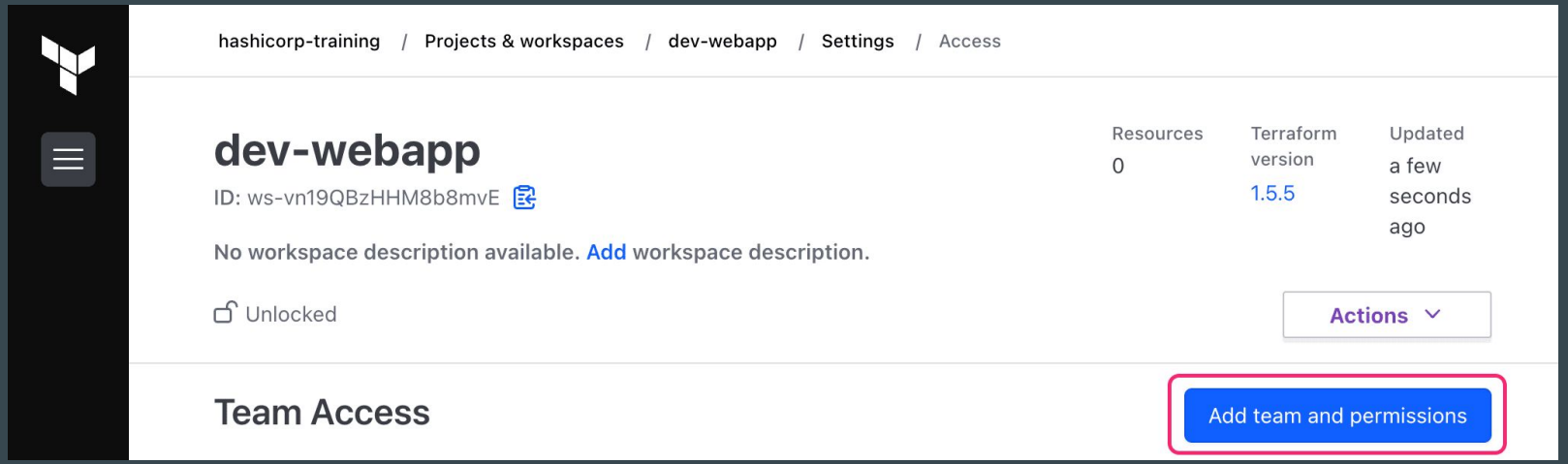
This team has every available permission in the organization, so it is important to create restricted team access before adding new members.




hashicorp-training / Settings / Teams / Dev-Team
Team: Dev-Team
Team name
Dev-Team
SSO team ID
Optional: SSO team ID is used only for customizing SAML SSO identification.
SSO team ID
Update team
Organization Access
Project permissions
None
Does not grant members any explicit access to projects. Permissions may still be granted on individual projects.
View all projects
Allow this team to view all projects in this organization.
Manage all projects
Grants members the ability to view, edit, delete, and assign team access to all projects in this organization, as well as the ability to create new workspaces in any project.
Workspace permissions
None
Does not grant members any explicit access to workspaces. Permissions may still be granted on individual workspaces.
View all workspaces
Allow this team to view all workspaces in this organization.
Manage all workspaces
Grants members the ability to view, edit, delete, and assign team access to all workspaces in this organization, as well as the ability to create new workspaces in the default project.
Settings permissions
Manage policies
Allow members to create, edit, read, list and delete the organization's policies
Manage policy overrides
Allow members to override soft-mandatory policy checks
Manage run tasks
Allow members to create, update, and delete run tasks on an organization
Manage version control settings
Allow members to manage the organization's VCS Providers and SSH keys
Manage membership
Allow members to add and remove users from the organization, and to manage the membership of teams. This permission allows members to assign themselves to other visible teams, and should only be granted to trusted users.
Private registry permissions
Manage private registry
Manage modules
Allow members to publish and delete modules in the organization's private registry
Manage providers
Allow members to publish and delete providers in the organization's private registry
Update team organization access

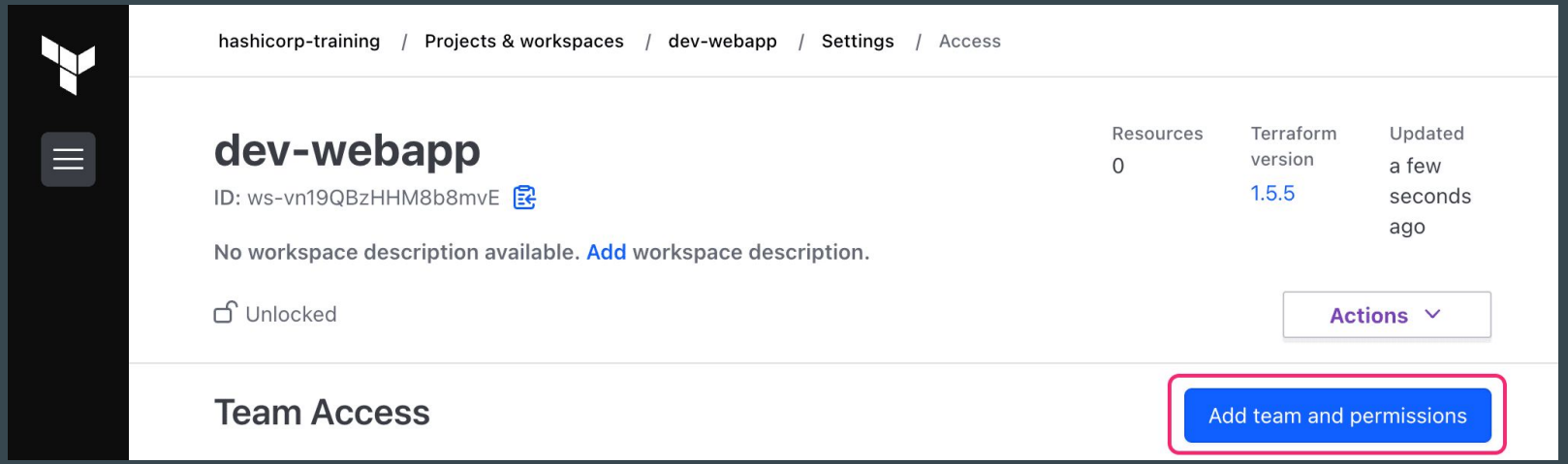
Workspace Level Permissions

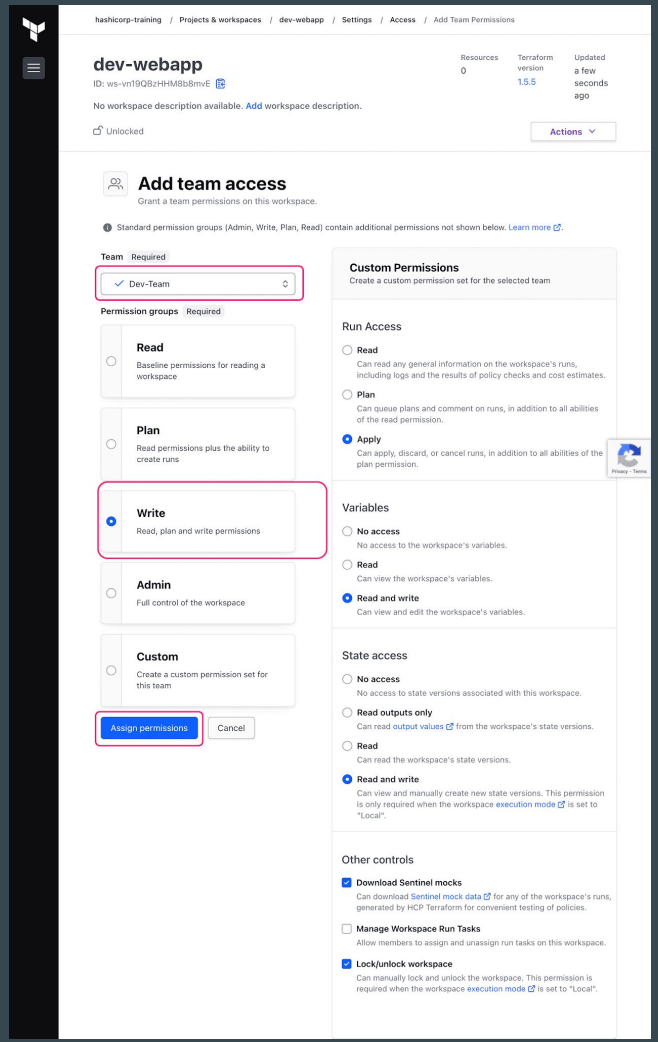
You can configure Workspace level permissions for Teams



Workspace Level Permissions

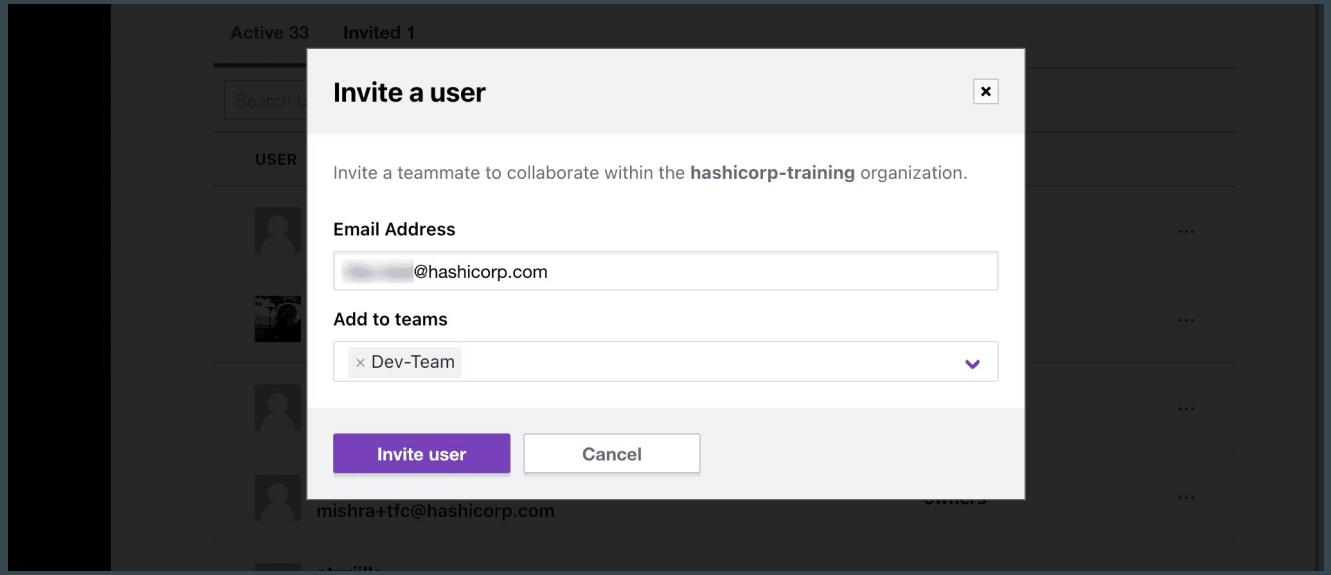
You can configure Workspace level permissions for Teams




hashicorp-training / Projects & workspaces / dev-webapp / Settings / Access / Add Team Permissions
dev-webapp
ID: ws-vn19QBzHHM8b8mvE
Resources
0
Terraform version
1.5.5
Updated
a few seconds ago
No workspace description available. Add workspace description.
Unlocked
Actions
Add team access
Grant a team permissions on this workspace.
Standard permission groups (Admin, Write, Plan, Read) contain additional permissions not shown below. Learn more.
Team Required
Dev-Team
Permission groups Required
Read
Baseline permissions for reading a workspace
Plan
Read permissions plus the ability to create runs
Write
Read, plan and write permissions
Admin
Full control of the workspace
Custom
Create a custom permission set for this team
Assign permissions
Cancel
Custom Permissions
Create a custom permission set for the selected team
Run Access
Read
Can read any general information on the workspace's runs, including logs and the results of policy checks and cost estimates.
Plan
Can queue plans and comment on runs, in addition to all abilities of the read permission.
Apply
Can apply, discard, or cancel runs, in addition to all abilities of the plan permission.
Variables
No access
No access to the workspace's variables.
Read
Can view the workspace's variables.
Read and write
Can view and edit the workspace's variables.
State access
No access
No access to state versions associated with this workspace.
Read outputs only
Can read output values from the workspace's state versions.
Read
Can read the workspace's state versions.
Read and write
Can view and manually create new state versions. This permission is only required when the workspace execution mode is set to "Local".
Other controls
Download Sentinel mocks
Can download Sentinel mock data for any of the workspace's runs, generated by HCP Terraform for convenient testing of policies.
Manage Workspace Run Tasks
Allow members to assign and unassign run tasks on this workspace.
Lock/unlock workspace
Can manually lock and unlock the workspace. This permission is required when the workspace execution mode is set to "Local".

Invite a user to your organization and team

To collaborate with your team members in HCP Terraform, you need to grant them access to the same HCP Terraform organization.



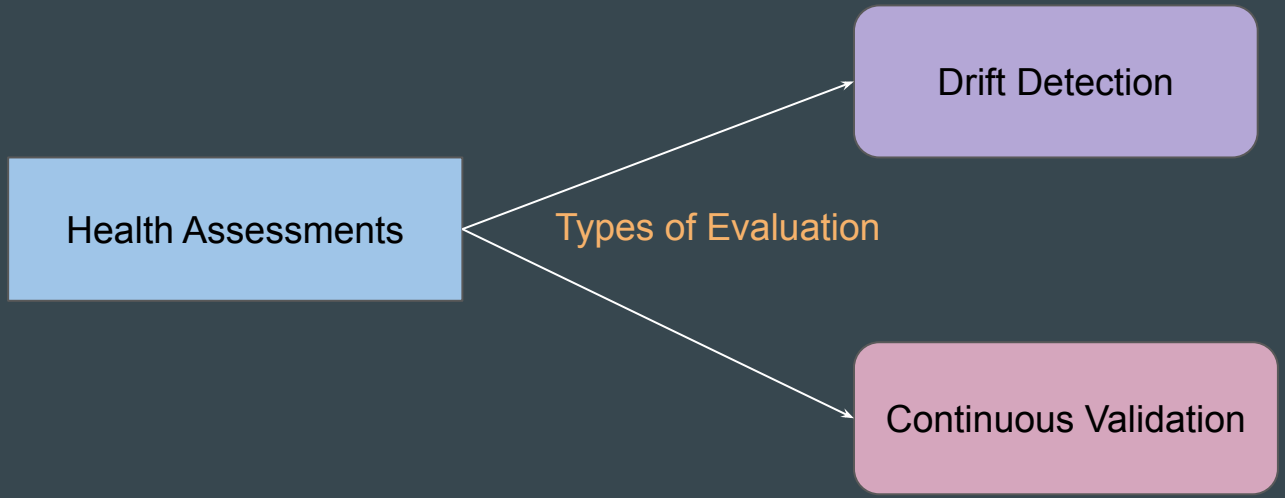
Point to Note

The owners team is the default team of an HCP Terraform organization.

HCP Terraform - Health Assessments

Setting the Base

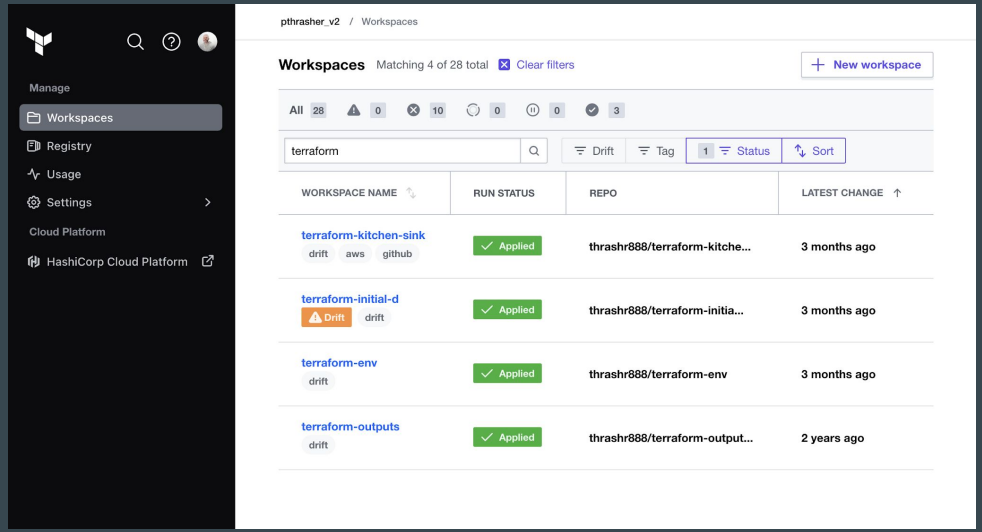
HCP Terraform can perform automatic health assessments in a workspace to assess whether its real infrastructure matches the requirements defined in its Terraform configuration.



Drift Detection

Drift detection determines whether your real-world infrastructure matches your Terraform configuration.

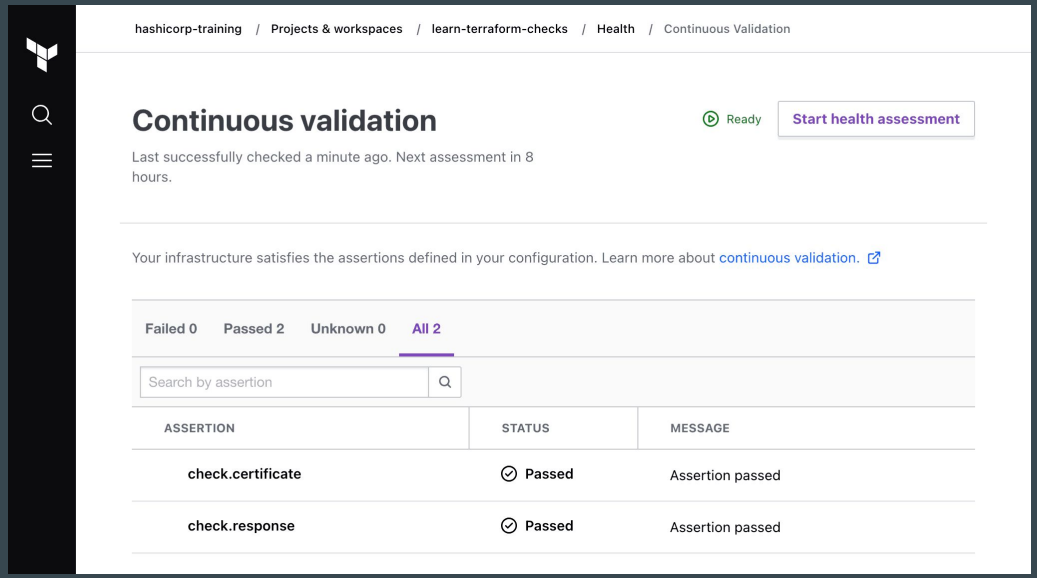
Configuration drift occurs when changes are made outside Terraform's regular process (manual changes)



Continuous Validation

Continuous validation determines whether custom conditions in the workspace's configuration continue to pass after Terraform provisions the infrastructure.

For example, you can monitor whether your website returns an expected status code.



Example - Continuous Validation

The following example uses the HTTP Terraform provider and a scoped data source within a check block to assert the Terraform website returns a 200 status code, indicating it is healthy.

```
check "health_check" {
  data "http" "terraform_io" {
    url = "https://kplabs.in"
  }

  assert {
    condition = data.http.terraform_io.status_code == 200
    error_message = "KPLABS returned an unhealthy status code"
  }
}
```

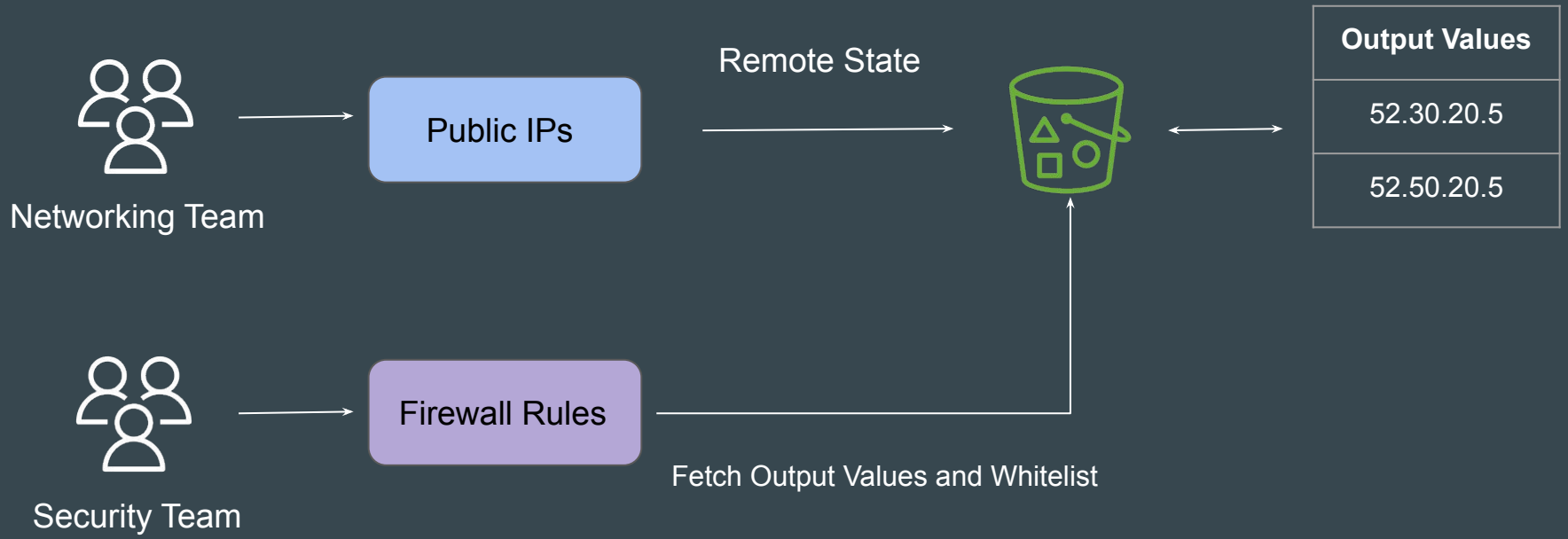
Point to Note

Health assessments are available in HCP Terraform Standard and Premium editions.

HCP Terraform - Run Triggers

Setting the Base

Network project contains state file whose outputs contains latest IP addresses that needs whitelisting in security project.



Possible Solution - Remote State Data Source

The terraform_remote_state data source allows us to fetch output values from a specific state backend

```
data "terraform_remote_state" "eip" {
  backend = "s3"
  config = {
    bucket = "kplabs-team-networking-bucket"
    key    = "eip.tfstate"
    region = "us-east-1"
  }
}
```

Step 1 - Define Remote State Source

```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = "${data.terraform_remote_state.eip.outputs.eip_addr}/32"
  from_port         = 443
  ip_protocol       = "tcp"
  to_port           = 443
}
```

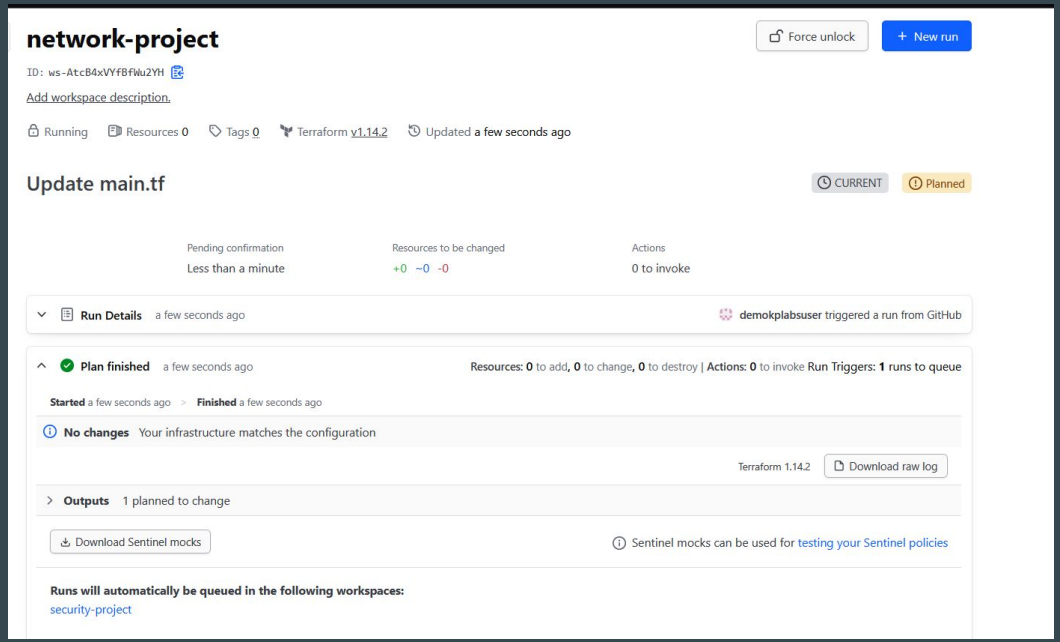
Step 2 - Define Data to Fetch

Challenge with Remote State Data Source

If you updated the Networking space(e.g., new Public IP), the Application space wouldn't know about it. The infrastructure was decoupled, but the process was manual.

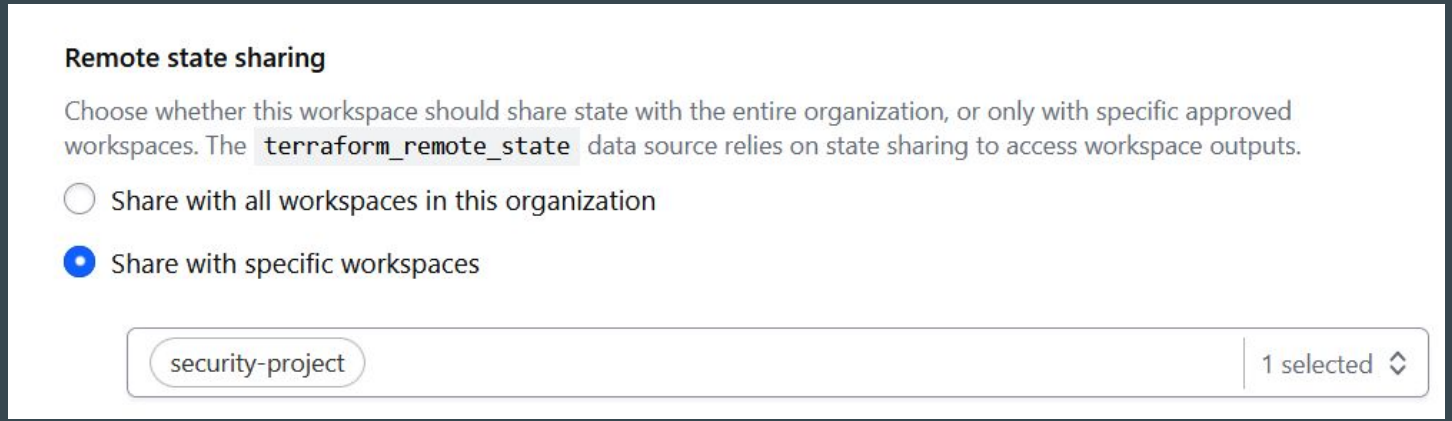
Introducing Run Triggers

If you need to update your workspace every time when a source workspace changes, you can use Run Triggers.



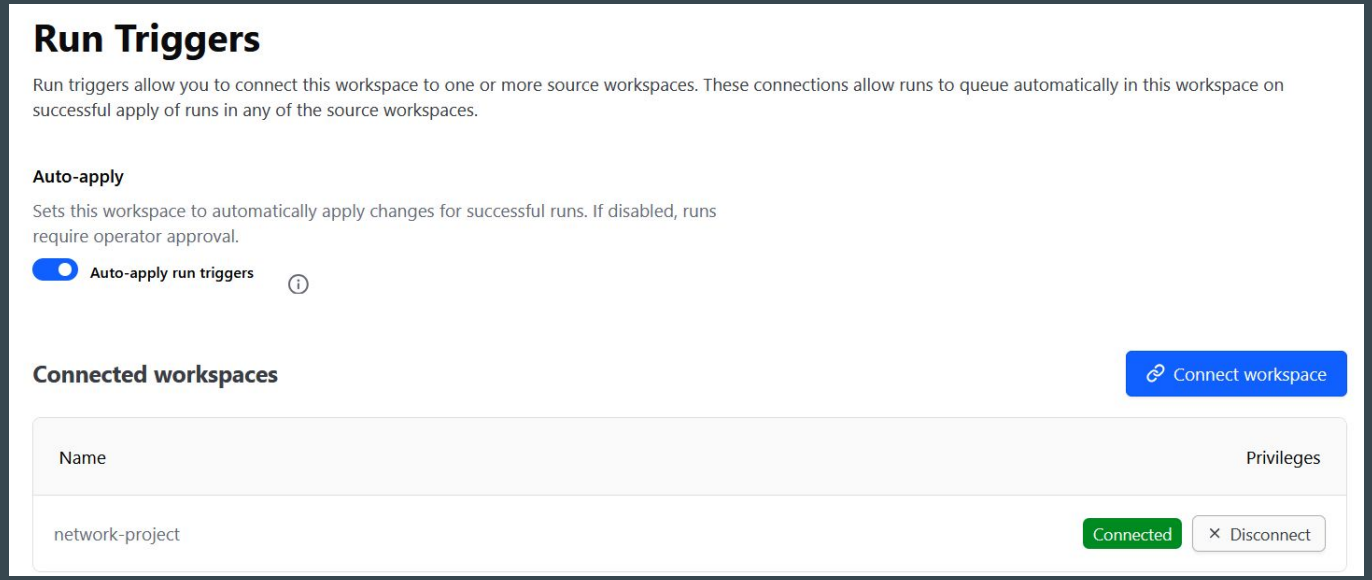
Important Configuration - 1

In the network-project workspace, you need to enable “Remote state sharing” and designate the workspace with which you want to share the state.



Important Configuration - 2

Within the security-project workspace, you need to configure the “Run Triggers” settings and designate the appropriate connected namespace.



Data Source - tfe_outputs

HashiCorp recommends using the tfe_outputs data source in the HCP Terraform/Enterprise Provider to access remote state outputs in HCP Terraform or Terraform Enterprise.

The tfe_outputs data source is more secure because it does not require full access to workspace state to fetch outputs.

```
data "tfe_outputs" "foo" {
  organization = "my-org"
  workspace = "my-workspace"
}

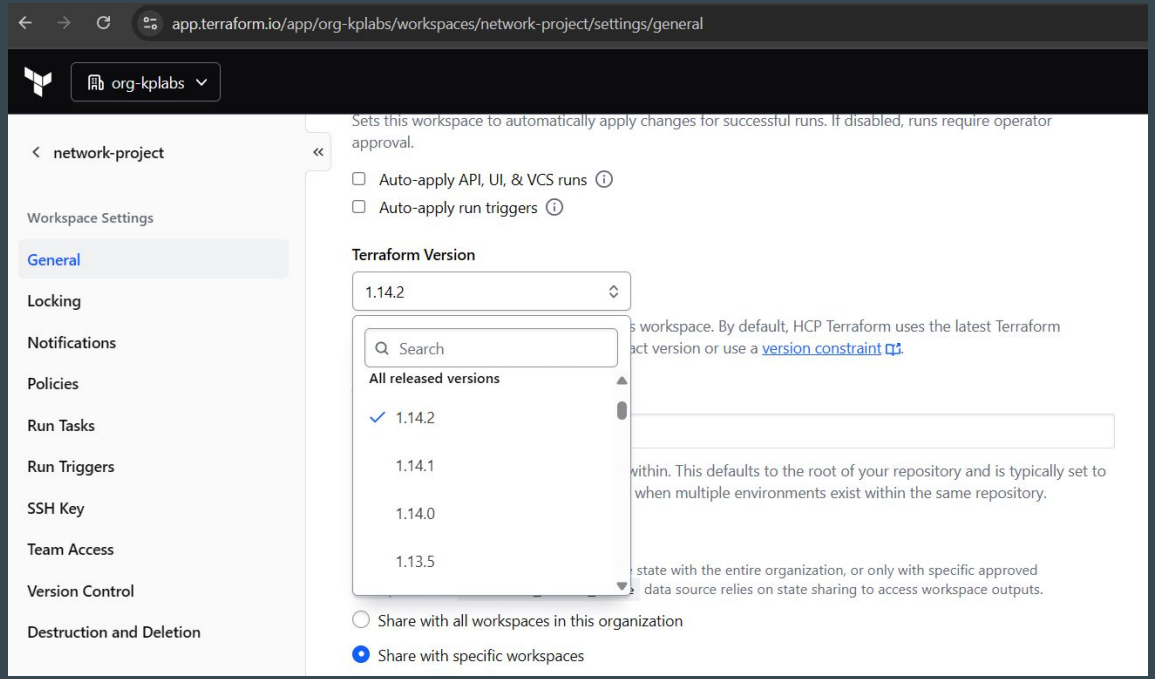
resource "random_id" "vpc_id" {
  keepers = {
    bar = data.tfe_outputs.foo.values.bar
  }

  byte_length = 8
}
```

HCP Terraform - Choosing Version

Setting the Base

Each HCP Terraform workspace has an assigned Terraform version that it uses for all remote operations in the workspace.



Terraform Challenges

Key Observations

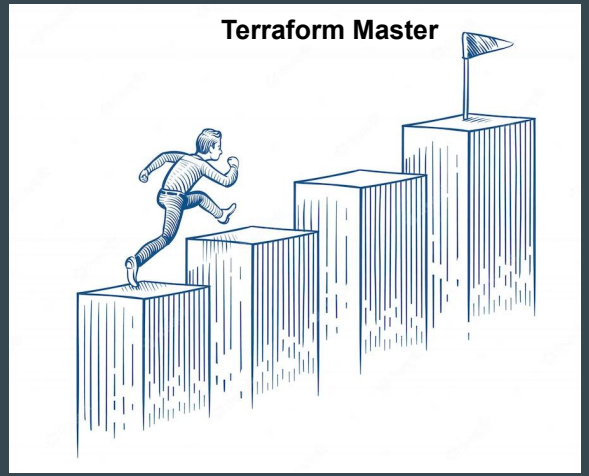
At this stage, we have been learning core concepts of Terraform step by step.

Whenever learning a new technology, small set of practical projects are always useful to grasp the practical aspects of a technology.



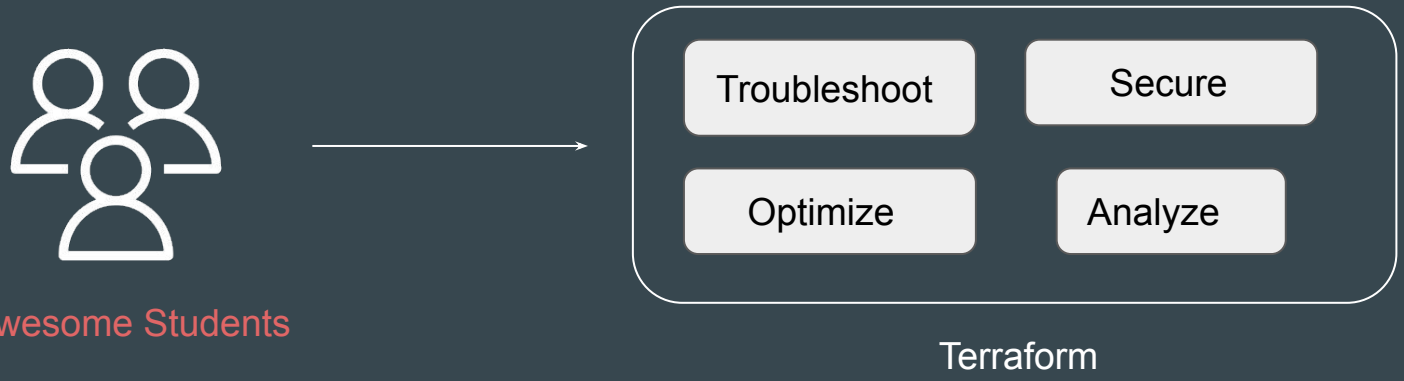
Introducing Terraform Challenges

With Terraform Challenges, we aim to reduce the gap between learning and gaining practical experience.



About the Challenges

Each Challenge will test you in different areas of Terraform that will help you gain some kind of hands-on experience.



Workflow Steps

We will have multiple sets of challenges.

After each challenge video, we will have a Solution Hints video and then the Practical Solution video.



Terraform Challenge 1

Understanding the Challenge

A Developer at Sample Small Corp had created a Terraform File for creating certain resources.

The code was written a few years back based on the old Terraform version.

```
provider "aws" {
  version = "~> 2.54"
  region  = "us-east-1"
  access_key = "AKIAIOSFODNN7EXAMPLE"
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
}

provider "digitalocean" {}

terraform {
    required_version = "0.12.31"
}


resource "aws_eip" "kplabs_app_ip" {
  vpc      = true
}
```

What you need to do?

1. Create Infrastructure using the provided code (without modifications).

2. Verify if the code works in the latest version of Terraform and Provider .

3. Modify and Fix the code so that it works with latest version of Terraform.

4. Feel free to edit the code as you like.

TF Challenge 1 - Solution Discussion and Hints

Hint 1 - Create Infrastructure with Base Code

Based on the initial code given to you, use appropriate version of binaries to ensure infrastructure gets created successfully.

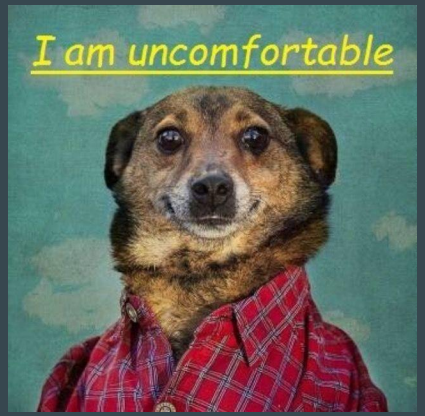
```
terraform_0.12.31
terraform_0.12.30
terraform_0.12.29
terraform_0.12.28
terraform_0.12.27
terraform_0.12.26
terraform_0.12.25
terraform_0.12.24
terraform_0.12.23
terraform_0.12.22
```

Hint 2 - Access/Secret Keys

There are hardcoded AWS Access/Secret keys with the code.

This MUST be be fixed.

```
provider "aws" {
  version = "~> 2.54"
  region  = "us-east-1"
  access_key = "AKIAIOSFODNN7EXAMPLE"
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
}
```



Hint 3 - Provider Block

Provider Block is used to define provider version along with 3rd party providers.

Instead, use the new required_provider block to define provider and constraints.

```
provider "aws" {
  version = "~> 2.54"
  region  = "us-east-1"
  access_key = "AKIAIOSFODNN7EXAMPLE"
  secret_key = "wJalrXUtnFEMI/K7MDENG/b
}

provider "digitalocean" {}
```

```
terraform {
  required_providers {
    mycloud = {
      source  = "mycorp/mycloud"
      version = "~> 1.0"
    }
  }
}
```

Hint 4 - Terraform Core Version Requirement

Since the challenge states that latest version of Terraform should be used, you can plan to remove the required_version block from the code.

```
terraform {
    required_version = "0.12.31"
}
```

Hint 5 - Code Upgrade

Does the resource block of “aws_eip” work with the latest version of Terraform?

It can happen that latest AWS provider requires some changes in the aws_eip resource block. Incorporate these changes to ensure EIP gets created.

```
resource "aws_eip" "kplabs_app_ip" {
  vpc      = true
}
```

Join us in our Adventure

Be Awesome



kplabs.in/chat



kplabs.in/linkedin

Terraform Challenge 2

Understanding the Challenge

A sample code has been provided to you that creates certain resources.

You are required to optimize the code following the Best Practices.

```
resource "aws_security_group" "security_group_payment_app" {
  name        = "payment_app"
    description = "Application Security Group"
  depends_on = [aws_eip.example]

# Below ingress allows HTTPS  from DEV VPC
  ingress {
       from_port        = 443
     to_port          = 443
    protocol         = "tcp"
      cidr_blocks      = ["172.31.0.0/16"]
  }

# Below ingress allows APIs access from DEV VPC

  ingress {
    from_port        = 8080
      to_port          = 8080
```

Conditions to Meet

1. Ensure the code is working and resource gets created.

2. Do NOT delete the existing terraform.lock.hcl file. File is free to be modified based on requirements.

3. Demonstrate ability to modify variable “splunk” from 8088 to 8089 without modifying the Terraform code.

TF Challenge 2 - Solution Discussion and Hints

Hint 1 - Indentation

Indentation issues are present in the code.

Make sure that code is properly indented.

```
# Below ingress allows HTTPS  from DEV VPC
  ingress {
       from_port         = 443
     to_port           = 443
    protocol          = "tcp"
      cidr_blocks       = ["172.31.0.0/16"]
  }
```

Hint 2 - Using Variables and TFVars

Many values are hard-coded as part of the code.

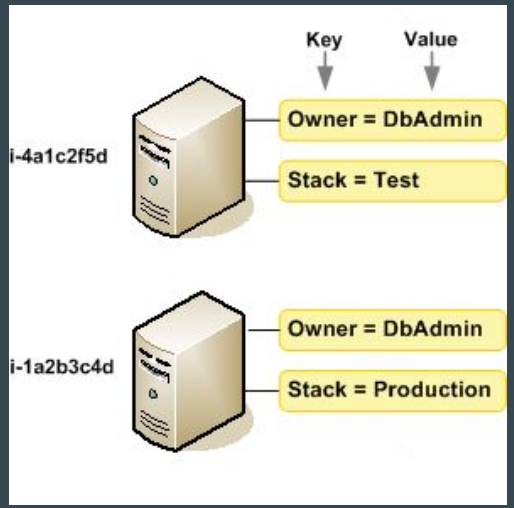
This makes it difficult to modify if code base becomes larger.

```
# Below ingress allows HTTPS  from DEV VPC
  ingress {
       from_port         = 443
     to_port           = 443
    protocol          = "tcp"
      cidr_blocks       = ["172.31.0.0/16"]
  }
```

Hint 3 - Using Tags

It is important that resources are properly tagged

This will make it easier to identify the resource among all others.



Hint 4 - Variable Precedence

Consider using appropriate variable precedence to override variables from Terraform code.

Hint 5 - Right Folder Structure

Having right naming convention for files is important.

Bad Structure: Everything in one single file named main.tf

Good Structure: providers.tf , variables.tf , ec2.tf and so on.

Terraform Challenge 3

Understanding the Requirements

You will be provided with a variable named instance_config

The variable type is map.

```
variable "instance_config" {
  type = map
  default = {
    instance1 = { instance_type = "t2.micro", ami = "ami-03a6eaae9938c858c" }
    instance2 = { instance_type = "t2.micro", ami = "ami-053b0d53c279acc90" }
  }
}
```

Conditions to Meet

1. Based on the values specified in map, EC2 instances should be created accordingly.

2. If key/value is removed from map, EC2 instances should be destroyed accordingly.

TF Challenge 3 - Hints

Hint 1 - Loops

The requirement indicates that based on key/value specified in map, the resources should be created and destroyed accordingly.

We need to use some kind of loops to achieve this.

```
variable "instance_config" {
  type = map
  default = {
    instance1 = { instance_type = "t2.micro", ami = "ami-03a6eaae9938c858c" }
    instance2 = { instance_type = "t2.micro", ami = "ami-053b0d53c279acc90" }
  }
}
```

Hint 2 - for_each

If a resource block includes a for_each argument whose value is a map or a set of strings, Terraform creates one instance for each member of that map or set.

Terraform Challenge 4

Requirement - 1

Clients wants a code that can create IAM user in AWS account with following syntax:

admin-user-{account-number-of-aws}



Requirement - 2

Client wants to have a logic that will show names of ALL users in AWS account in the output.

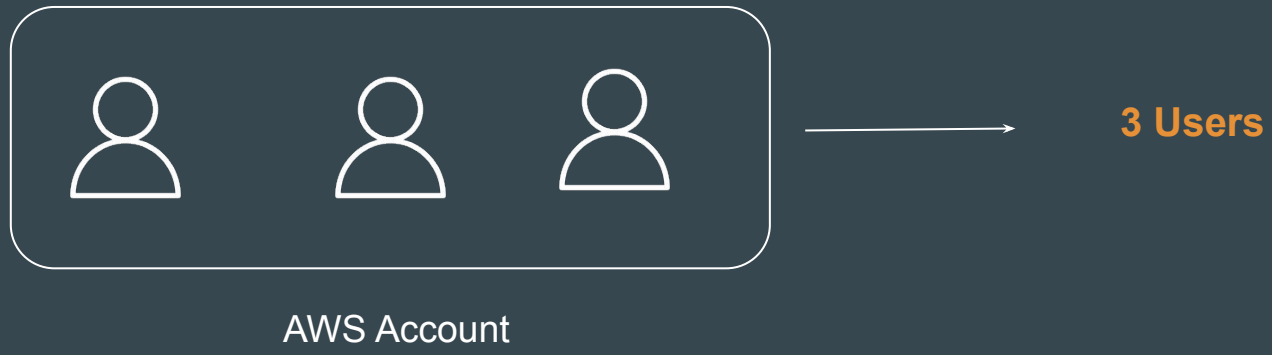


AWS Account

```
+ users       = [
    + "Ankit",
    + "DemoUser",
    + "cloudformation-user",
    + "demo-user",
    + "kplabs-demo-user",
    + "terraform",
  ]
```

Requirement - 3

Along with list of users in AWS, client also wants Terraform to show Total number of users in AWS.



TF Challenge 4 - Solution Hints

Hint 1 - Data Sources

Data Sources allows us to dynamically fetch information from the infrastructure resource or other state backends.

You can try to dynamically fetch information like AWS Account ID, User names using Data Sources.

Hint 2 - Functions

To calculate number of users is outside scope of Data Source.

You need to make use of Terraform Function that can calculate total number of users and output it.

Join us in our Adventure

Be Awesome



kplabs.in/chat

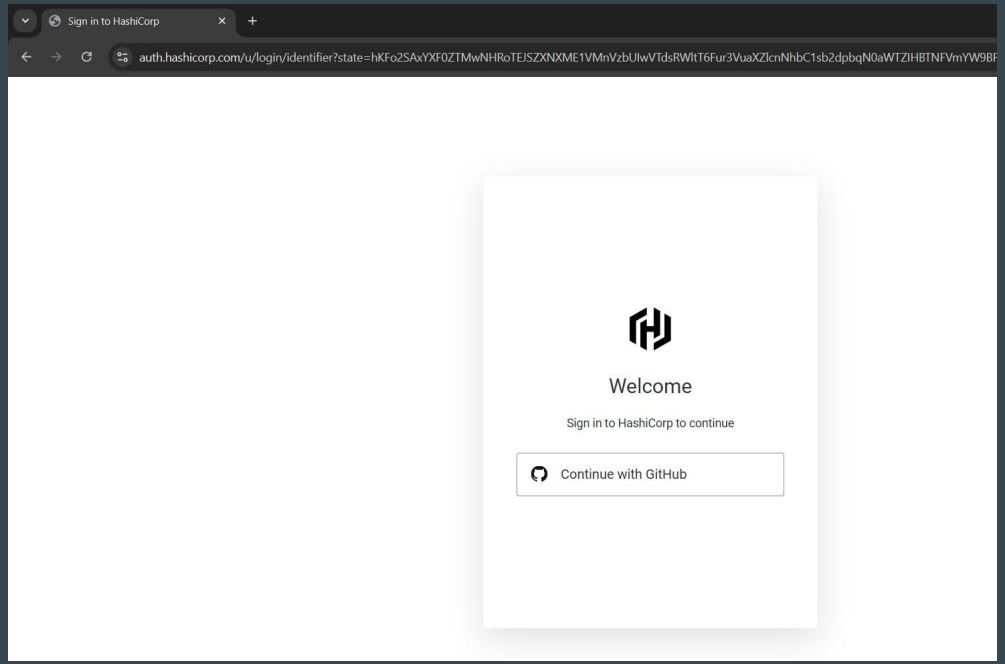


kplabs.in/linkedin

Exam appointment rules and requirements

Requirement 1 - GitHub

Exam portals are attached to your GitHub account. You will be prompted to login to GitHub when first creating your Exam Portal account.



Identification Requirements

You must have a unexpired government-issued physical ID card to enter your appointment. Digital IDs are not allowed.

All names on your ID must exactly match the names in your HashiCorp Certification Exam Portal account



Point to Note

Your account must have ALL names that are on your ID.

If you have a middle name, or more than one given or surname, these must ALL be accounted for in your account.

The names in your account must be in the same language and characters as the names on your ID.

Few Recommendations

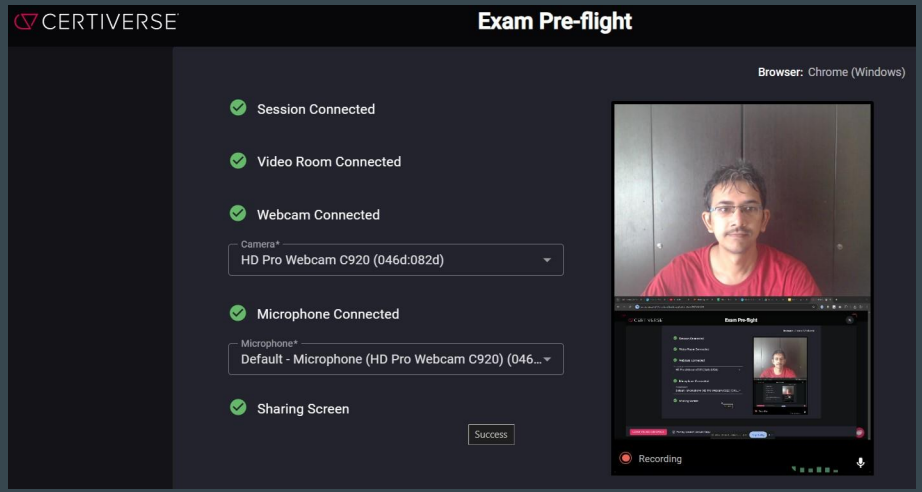
If text in ID are in small font, it might look blurry in the camera.

Keep a backup ID (preferably passport) as text are much larger than traditional identity proofs.

System Requirements

You will take your HashiCorp certification exam on your computer through an online exam delivery and proctoring system called Certiverse.

Your system must comply with all system requirements listed on Certiverse's Knowledge Base and pass the Network Test



Important Points to Note

HashiCorp recommends using Google Chrome browser for exams.

You may only use one monitor during your exam appointment.

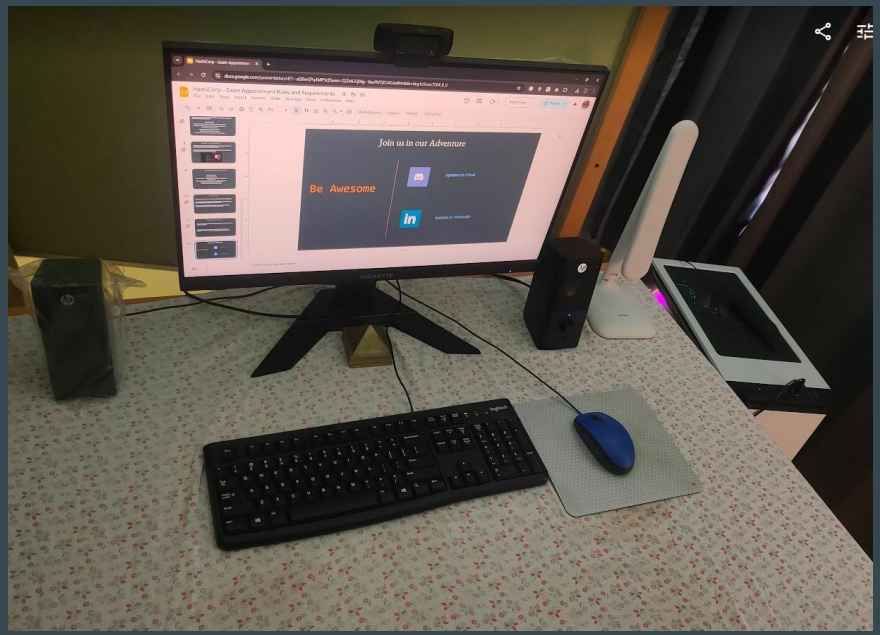
If you are using an external monitor, your laptop screen must be closed.

Requirement 3 - Physical Space

You will have to scan your exam environment to show compliance with these rules prior to accessing your exam.

Room Requirements
No one else is permitted to be in your testing room for the duration of your exam.
Be sure your space is adequately lit, so the proctor can see you and your space.
Your desk and work area must be clear.
Any electronics in the room must not be operational.
Background noise must be as limited as possible.
No phones, smartwatches, or other similar devices are allowed in the room.

Example - My Physical Desk



Important Behavior Requirements

Behavior Requirements
You may not leave your seat.
If your exam allows a break (Professional-level exams), follow all proctor prompts regarding your break.
Talking or communicating outside of the exam environment is not allowed during your appointment, including reading or mouthing questions.
Beverages must be in clear containers with no writing.

Points to Note

To make your check in easy, remove as much clutter as you find reasonable.

Your proctor will be looking for places recording devices could be hidden; the fewer of these present, the easier your check in will be.

If your proctor has a question about a part of your room, they may ask to examine the area via the webcam to protect exam security

Scheduling Requirements

If you need to cancel or reschedule your appointment, you must do so at least 48 hours in advance.

Cancellation with less than 48 hours notice or not appearing for your appointment without documentation may result in loss of your exam fees.

Point to Note

Make sure there are no notification that appear on your screen while giving the exams.

Always verify the updated guidelines released by HashiCorp for the exams to ensure you get the latest update before sitting for the exams.

Exam Preparation - Part 1

Providers in AWS

A provider is responsible for understanding API interactions and exposing resources.

When we run terraform init, plugins required for the provider are automatically downloaded and saved locally to a .terraform directory.

```
C:\Users\zealv\Desktop\kplabs-terraform>terraform init

Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v4.60.0...
- Installed hashicorp/aws v4.60.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Interesting Question

Is provider block {..} mandatory to be added as part of your Terraform configuration? Yes/No

```
provider "aws" {
  region = "us-east-1"
}
```

Two Pointers from Documentation

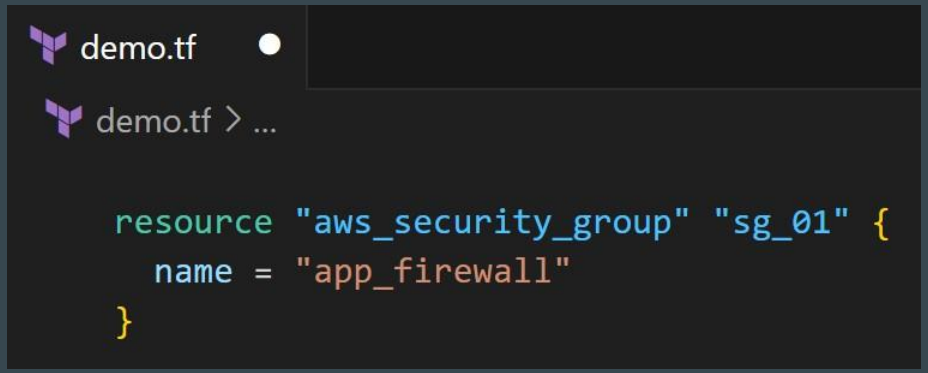
All Terraform configurations must declare which providers they require so that Terraform can install and use them.

A provider block may be omitted if its contents would otherwise be empty.

Concluding this Question

If you plan to explicitly add some contents to provider {} like region, credentials, then defining this block is required.

Otherwise, even if you skip, the terraform apply will work fine.



Alias in Providers

alias can be used for using the same provider with different configurations for different resources

```
provider "aws" {
    region = "ap-southeast-1"
}

provider "aws" {
    alias  = "mumbai"
    region = "ap-south-1"
}

provider "aws" {
    alias = "usa"
    region = "us-east-1"
}
```

```
resource "aws_instance" "myec2" {
  provider = aws.mumbai
  ami           = "ami-08a0d1e16fc3f61ea"
  instance_type = "t2.micro"
}
```

```
resource "aws_security_group" "allow_tls" {
  provider    =  aws.usa
  name        = "staging_firewall"
}
```

Point to Note - Providers

It is a good practice to store the credentials outside of Terraform configuration, such as in Environment Variables.

Terraform Settings

Terraform Settings are used to configure project-specific Terraform behaviours, such as requiring a minimum Terraform version to apply to your configuration.

```
terraform {
  required_version = "2.0"

  required_providers {
    aws = {
      version = "5.54.1"
      source = "hashicorp/aws"
    }
  }
}
```

Options That Can be Defined
Required Terraform Version
Required Provider and Version
BackEnd Configuration
Experimental Features

Point to Note

You cannot define configuration related to regions, Access/Secret keys inside required_provider block.

For these, you have to use a provider {} block.

Versioning Constraint

Version Constraint allows you to specify mix of multiple operators to select a suitable version of Terraform and Provider Plugins.

Operators and Examples	Description
>=1.0	Greater than equal to the version
<=1.0	Less than equal to the version
~>2.0	Any version in the 2.X range.
>=2.10,<=2.30	Any version between 2.10 and 2.30

Provider Tiers

There are 3 primary type of provider tiers in Terraform.

Provider Tiers	Description
Official	Owned and Maintained by HashiCorp.
Partner	Owned and Maintained by Technology Company that maintains direct partnership with HashiCorp.
Community	Owned and Maintained by Individual Contributors.

Terraform Init

The terraform init command initializes a working directory.

Initialization includes Installing Provider Plugins, Backend initialization, Copy Source Module, etc.

This is the first command that should be run after writing a new Terraform configuration. It is safe to run multiple times.

```
C:\Users\zealv\Desktop\kplabs-terraform>terraform init

Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/aws...
- Installing hashicorp/aws v4.60.0...
- Installed hashicorp/aws v4.60.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Terraform Init Upgrade

The terraform init -upgrade installs the latest module and provider versions allowed within configured constraints.

If you have latest provider plugins installed and if you define new version constraints that matches different version, you will have to run the init -upgrade

```
C:\kplabs-terraform>terraform plan

  Error: Inconsistent dependency lock file

  The following dependency selections recorded in the lock file are inconsistent with the current configuration:
    - provider registry.terraform.io/hashicorp/aws: locked version selection 5.54.1 doesn't match the updated version
ints "5.50.1"

  To update the locked dependency selections to match a changed configuration, run:
    terraform init -upgrade
```

Terraform Plan

The terraform plan command is used to create an execution plan.

The infrastructure is not modified as part of this plan.

The state file is not modified even when it detects drift in real-world and current infrastructure.

```
C:\kplabs-terraform>terraform plan

Terraform used the selected providers to generate the following
following symbols:
  + create

Terraform will perform the following actions:

  # aws_security_group.sg_01 will be created
  + resource "aws_security_group" "sg_01" {
      + arn                    = (known after apply)
      + description            = "Managed by Terraform"
      + egress                 = (known after apply)
      + id                     = (known after apply)
      + ingress                = (known after apply)
      + name                   = "app_firewall"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
```

Saving Plan to File

You can use the optional -out=FILE option to save the generated plan to a file on disk, which you can later execute by passing the file to terraform apply as an extra argument.

This ensures consistent infrastructure as defined in the plan.

terraform plan -out ec2.plan



```
Saved the plan to: ec2.plan

To perform exactly these actions, run the following command to apply:
    terraform apply "ec2.plan"
```

Terraform Apply

terraform apply command is used to apply the changes required to reach the desired state of the configuration.

The state file gets modified in this command.

Name of state file = terraform.tfstate

Terraform Apply can change, destroy and provision resources but cannot import any resource.

Terraform Destroy

terraform destroy command is used to destroy the Terraform-managed infrastructure.

terraform destroy command is not the only command through which infrastructure can be destroyed.

Terraform Format

terraform fmt command is used to rewrite Terraform configuration files to a canonical format and style. It will directly perform “write” operation and not “read’

For use-case, where the all configuration written by team members needs to have a proper style of code, terraform fmt can be used.

```
resource "aws_instance" "myec2" {
  ami = "ami-00c39f71452c08778"
       instance_type = "t2.micro"
      tags ={
    Name = "test-ec2"
  }
}
```

Before


```
resource "aws_instance" "myec2" {
  ami           = "ami-00c39f71452c08778"
  instance_type = "t2.micro"
  tags = {
    Name = "test-ec2"
  }
}
```

After

terraform fmt options

You have to keep a note of two important flags for terraform fmt command

Command	Description
-check	Check if the input is formatted. Files are not modified.
-recursive	Also process files in subdirectories. By default, only the given directory (or current directory) is processed.

Exam Preparation - Part 2

Terraform Validate

terraform validate command validates the configuration files in a directory.

Requires an initialized working directory with any referenced plugins and modules installed.

terraform plan uses implied validation check.

```
C:\kplabs-terraform>terraform validate
Success! The configuration is valid.
```

Terraform Refresh

terraform refresh command reads the current settings from all managed remote objects and updates the Terraform state to match.

This won't modify your real remote objects, but it will modify the Terraform state.

This command is deprecated, because its default behavior is unsafe.

Resource Blocks

A resource block declares a resource of a given type ("aws_instance") with a given local name ("web").

Resource type and Name together serve as an identifier for a given resource and so must be unique.

Address of the following resource is: aws_instance.web

```
resource "aws_instance" "web" {
  ami           = ami-123
  instance_type = "t2.micro"
}
```

Important Terminology

```
resource "aws_instance" "myec2" {
  ami           = "ami-123"
  instance_type = "t2.micro"
}
```


aws_instance	Resource Type
myec2	Local name for the resource
ami	Argument Name
ami-123	Argument value

Data Types in Terraform

Data Types	Description
string	a sequence of Unicode characters representing some text, like "hello".
number	A Numeric value
bool	a boolean value, either true or false
list	a sequence of values, like ["us-west-1a", "us-west-1c"]
set	a collection of unique values that do not have any secondary identifiers or ordering.
map	a group of values identified by named labels, like {name = "Mabel", age = 52}.
null	a value that represents absence or omission.

Point to Note - Data Types

Array data types are not supported in Terraform

State Management

The terraform state command is used for advanced state management

Sub-Commands	Description
list	List resources within terraform state file.
mv	Moves item with terraform state.
pull	Manually download and output the state from remote state.
push	Manually upload a local state file to remote state.
rm	Remove items from the Terraform state
show	Show the attributes of a single resource in the state.
replace-provider	Used to replace the provider for resources in a Terraform state.

Use-Case - Removing Item from State

There are 5 EC2 instances created through Terraform using count =5

The Team wants to destroy all the EC2 instances except the second instance with the resource address of aws_instance.web[1].

1. This is not possible since the instance created through Count.
2. Apply taint on the EC2 instance.
3. Use terraform state rm aws_instance.web[1]
4. Use terraform state mv aws_instance.web[1]
5. None of the Above

Debugging in Terraform

Terraform has detailed logs that can be enabled by setting the TF_LOG environment variable to any value.

You can set TF_LOG to one of the log levels TRACE, DEBUG, INFO, WARN or ERROR to change the verbosity of the logs.

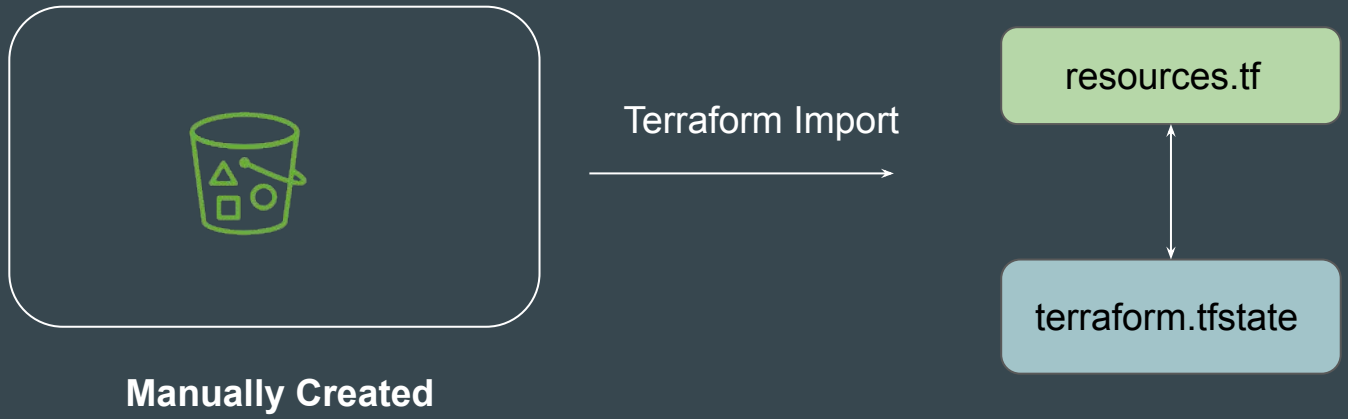
To persist logged output, you can set TF_LOG_PATH

Terraform Import

Allows importing existing infrastructure to Terraform.

Automatic code generation for imported resources is supported.

You can use import blocks to import more than one resource at a time.



Import Workflow Steps

```
import {
  to = aws_security_group.mysg
  id = "sg-07f13feb262ba8b6f"
}

# terraform plan -generate-config-out="mysg.tf"

# terraform apply -auto-approve
```

Local Values

Locals are used when you want to avoid repeating the same expression multiple times.

Local values are created by a locals block (plural), but you reference them as attributes on an object named local (singular)

Local value can reference values from other variables, locals etc.

```
locals {
  common_tags = {
    Team = "payments-team"
 }
}
```

Terraform Workspace

Terraform workspaces enable us to manage multiple sets of deployments from the same sets of configuration files.

State File Directory = terraform.tfstate.d

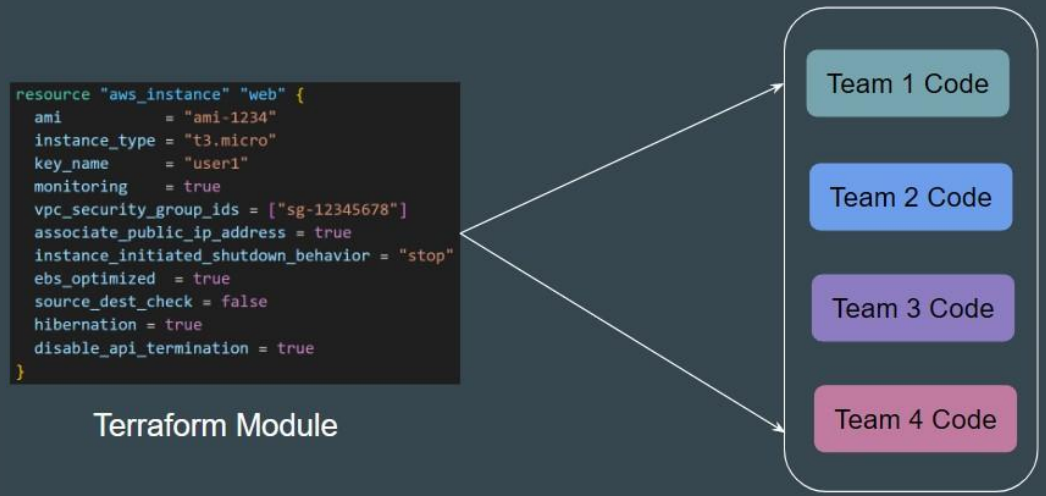
Not suitable for isolation for strong separation between workspace (stage/prod)

Use-Case	Command
Create New Workspace	terraform workspace new kplabs
Switch to a specific Workspace	terraform workspace select prod

Terraform Modules

Terraform Modules allow us to centralize the resource configuration, and it makes it easier for multiple projects to re-use the Terraform code.

Instead of writing code from scratch, we can use multiple ready-made modules available.



Calling a Module

Module source code can be present in a wide variety of locations including:

GitHub, Local Paths, Terraform Registry, S3 Buckets, HTTP URLs

To reference a module, you need to make use of module block and source

Terraform uses this during the module installation step of terraform init to download the source code to a directory on local disk so that other Terraform commands can use it.

```
module "ec2" {
  source = "https://example.com/vpc-module.zip"
}
```

Example 1 - Local Paths

Local paths are used to reference to a module that is available in local filesystem.

A local path must begin with either ./ or ../ to indicate that a local path.

Modules sourced from local paths do NOT support versions.

```
module "ec2" {
  source = "../modules/ec2"
}
```

Example 2 - Generic Git Repository

Arbitrary Git repositories can be used by prefixing the address with the special git:: prefix.

```
module "vpc" {
  source = "git::https://example.com/vpc.git"
}
```

Root vs Child Modules

Root Module resides in the main working directory of your Terraform configuration. This is the entry point for your infrastructure definition

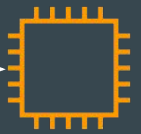
A module that has been called by another module is often referred to as a child module.

```
resource "aws_instance" "myec2" {
    ami = var.ami
    instance_type = var.instance_type
}
```

Child Module

```
module "ec2" {
  source = "../../modules/ec2"
}
```

Root Module



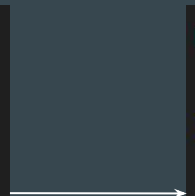
Module Outputs

A child module can use outputs to expose a subset of its resource attributes to a parent module.

Format: <MODULE NAME>.<OUTPUT NAME>

```
resource "aws_instance" "myec2" {
    ami = var.ami
    instance_type = var.instance_type
}

output "instance_id" {
  value = aws_instance.myec2.id
}
```



```
module "ec2" {
  source = "../../modules/ec2"
}

resource "aws_eip" "lb" {
  instance = module.ec2.instance_id
  domain   = "vpc"
}
```

Module Versioning

When using modules installed from a module registry, HashiCorp recommends explicitly constraining the acceptable version numbers to avoid unexpected or unwanted changes.

It is not mandatory to specify a version argument.

```
module "ec2-instance" {
  source  = "terraform-aws-modules/ec2-instance/aws"
  version = "5.6.1"
}
```

Terraform Registry

- Hosts a broad collection of publicly available Terraform modules.
- Each Terraform module has an associated address.
- A module address has the syntax hostname/namespace/name/system

The hostname/ portion of a module is optional, and if omitted, it defaults to registry.terraform.io/.

```
module "ec2-instance" {
  source  = "terraform-aws-modules/ec2-instance/aws"
  version = "5.6.1"
}
```

Functions in Terraform

The Terraform language includes a number of built-in functions that you can use to transform and combine values.

NO SUPPORT for User-Defined Functions.

Function Categories	Functions Available
Numeric Functions	abs, ceil, floor, max, min
String Functions	concat, replace, split, join, tolower,toupper
Collection Functions	element, keys, length, merge, sort, slice
Filesystem Functions:	file, filebase64, dirname

Lookup function

lookup retrieves the value of a single element from a map, given its key. If the given key does not exist, the given default value is returned instead.

```
> lookup({a="ay", b="bee"}, "a", "what?")
ay
> lookup({a="ay", b="bee"}, "c", "what?")
what?
```

Zipmap function

zipmap constructs a map from a list of keys and a corresponding list of values.

```
←[J> zipmap(["pineapple","oranges","strawberry"], ["yellow","orange","red"])
{
  "oranges" = "orange"
  "pineapple" = "yellow"
  "strawberry" = "red"
}
```

Index function

index finds the element index for a given value in a list.

```
C:\kplabs-terraform>terraform console
> index(["a", "b", "c"], "b")
1
```

Element Function

element retrieves a single element from a list.

Format: element(list, index)

```
> element(["a", "b", "c"], 1)
b
```

Testing Element Function

Code: Name = element(var.tags,count.index)

```
variable "tags" {
  type = list
  default = ["firstec2","secondec2"]
}
```



```
> element(["firstec2","secondec2"],0)
"firstec2"
```

```
> element(["firstec2","secondec2"],1)
"secondec2"
```

toset Function

toset function will convert the list of values to SET

```
> toset(["a", "b", "c","a"])
toset([
  "a",
  "b",
  "c",
])
```

TimeStamp Function

timestamp returns a UTC timestamp string in RFC 3339 format.

```
> formatdate("DD MMM YYYY hh:mm ZZZ", "2018-01-02T23:12:01Z")
02 Jan 2018 23:12 UTC
> formatdate("EEEE, DD-MMM-YY hh:mm:ss ZZZ", "2018-01-02T23:12:01Z")
Tuesday, 02-Jan-18 23:12:01 UTC
> formatdate("EEE, DD MMM YYYY hh:mm:ss ZZZ", "2018-01-02T23:12:01-08:00")
Tue, 02 Jan 2018 23:12:01 -0800
> formatdate("MMM DD, YYYY", "2018-01-02T23:12:01Z")
Jan 02, 2018
> formatdate("HH:mmaa", "2018-01-02T23:12:01Z")
11:12pm
```

File Function

File function can reduce the overall Terraform code size by loading contents from external sources during terraform operations.

```
resource "aws_iam_user_policy" "lb_ro" {
  name = "demo-user-policy"
  user = aws_iam_user.this.name

  policy = jsonencode({
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": "ec2:*",
            "Effect": "Allow",
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": "elasticloadbalancing:*",
            "Resource": "*"
        },
```

Before



```
resource "aws_iam_user_policy" "lb_ro" {
  name = "demo-user-policy"
  user = aws_iam_user.this.name

  policy = file("./ec2-policy.json")
}
```

After

Meta Arguments in Terraform

Terraform allows us to include meta-arguments within the resource block, which allows some details of this standard resource behaviour to be customized on a per-resource basis.

Inside resource block


```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"

    lifecycle {
      ignore_changes = [tags]
  }
}
```

Different Meta-Arguments

Meta-Argument	Description
depends_on	Handle hidden resource or module dependencies that Terraform cannot automatically infer.
count	Accepts a whole number, and creates that many instances of the resource
for_each	Accepts a map or a set of strings, and creates an instance for each item in that map or set.
lifecycle	Allows modification of the resource lifecycle.
provider	Specifies which provider configuration to use for a resource, overriding Terraform's default behavior of selecting one based on the resource type name

Lifecycle Meta-Argument

Some details of the default resource behaviour can be customized using the special nested lifecycle block within a resource block body:

```
resource "aws_instance" "myec2" {
    ami = "ami-0f34c5ae932e6f0e4"
    instance_type = "t2.micro"

    tags = {
        Name = "HelloEarth"
    }

    lifecycle {
        ignore_changes = [tags]
    }
}
```

Arguments Available for LifeCycle Block

There are four argument available within lifecycle block.

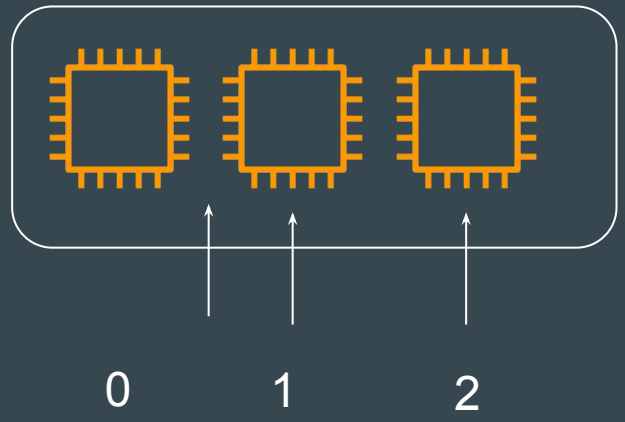
Arguments	Description
create_before_destroy	New replacement object is created first, and the prior object is destroyed after the replacement is created.
prevent_destroy	Terraform to reject with an error any plan that would destroy the infrastructure object associated with the resource
ignore_changes	Ignore certain changes to the live resource that does not match the configuration.
replace_triggered_by	Replaces the resource when any of the referenced items change

Count and Count Index

The count argument accepts a whole number, and creates that many instances of the resource.

count.index — The distinct index number (starting with 0) corresponding to this instance.

```
resource "aws_instance" "myec2" {
    ami = "ami-00c39f71452c08778"
    instance_type = "t2.micro"
    count = 3
}
```



Exam Preparation - Part 3

Find the Issue - Use-Cases

You can expect a use case in exam with a sample Terraform code, and you must find what should be removed as part of Terraform best practice.

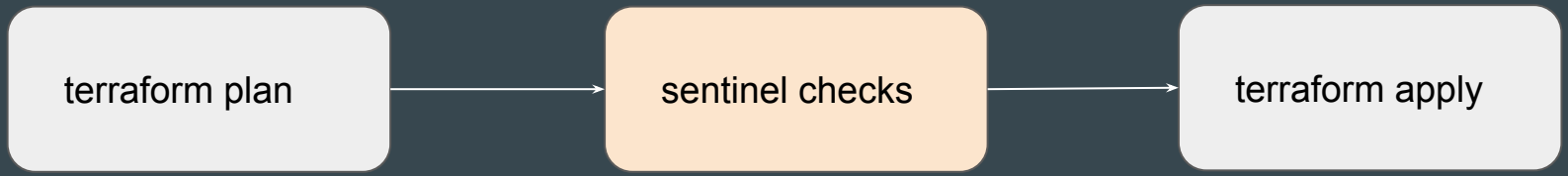
```
terraform {
  backend "s3" {
    bucket = "mybucket"
    key    = "path/to/my/key"
    region = "us-east-1"
    access_key = "AKIAIOSFODNN7EXAMPLE"
    aecret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEK"
  }
}
```

Sentinel

Sentinel is an embedded policy-as-code framework integrated with the HashiCorp Enterprise products. Sentinel is a proactive service.

Can be used for various use-cases like:

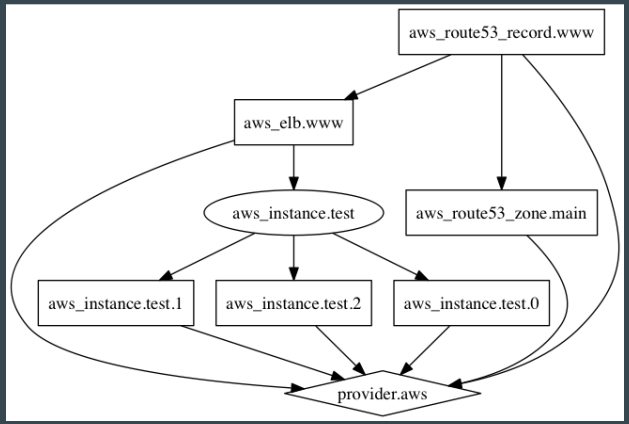
- Verify if EC2 instance has tags.
- Verify if the S3 bucket has encryption enabled.



Terraform Graph

Terraform graph refers to a visual representation of the dependency relationships between resources defined in your Terraform configuration.

The output of terraform graph is in the DOT format, which can easily be converted to an image.



Input Variables

Terraform input variables are used to pass certain values from outside of the configuration

Name	Value
vpn_ip	101.0.62.210/32
app_port	8080

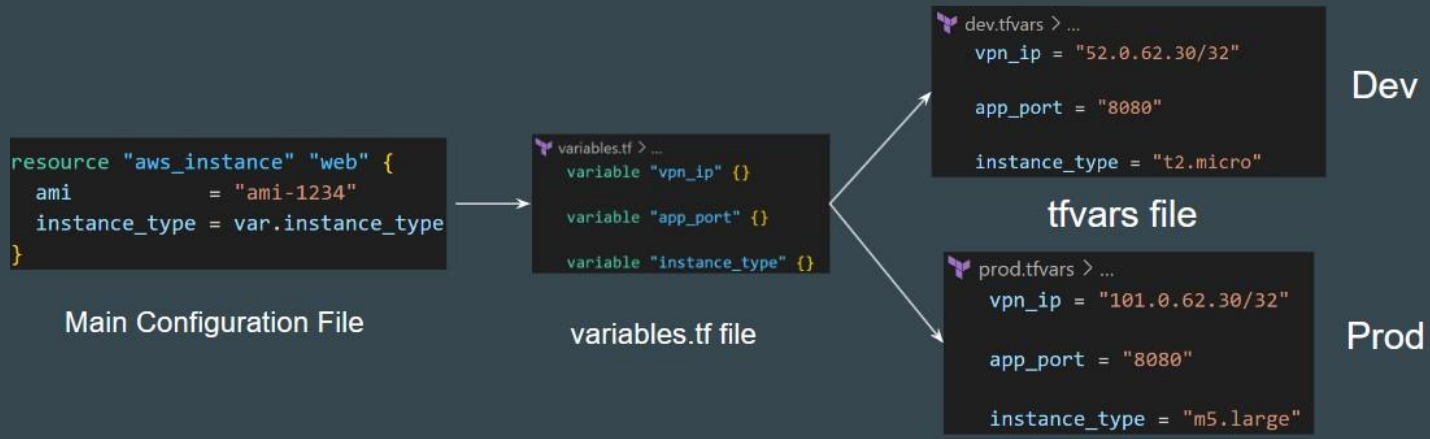
Variable File


```
resource "aws_vpc_security_group_ingress_rule" "allow_tls_ipv4" {
  security_group_id = aws_security_group.allow_tls.id
  cidr_ipv4         = var.vpn_ip
  from_port         = var.app_port
  ip_protocol       = "tcp"
  to_port           = var.app_port
}
```

Terraform TFVARS

terraform.tfvars file can be used to define value to all the variables.

This approach leads to easier setup for multi-project deployments.



Selecting tfvars File

If you have multiple variable definitions file (*.tfvars) file, you can manually define the file to use during the command line.

```
C:\Users\zealv\kplabs-terraform>terraform plan -var-file="prod.tfvars"

Terraform used the selected providers to generate the following execution plan.
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami                                  = "ami-1234"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
```

Declaring Variable Values

When variables are declared in your configuration, they can be set in a number of ways:

1. Variable Defaults.
2. Variable Definition File (*.tfvars)
3. Environment Variables
4. Setting Variables in the Command Line.

variables.tf > ...

```
variable "app_port" {
    default = "8080"
}
```

prod.tfvars > ...

```
vpn_ip = "101.0.62.30/32"

app_port = "8080"

instance_type = "m5.large"
```

Setting Variable in Command Line

To specify individual variables on the command line, use the -var option when running the terraform plan and terraform apply commands:

```
C:\Users\zealv\kplabs-terraform>terraform plan -var="instance_type=m5.large"

Terraform used the selected providers to generate the following execution plan. Resource
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami                                  = "ami-1234"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "m5.large"
```

Setting Variable through Environment Variables

Terraform searches the environment of its own process for environment variables named TF_VAR_ followed by the name of a declared variable.

```
C:\Users\zealv\kplabs-terraform>echo %TF_VAR_instance_type%
t2.large

C:\Users\zealv\kplabs-terraform>terraform plan

Terraform used the selected providers to generate the following execution plan
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.web will be created
  + resource "aws_instance" "web" {
      + ami                                  = "ami-1234"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
```

Variable Definition Precedence

Terraform loads variables in the following order, with later sources taking precedence over earlier ones:

1. Environment variables
2. The terraform.tfvars file, if present.
3. The terraform.tfvars.json file, if present.

4. Any *.auto.tfvars or *.auto.tfvars.json files, processed in lexical order of their filenames.

5. Any -var and -var-file options on the command line

Variables with undefined values

If you have variables with undefined values, it will NOT directly result in an error.

Terraform will ask you to supply the value associated with them.

```
C:\Users\zealv\kplabs-terraform>terraform plan
var.instance_type
  Enter a value:
```

Not Allowed Variable Names

We cannot use all words within variable names.

Terraform reserves some additional names that can no longer be used as input variable names for modules. These reserved names are:

- count
- depends_on
- for_each
- lifecycle
- providers
- source

```
Error: Invalid variable name

  on demo.tf line 6, in variable "count":
   6: variable "count" {

The variable name "count" is reserved due to its special meaning
```

Points to Note - State File

Terraform state file generally stores details about the resources that it manages.

Various aspects like “Input Variables” are not stored.

Output value will be stored in state file but not the description

```
variable "instance_type" {
  type =    string
  default = "t2.micro"
  description = "Default EC2 Instance type for Module"
}

output "variable" {
  value = var.instance_type
  description = "Instance Type of EC2"
}
```

Terraform Console

Terraform Console provides an interactive environment specifically designed to test functions and experiment with expressions before integrating them into your main code.

```
C:\Users\zealv\kplabs-terraform>terraform console
> max(10,20,30)
30
```

Dependency Lock File

Terraform dependency lock file allows us to lock to a specific version of the provider. Name is terraform.lock.hcl. For tracking provider dependencies.

If a particular provider already has a selection recorded in the lock file, Terraform will always re-select that version for installation, even if a newer version has become available.

You can override that behavior by running terraform init -upgrade

Dependencies - Implicit

With implicit dependency, Terraform can automatically find references of the object, and create an implicit ordering requirement between the two resources.

In the following screenshot, Terraform will create EC2 first before EIP.

```
resource "aws_eip" "lb" {
  domain   = "vpc"
  instance = aws_instance.myec2.id
}

resource "aws_instance" "myec2" {
  ami           = "ami-123"
  instance_type = "t2.micro"
}
```

Dependencies - Explicit

Explicitly specifying a dependency is only necessary when a resource relies on some other resource's behavior but doesn't access any of that resource's data in its arguments.

Uses the depends_on meta-argument

```
resource "aws_iam_role" "example" {
  name = "example"
  assume_role_policy = "..."
}

resource "aws_instance" "example" {
  ami           = "ami-a1b2c3d4"
  instance_type = "t2.micro"
  depends_on = [aws_iam_role_policy.example]
}
```

Data Sources

Data sources allow Terraform to use / fetch information defined outside of Terraform

```
data "aws_instances" "example" {}
```

```
data "local_file" "foo" {
  filename = "${path.module}/demo.txt"
}
```

Terraform Enterprise

Terraform Enterprise provides several added advantages compared to Terraform Cloud.

Some of these include:

- Single Sign-On
- Auditing
- Private Data Center Networking
- Clustering

Team & Governance features are not available for Terraform Cloud Free (Paid)

Remote Backend

The remote backend stores Terraform state and may be used to run operations in Terraform Cloud.

When using full remote operations, operations like terraform plan or terraform apply can be executed in Terraform Cloud's run environment, with log output streaming to the local terminal.

The remote backend was the primary implementation of HCP Terraform's CLI-driven run workflow for Terraform versions 0.11.13 through 1.0.x. We recommend using the native cloud integration for Terraform versions 1.1 or later, as it provides an improved user experience and various enhancements.

Points to Note

HCP = HashiCorp Cloud Platform

Secure Variable Storage is available in Terraform Enterprise and Cloud but not in the normal version of Terraform.

Terraform Cloud / Enterprise comes with a Private Module registry which allows organizations to restrict access based on requirements.

Terraform Cloud provides the feature of Remote State storage.

Encryption of state file is available

Points to Note

In a HCP workspace linked to a VCS repository, runs start automatically when you merge or commit changes to version control.

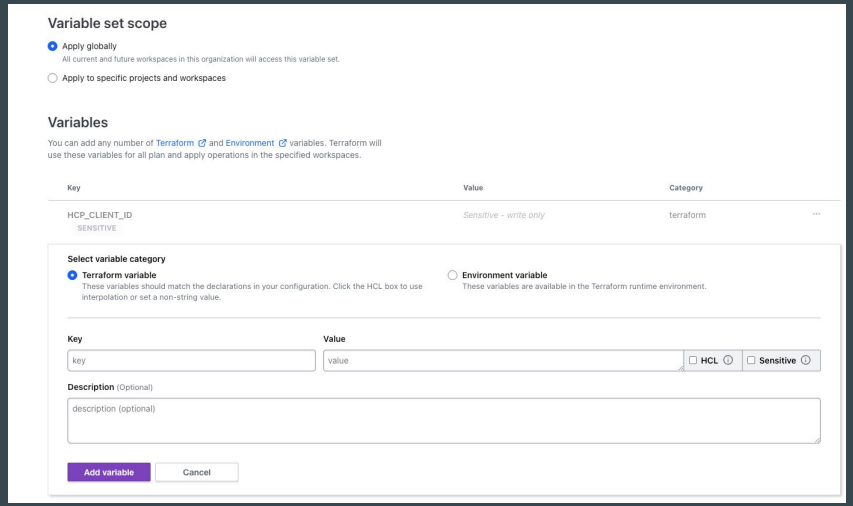
A workspace is linked to one branch of a VCS repository and ignores changes to other branches.

To protect secret values in HCP, you can mark any Terraform or environment variable as sensitive data by clicking its Sensitive checkbox that is visible during editing. Marking a variable as sensitive makes it write-only and prevents all users (including you) from viewing its value

Sensitive Values in HCP

To protect secret values in HCP, you can mark any Terraform or environment variable as sensitive data by clicking its Sensitive checkbox that is visible during editing.

Marking a variable as sensitive makes it write-only and prevents all users (including you) from viewing its value

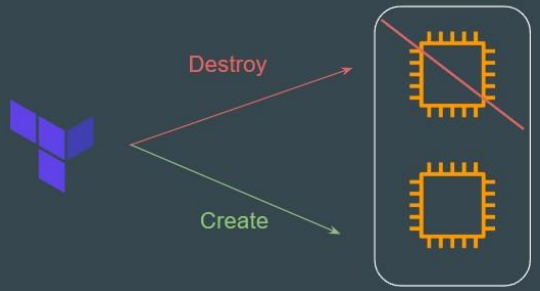


Recreating the Resource

The -replace option with terraform apply to force Terraform to replace an object even though there are no configuration changes that would require it.

terraform apply -replace="aws_instance.web"

A similar kind of functionality was achieved using the terraform taint command in older versions of Terraform. Not recommended now.



Benefits of IAC Tool

There are three primary benefits of Infrastructure as Code tools:

Automation, Versioning, and Reusability.

Various IAC Tools Available in the market:

- Terraform
- CloudFormation
- Azure Resource Manager
- Google Cloud Deployment Manager

terraform output

The terraform output command is used to extract the value of an output variable from the state file.

```
variable "instance_type" {
  type =    string
  default = "t2.micro"
  description = "Default EC2 Instance type for Module"
}

output "instance_type" {
  value = var.instance_type
  description = "Instance Type of EC2"
}
```



```
C:\kplabs-terraform>terraform output
instance_type = "t2.small"
```

Module Source and Git Branches

By default, Terraform will clone and use the default branch (referenced by HEAD) in the selected repository.

You can override this using the ref argument.

Format: ?ref=<version-number>

```
module "vpc" {
  source = "git::https://example.com/vpc.git?ref=v1.2.0"
}
```

Splat Expressions

Splat Expression allows us to get a list of all the attributes.

Resources that use the for_each argument will appear in expressions as a map of objects, so you can't use splat expressions with those resources.

```
resource "aws_iam_user" "lb" {
  name = "iamuser.${count.index}"
  count = 3
  path = "/system/"
}

output "arns" {
  value = aws_iam_user.lb[*].arn
}
```

Important Documentation Reference - Splat

A *splat expression* provides a more concise way to express a common operation that could otherwise be performed with a `for` expression.

If `var.list` is a list of objects that all have an attribute `id`, then a list of the ids could be produced with the following `for` expression:

```
[for o in var.list : o.id]
```

This is equivalent to the following *splat expression:*

```
var.list[*].id
```

Point to Note

Will this code block display the names of all the IAM usernames created?

Answer = NO

```
resource "aws_iam_user" "lb" {
  name = "iamuser.${count.index}"
  count = 3
  path = "/system/"
}

output "arns" {
  value = aws_iam_user.lb.[*].name
}
```

Legacy Splat Expression

Earlier versions of the Terraform language had a slightly different version of splat expressions, which Terraform continues to support for backward compatibility.

The legacy "attribute-only" splat expressions use the sequence .*, instead of [*]:

```
resource "aws_iam_user" "lb" {
  name = "iamuser.${count.index}"
  count = 3
  path = "/system/"
}

output "arns" {
  value = aws_iam_user.lb.*.name
}
```

Fetching Values from List

To fetch the instance_type value of m5.xlarge from the list, you can reference to the key of 1

var.size[1]

```
variable "size" {
  type = list
  default = ["m5.large","m5.xlarge","t2.medium"]
}


resource "aws_instance" "myec2" {
   ami = "ami-082b5a644766e0e6f"
   instance_type = var.size[1]
}
```

Fetching Values from Map

To reference the “t2.small” instance type from the below map, the following approaches need to be used:

var.types[“ap-south-1”]

```
variable "types" {
  type = map
  default = {
    us-east-1 = "t2.micro"
    us-west-2 = "t2.nano"
    ap-south-1 = "t2.small"
  }
}


resource "aws_instance" "myec2" {
   ami = "ami-082b5a644766e0e6f"
   instance_type = var.types["ap-south-1"]
}
```

Dealing with Larger Infrastructure

Cloud Providers have set a certain rate limit, so Terraform can only request a certain number of resources over a period of time.

It is important to break larger configurations into multiple smaller configurations that can be independently applied.

Alternatively, you can make use of -refresh=false and target flag for a workaround (not recommended)

BackEnds

Backends primarily determine where Terraform stores its state.

By default, Terraform implicitly uses a backend called local to store state as a local file on disk.

If required, you can store state file in other backend as well.

```
terraform {
  backend "s3" {
    bucket = "mybucket"
    key    = "path/to/my/key"
    region = "us-east-1"
  }
}
```

Points to Note - Initializing Backend

When configuring a backend for the first time (moving from no defined backend to explicitly configuring one), Terraform will give you the option to migrate your state to the new backend.

This lets you adopt new backends without losing any existing state.

Local Backend

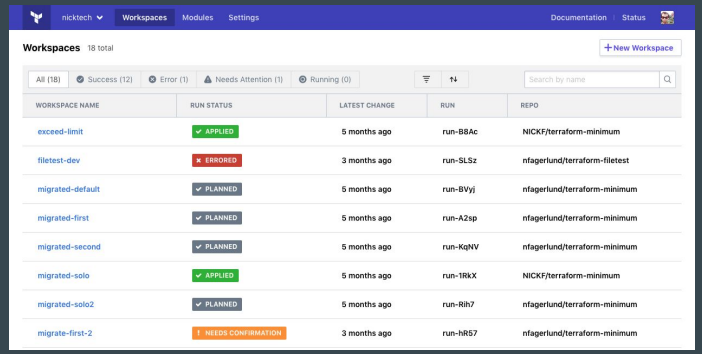
The local backend stores the state on the local filesystem, locks that state using system APIs, and performs operations locally.

By default, Terraform uses the "local" backend, which is the normal behavior of Terraform you're used to

```
terraform {
  backend "local" {
    path = "relative/path/to/terraform.tfstate"
  }
}
```

Air Gapped Environments

An air gap is a network security measure employed to ensure that a secure computer network is physically isolated from unsecured networks, such as the public Internet.

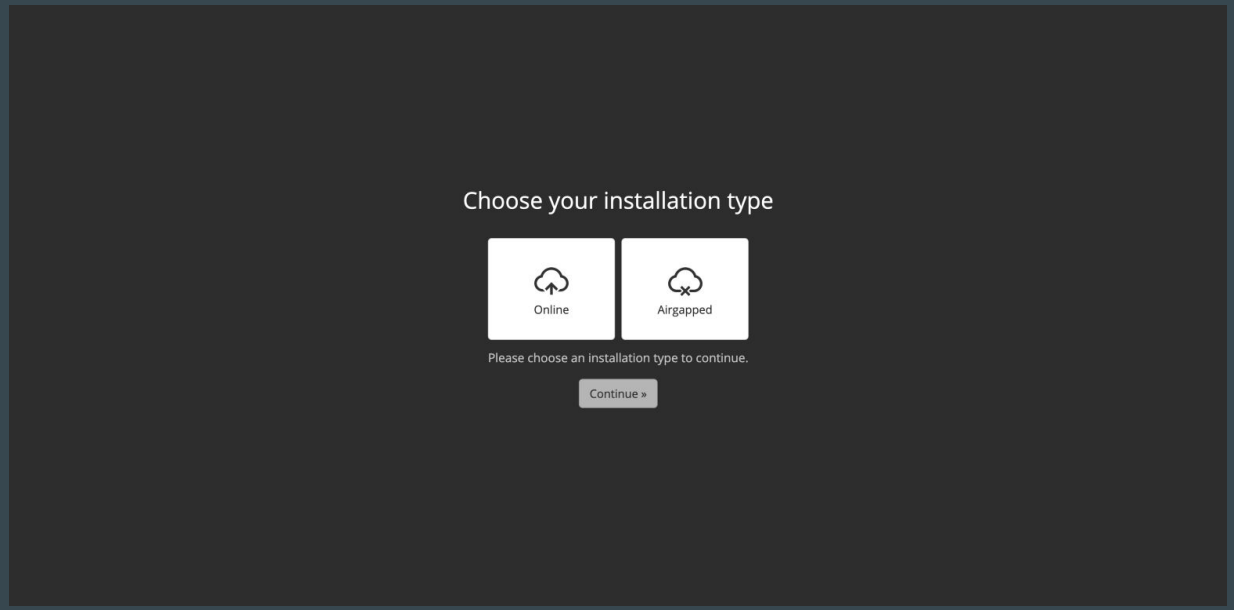


Terraform Enterprise

Air Gap Install



Isolated Server


Choose your installation type
Online
Airgapped
Please choose an installation type to continue.
Continue »

Requirements for Publishing Module in Registry

Core Requirements	Description
GitHub	The module must be on GitHub and must be a public repo. This is only a requirement for the public registry and not for private registry.
Named terraform-<PROVIDER>-<NAME>	Example: terraform-aws-ec2-instance
Repository description	The GitHub repository description is used to populate the short description of the module.
Standard module structure	he module must adhere to the standard module structure. This allows the registry to inspect your module and generate documentation, track resource usage, parse submodules and examples, and more.
x.y.z tags for releases	For example, v1.0.4 and 0.9.2

Comments in Terraform

A comment is a text note added to the source code to provide explanatory information, usually about the function of the code.

```
/* This is Terraform Backend
State file stored in S3
*/

terraform {
  backend "s3" {
    bucket = "mybucket"
    key    = "path/to/my/key"
    region = "us-east-1"
  }
}

#  This is single line comment
// This is single line comment
```

Dynamic Blocks

Dynamic Block allows us to dynamically construct repeatable nested blocks, which are supported inside resource, data, provider, and provisioner blocks.

Overuse of Dynamic blocks can make configuration hard to read and maintain.

```
variable "sg_ports" {
  type        = list(number)
  description = "list of ingress ports"
  default     = [8200, 8201,8300, 9200, 9500]
}

resource "aws_security_group" "dynamicsg" {
  name        = "dynamic-sg"

  dynamic "ingress" {
    for_each = var.sg_ports
    iterator = port
    content {
      from_port   = port.value
      to_port     = port.value
      protocol    = "tcp"
      cidr_blocks = ["0.0.0.0/0"]
    }
  }
```

Miscellaneous Pointers

GitHub is not the supported backend type in Terraform.

API and CLI access for Terraform Cloud can be managed through API tokens that can be generated from Terraform Cloud UI.

Terraform uses Parallelism to reduce the time it takes to create the resource. By default, this value is set to 10

Code Formatting Recommended Practices

Indent two spaces for each nesting level

When multiple arguments with single-line values appear on consecutive lines at the same nesting level, align their equals signs:

```
ami           = "abc123"
instance_type = "t2.micro"
```

Miscellaneous Pointers

Terraform does not require go as a prerequisite.

Terraform providers are NOT always installed through the internet. There is a different offline approach for air-gapped systems.

Terraform and Terraform Provider NEED NOT have the same major version for compatibility.

Sensitive Parameter

Adding sensitive parameter ensures that you do not accidentally expose this data in CLI output, log output.

The sensitive value will be present as part of the state file.

```
variable "sensitive_content" {
  sensitive = true
  default = "supersecretpassw0rd"
}

resource "local_file" "foo" {
  content  = var.sensitive_content
  filename = "password.txt"
}
```



```
Terraform will perform the following actions:

  # local_file.foo will be created
  + resource "local_file" "foo" {
      + content              = (sensitive value)
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "password.txt"
      + id                   = (known after apply)
    }
```

Actions Forbidden When State File is Locked

When the state file is locked, the following actions are forbidden:

Running terraform apply or any other command that modifies the state file (e.g. terraform plan, terraform destroy, etc.)

Running terraform refresh, which updates the state file to reflect the current state of the infrastructure

terraform state [push, rm, mv]

Miscellaneous Pointers

1. terraform.tfstate will NOT always match the current state infrastructure.

If you are making use of the GIT repository for commiting terraform code, the .gitignore should be configured to ignore certain terraform files that might contain sensitive data.

Some of these can include:

terraform.tfstate file (this can include sensitive information)

*.tfvars (may contain sensitive data like passwords)

Points to Note - State

1. If supported by your backend, Terraform will lock your state for all operations that could write state.

2. Not all backends support locking functionality.

3. Terraform has a force-unlock command to manually unlock the state if unlocking failed. *terraform force-unlock LOCK_ID [DIR]*

Join us in our Adventure

Be Awesome



kplabs.in/chat

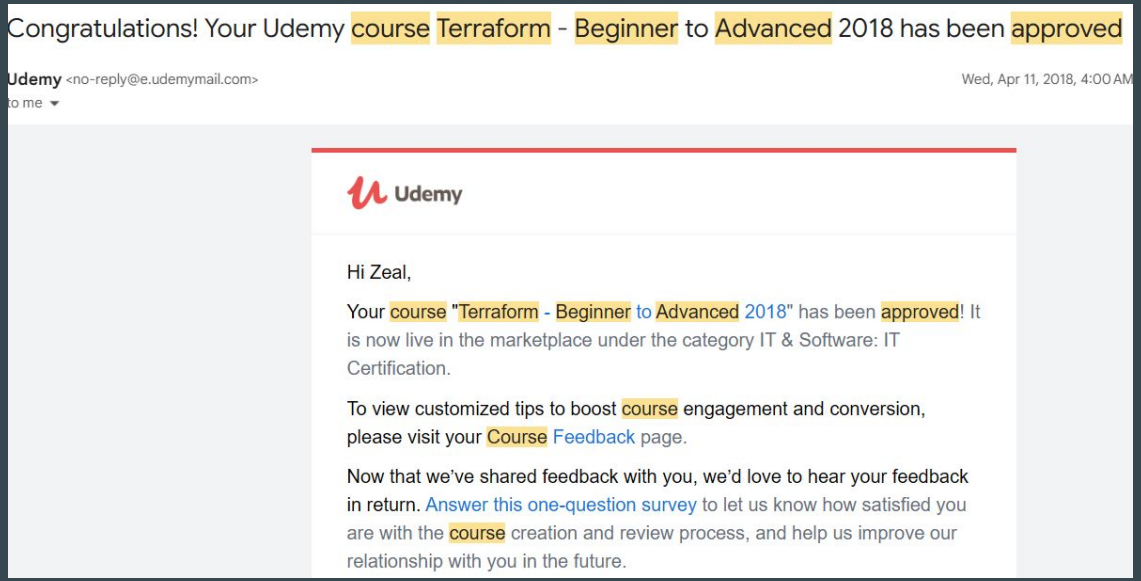


kplabs.in/linkedin

What’s Next - Terraform Professional

Our First Terraform Course

We had launched our first Terraform course in 2018.



Congratulations! Your Udemy course Terraform - Beginner to Advanced 2018 has been approved

Udemy <no-reply@e.udemymail.com> Wed, Apr 11, 2018, 4:00 AM

to me

Udemy

Hi Zeal,

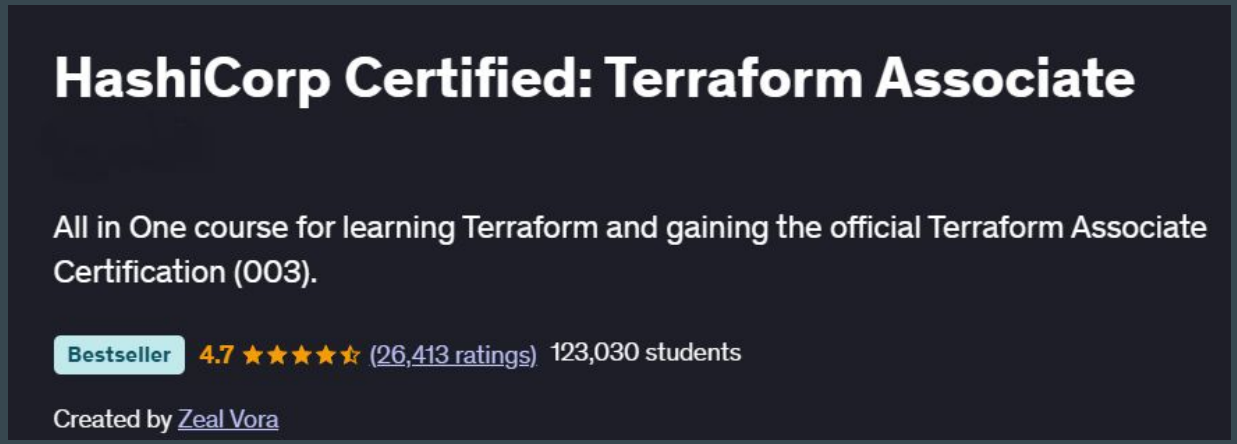
Your course "Terraform - Beginner to Advanced 2018" has been approved! It is now live in the marketplace under the category IT & Software: IT Certification.

To view customized tips to boost course engagement and conversion, please visit your Course Feedback page.

Now that we've shared feedback with you, we'd love to hear your feedback in return. Answer this one-question survey to let us know how satisfied you are with the course creation and review process, and help us improve our relationship with you in the future.

Setting the Base

This course goes above and beyond to explain all the topics of Terraform Associate certification in a practical way that will help you in real world use-cases.



Terraform Associate Exam

Terraform Associate certification is primarily an MCQ-based exam and is relatively easier to pass even by memorizing the concepts.



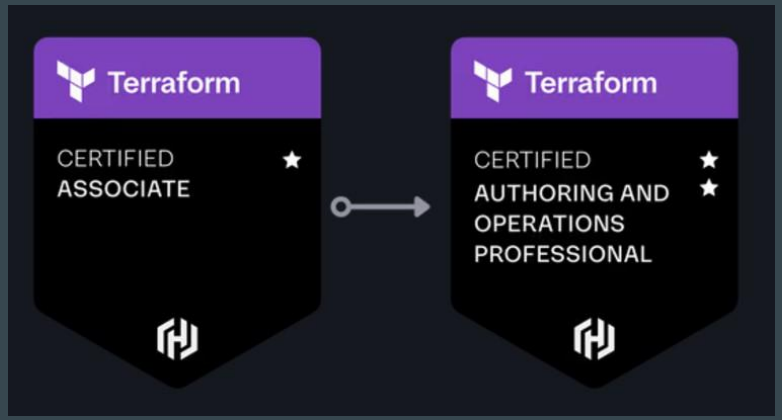
What are Providers in Terraform?
Option A
Option B
Option C



Sample Question

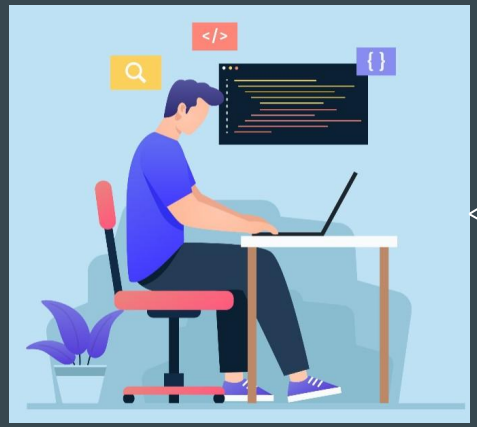
Introducing Terraform Professional Certification

Terraform Authoring and Operations Professional exam assesses both advanced configuration authoring and a deep understanding of Terraform workflows.



Lab Based Exams

Terraform Professional exam is a 4-hour intensive lab-based exam where you will have to solve multiple scenarios presented to you.



Challenge 1

Challenge 2

Challenge 3

Challenge 4

Our Terraform Professional Course

We are the first one to launch dedicated course on Terraform Professional certification exam.

This course covers many advance concepts in Terraform.



Access Terraform Professional Course

https://kplabs.in/tfpro

Join us in our Adventure

Be Awesome



kplabs.in/chat



kplabs.in/linkedin